

MODUL PRAKTIK/PRAKTIKUM

IF330 – PEMROGRAMAN WEB
PROGRAM SARJANA S1 INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA

Date Issued :



PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA

Gedung A Lantai 5, Kampus UMN

Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia

Telp: +62-21.5422.0808 web: www.umn.ac.id

DAFTAR ISI

CAPAIAN PEMBELAJARAN PRAKTIKUM	5
MINGGU 1	
PERKENALAN DENGAN SERVER-SIDE PROGRAMMING MENGUNAKAN BAHASA SCRIPTING PHP	6
MINGGU 2	
DASAR-DASAR PEMROGRAMAN PHP	10
MINGGU 3	
PHP FORMS, SESSION, COOKIE	13
MINGGU 4	
PHP OBJECT ORIENTED PROGRAMMING (OOP)	21
MINGGU 5	
UPLOAD DAN MANIPULASI FILE	27
MINGGU 6	
PHP & MYSQL	34
MINGGU 7	
PHP SECURITY & AUTHENTICATION	43
MINGGU 8	
DASAR-DASAR FRAMEWORK LARAVEL	49
MINGGU 9	
CONTROLLER & VIEW	54
MINGGU 10	
MODEL & MIGRATION	66
MINGGU 11	
ELOQUENT RELATIONSHIP	98
MINGGU 12	
FORMS & VALIDATION	107
MINGGU 13	
SECURITY & AUTHENTICATION	122

1. Mahasiswa mampu menjelaskan perbedaan client-side dan server-side scripting serta teknologi-teknologi yang dapat digunakan (C2)
2. Mahasiswa mampu menjelaskan dasar-dasar pemrograman web dan menggunakan struktur control dengan Bahasa pemrograman PHP (C2, C3)
3. Mahasiswa mampu menggunakan pre-defined variables dan functions pada PHP dan menggunakan fungsionalitas PHP untuk memproses data dari web form (C3)
4. Mahasiswa mampu menerapkan konsep pemrograman berbasis obyek pada PHP (C3)
5. Mahasiswa mampu merancang dan membangun aplikasi web untuk melakukan upload dan manipulasi file menggunakan PHP (C5, C6)
6. Mahasiswa mampu merancang dan membangun aplikasi web yang terkoneksi dengan basis data MySQL dan melakukan proses CRUD pada basis data tersebut (C5, C6)
7. Mahasiswa mampu merancang dan membangun aplikasi web yang aman dengan fitur autentikasi pengguna (C5, C6)
8. Mahasiswa mampu menjelaskan dasar-dasar pengembangan web menggunakan Laravel Framework (C2)
9. Mahasiswa mampu menerapkan Controller dan View menggunakan Framework Laravel (C3)
10. Mahasiswa mampu menerapkan Model dan Migration menggunakan Framework Laravel (C3)
11. Mahasiswa mampu menerapkan Eloquent Relationship menggunakan Framework Laravel (C3)
12. Mahasiswa mampu menerapkan form, upload file dan validasi menggunakan Framework Laravel (C3)
13. Mahasiswa mampu merancang dan membangun aplikasi web yang aman menggunakan Framework Laravel (C5, C6)
14. Mahasiswa mampu merancang dan membangun RESTful API menggunakan Framework Laravel (C5, C6)

MINGGU 1

PERKENALAN DENGAN SERVER-SIDE PROGRAMMING MENGUNAKAN BAHASA SCRIPTING PHP



DESKRIPSI TEMA

Mahasiswa mempersiapkan development environment untuk melakukan pengembangan aplikasi berbasis web, yaitu melakukan instalasi XAMPP sebagai local web server, Visual Studio Code sebagai code editor dan Postman untuk melakukan pengujian API.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menjelaskan perbedaan client-side dan server-side scripting serta teknologi-teknologi yang dapat digunakan (C2)

PERALATAN YANG DIGUNAKAN

1. XAMPP
2. VS Code
3. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

PHP merupakan salah satu Bahasa scripting sisi server yang paling populer saat ini. Kode PHP dapat berdiri sendiri atau disisipkan pada kode HTML. Kode PHP diproses oleh sebuah interpreter PHP yang didukung oleh Zend Engine.

Fakta Seputar PHP:

- PHP awalnya merupakan singkatan dari “Personal Home Page”, yang sekarang dikenal sebagai “Hypertext Preprocessor” dirilis pada tahun 1995 oleh Rasmus Lerdorf.
- PHP merupakan Free Open Source Software (FOSS) yang dirilis dibawah lisensi PHP.
- PHP dapat digunakan di semua sistem operasi populer, yaitu Linux, Microsoft Windows dan MacOS.
- Beberapa website terkenal yang dikembangkan menggunakan PHP, yaitu Facebook, Digg, Flickr, Technorati, Yahoo, dll.

PHP Development Tools

Untuk mengembangkan aplikasi dengan PHP, dibutuhkan beberapa tools, yaitu:

- Apache Server – web server yang mengirimkan konten menggunakan protocol HTTP atau HTTPS melalui World Wide Web (WWW)
- MySQL/MariaDB – Database untuk menyimpan data
- PHP – Interpreter untuk membaca kode PHP

Terdapat variasi software bundle untuk mempermudah developer melakukan setup atau instalasi awal, antara lain:

- LAMP – Linux, Apache, MySQL dan PHP
- WAMP – Windows, Apache, MySQL dan PHP
- MAMP – MacOS, Apache, MySQL dan PHP
- XAMPP – Cross-Platform (X), Apache, MariaDB dan PHP

Instalasi:

- XAMPP: <https://www.apachefriends.org/download.html>
- Visual Studio Code: <https://code.visualstudio.com/>

- Postman: <https://www.postman.com/>

Kustomisasi Instalasi PHP

Konfigurasi utama dari PHP terdapat pada file `php.ini` yang biasanya dapat ditemukan pada direktori XAMPP seperti: `C:\xampp\php\php.ini`. Beberapa konfigurasi utama yang tersedia, antara lain: `max_execution_time`, `max_input_time`, `memory_limit`, `file_uploads`, `uploads_max_filesize`, dll.

Sintaks PHP

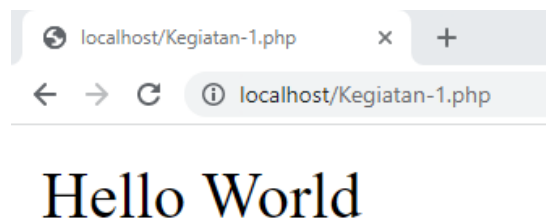
- Kode PHP selalu dibungkus menggunakan tag `<?php` dan `?>`
- Setiap baris kode diakhiri dengan titik-koma (`;`)
- Untuk menulis komentar, menggunakan double-slash (`//`) atau `/*` dan `*/`

Hello World

Buatlah sebuah file dengan nama `"index.php"`, simpan pada folder `"htdocs"`, dan ketikkan kode berikut:

```
<?php  
echo "Hello World";
```

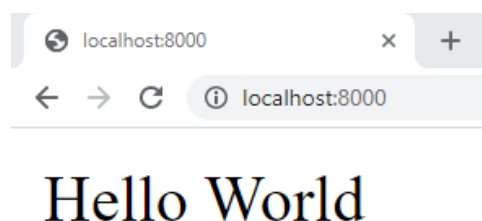
Setelah menjalankan servis Apache, arahkan browser URL ke <http://localhost/path/index.php> dan lihat hasilnya.



Selain menggunakan Apache Service melalui XAMPP, local webserver juga bisa diaktifkan menggunakan PHP CLI. Ketikkan command berikut melalui terminal, pada path dimana file `index.php` berada.

```
Week-01> php -S localhost:8000
```

Kemudian arahkan URL pada web browser ke <http://localhost:8000> dan lihat hasilnya.



HTML + PHP

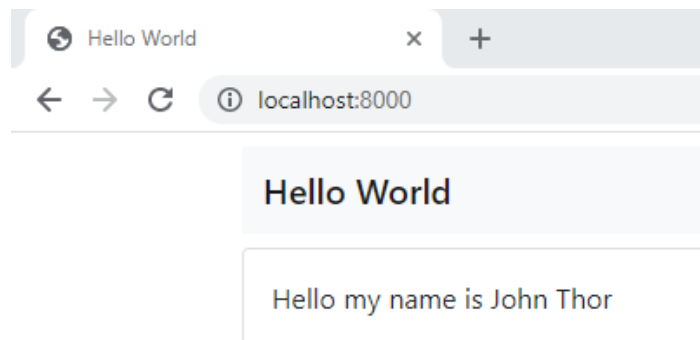
PHP juga dapat disisipkan kedalam kode HTML, ubah kode pada file `index.php` menjadi seperti berikut:

```

1  <?php
2  $nama = "John Thor";
3  ?>
4  <!doctype html>
5  <html>
6  <head>
7      <title>Hello World</title>
8      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
9          rel="stylesheet" />
10 </head>
11 <body>
12     <div class="container">
13         <nav class="navbar navbar-light bg-light m-2">
14             <div class="container">
15                 <span class="navbar-brand mb-0 h1">Hello World</span>
16             </div>
17         </nav>
18         <div class="card m-2">
19             <div class="card-body">
20                 Hello my name is <?php echo $nama ?>
21             </div>
22         </div>
23     </div>
24 </body>
25 </html>

```

Baris 1-3 dan baris ke-20 merupakan kode PHP. Kode diatas jika dibuka pada web browser akan tampil seperti berikut. Baris ke-2 dari kode diatas merupakan deklarasi variable `$nama`, dan pada baris ke-20 isi dari variable tersebut dicetak.



Include & Require

Pada PHP terdapat function include dan require yang dapat digunakan untuk menghubungkan dua buah file PHP. Perbedaan antara function include dan require adalah saat file yang akan digunakan tidak tersedia, maka jika menggunakan function include maka program akan berjalan terus, sedangkan jika menggunakan function require maka program akan terhenti jika file yang akan digunakan tidak tersedia.

Buat file navbar.php dan data.php, simpan pada direktori yang sama dengan index.php. Potong kode dari index.php, sehingga menjadi seperti berikut.

```

1  <?php
2      include("data.php");
3  ?>
4  <!doctype html>
5  <html>
6      <head>
7          <title>Hello World</title>
8          <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
9              rel="stylesheet" />
10     </head>
11     <body>
12         <div class="container">
13             <?php include("navbar.php"); ?>
14             <div class="card m-2">
15                 <div class="card-body">
16                     Hello my name is <?php echo $nama ?>
17                 </div>
18             </div>
19         </div>
20     </body>
21 </html>

```

index.php

```

1  <nav class="navbar navbar-light bg-light m-2">
2      <div class="container">
3          <span class="navbar-brand mb-0 h1">Hello World</span>
4      </div>
5  </nav>

```

navbar.php

```

1  <?php
2  $nama = "John Thor";

```

data.php

Ketika dibuka di browser, maka akan menampilkan halaman web yang sama seperti sebelumnya. Selanjutnya coba ganti nama file pada baris ke-2 dan ke-13, sehingga nama file tidak sesuai (misal navbar1.php dan data1.php), kemudian jalankan Kembali di browser, kode php akan tetap di proses dan halaman web ditampilkan meskipun tidak lengkap.

TUGAS

1. Ganti function include menjadi require, jelaskan apa yang terjadi.
2. Pelajari dan jelaskan perbedaan antara function include() dan include_once() serta require() dan require_once().

MINGGU 2

DASAR-DASAR PEMROGRAMAN PHP



DESKRIPSI TEMA

Mahasiswa mempraktikkan dasar-dasar pemrograman dengan PHP, yaitu variable, array, function dan struktur kontrol.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menjelaskan dasar-dasar pemrograman web dan menggunakan struktur control dengan Bahasa pemrograman PHP

PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor (VS Code, PHP Storm, etc)
3. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

1. **[TUGAS 1]** Buatlah sebuah **array dua dimensi** bernama **negara**. Dimensi pertama terdiri dari **nama negara**, dan dimensi kedua adalah **associative arrays** (menggunakan string sebagai indeksnya) yang berisi atribut :
 - a. **ibuKota**,
 - b. **kodeTel** dan
 - c. **mataUang**

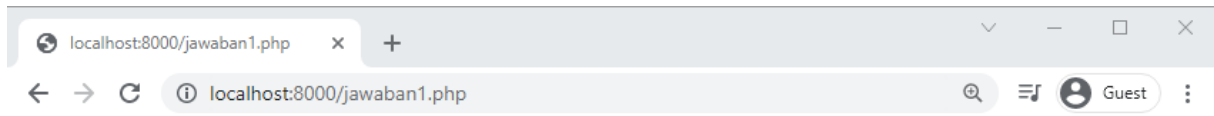
Dengan data sebagai berikut:

Nama Negara	Ibu Kota	Kode Telepon	Mata Uang
India	New Delhi	91	INR
Indonesia	Jakarta	62	IDR
Japan	Tokyo	81	JPY

Panduan cara membuat array dua dimensi:

```
<?php
$arr2dim = array(
    "namaNegara1" => array(
        "atribut1" => "aaa",
        "atribut2" => 1
    ),
    "namaNegara2" => array(
        "atribut1" => "bbb",
        "atribut2" => 2
    )
);
?>
```

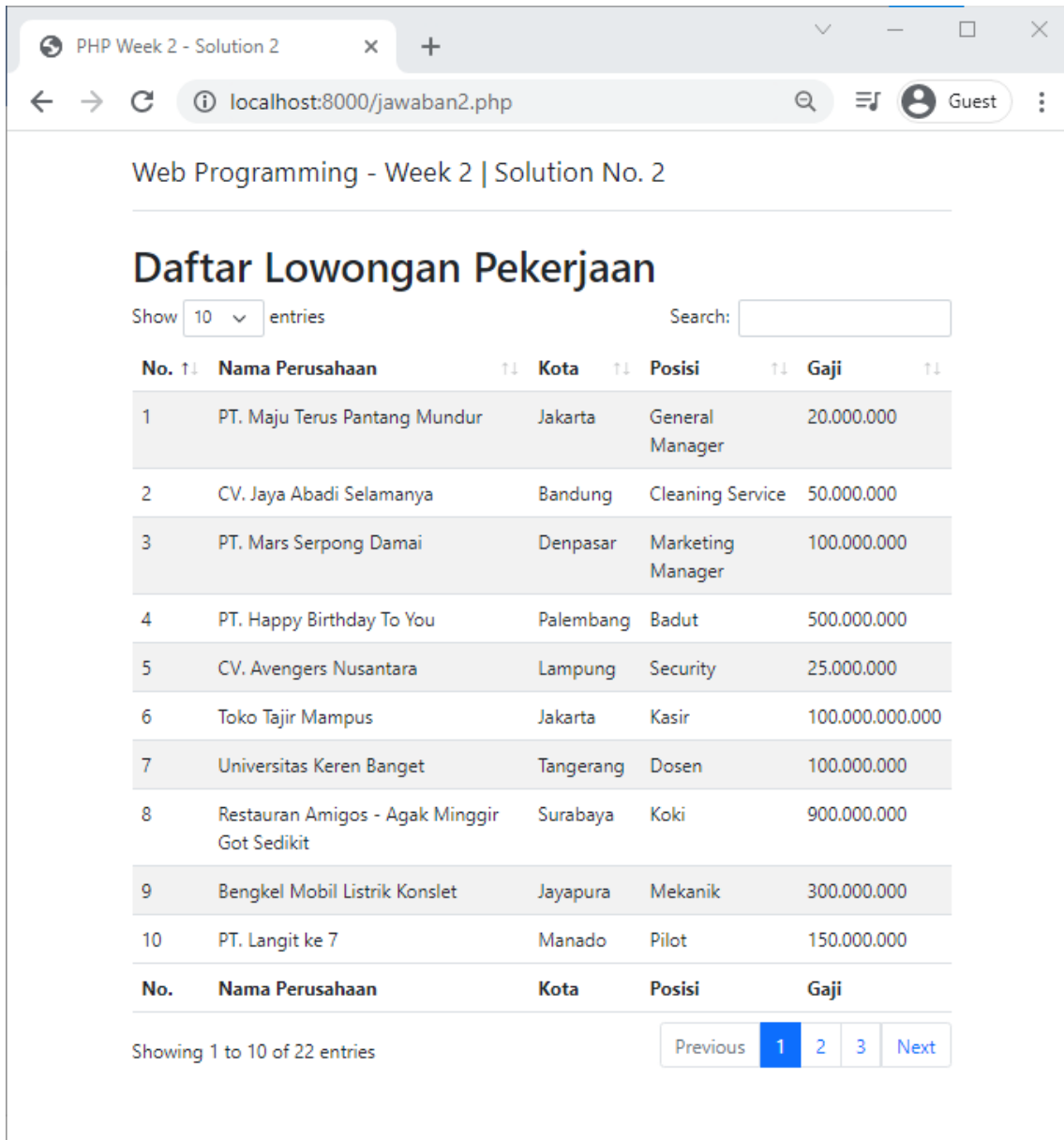
Kemudian tampilkan isi array seperti berikut (gunakan foreach/for):



New Delhi is capital city of **India**. India's calling code is 91 and has "INR" as currency code.
Jakarta is capital city of **Indonesia**. Indonesia's calling code is 62 and has "IDR" as currency code.
Tokyo is capital city of **Japan**. Japan's calling code is 81 and has "JPY" as currency code.

Simpan dalam file **jawaban1.php**.

2. **[TUGAS 2]** Buatlah website dengan tampilan sebagai berikut:

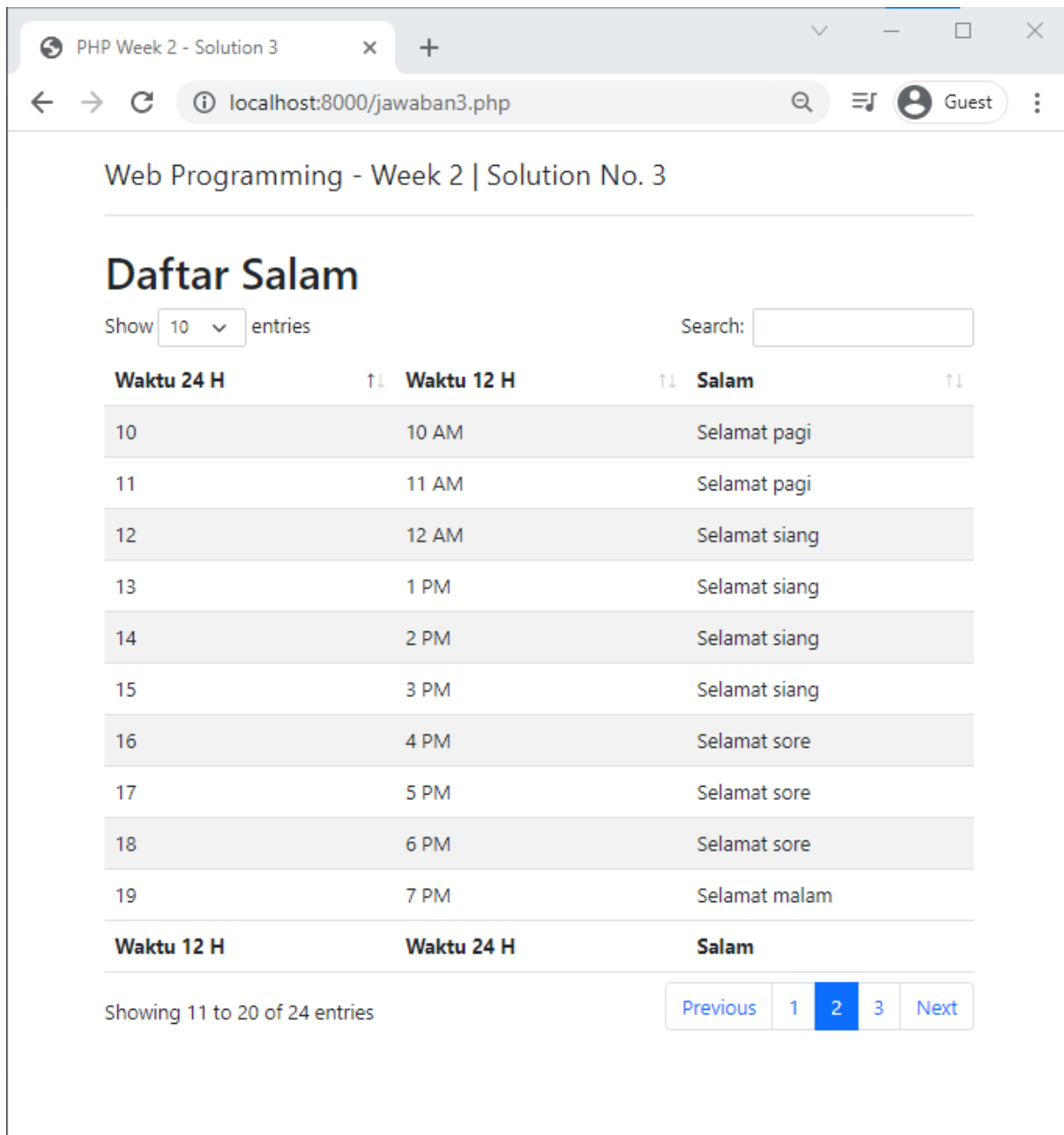


Ketentuan: masing-masing data dibuat dengan **array 2 dimensi**, dengan struktur array/penamaan bebas.

Dengan menggunakan **foreach/for**, tampilkan seluruh data (data bebas) dalam Bootstrap's Data Table (<https://datatables.net/manual/styling/bootstrap5>).

Simpan dalam file **jawaban2.php**.

3. **[TUGAS 3]** Buatlah halaman web yang menampilkan daftar waktu dalam format 24 jam dan 12 jam seperti berikut:



Web Programming - Week 2 | Solution No. 3

Daftar Salam

Show entries Search:

Waktu 24 H	Waktu 12 H	Salam
10	10 AM	Selamat pagi
11	11 AM	Selamat pagi
12	12 AM	Selamat siang
13	1 PM	Selamat siang
14	2 PM	Selamat siang
15	3 PM	Selamat siang
16	4 PM	Selamat sore
17	5 PM	Selamat sore
18	6 PM	Selamat sore
19	7 PM	Selamat malam

Showing 11 to 20 of 24 entries

Previous 1 2 3 Next

Ketentuan: Buat menggunakan **perulangan sebanyak 24 kali** untuk menampilkan jam dalam format 24 jam, dan buat **function** untuk menampilkan **format waktu 12 jam** dan **salam**.

Berikut adalah contoh potongan kode untuk menampilkan baris data:

```
<tr>
    <td><?= $i ?></td>
    <td><?= time12format($i) ?></td>
    <td><?= greetings($i) ?></td>
</tr>
```

MINGGU 3

PHP FORMS, SESSION, COOKIE

DESKRIPSI TEMA

Mahasiswa mempraktikkan bagaimana melakukan pemrosesan data dari formulir menggunakan PHP, menggunakan Session, dan melakukan penyimpanan data pada Cookie.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menggunakan pre-defined variables dan functions pada PHP dan menggunakan fungsionalitas PHP untuk memproses data dari web form

PERALATAN YANG DIGUNAKAN

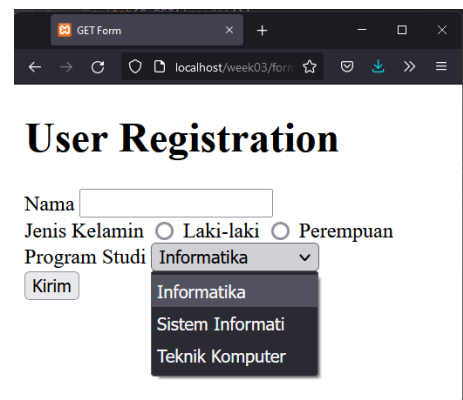
1. XAMPP
2. Code Editor (VS Code, PHP Storm, etc)
3. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

Proses PHP Form

1. Buatlah sebuah file "form_get.php", simpan dalam folder "Week-03 NIM Nama", kemudian buatlah sebuah form seperti gambar berikut menggunakan HTML:

```
<h1>User Registration</h1>
<form action="proses_get.php" method="get">
  <label>Nama</label>
  <input type="text" name="nama" />
  <br />
  <label>Jenis Kelamin</label>
  <input type="radio" name="gender" value="m" /> Laki-laki
  <input type="radio" name="gender" value="f" /> Perempuan
  <br />
  <label>Program Studi</label>
  <select name="prodi">
    <option value="if">Informatika</option>
    <option value="si">Sistem Informatika</option>
    <option value="tk">Teknik Komputer</option>
  </select>
  <br />
  <button type="submit">Kirim</button>
</form>
```

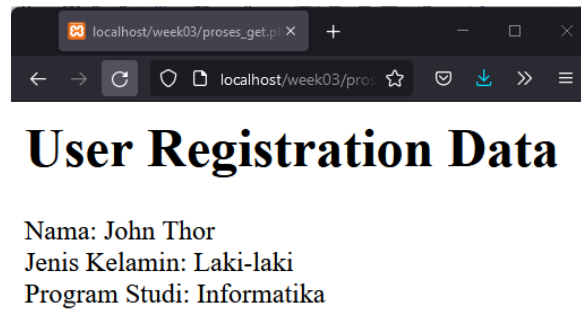


2. Buatlah sebuah file "proses_get.php", kemudian tuliskan kode berikut:

```
<?php
switch($_GET['prodi']){
    case "if": $prodi = "Informatika"; break;
    case "si": $prodi = "Sistem Informatika"; break;
    case "tk": $prodi = "Teknik Komputer"; break;
}

echo "<h1>User Registration Data</h1>";
echo "Nama: " . $_GET['nama'] . "<br />";
echo "Jenis Kelamin: " . ($_GET['gender']=="m" ? "Laki-laki":"Perempuan") . "<br />";
echo "Program Studi: " . $prodi . "<br />";
```

3. Coba lakukan submit data pada form tersebut, maka akan muncul tampilan seperti gambar berikut:



4. Kerjakan Tugas 1

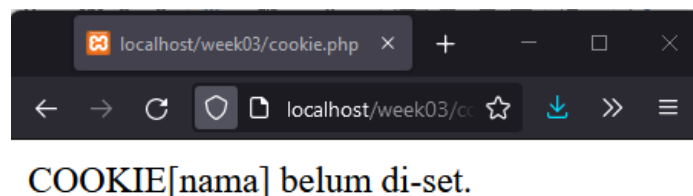
Cookie Sederhana

1. Buatlah sebuah file "cookie.php", kemudian tuliskan kode berikut:

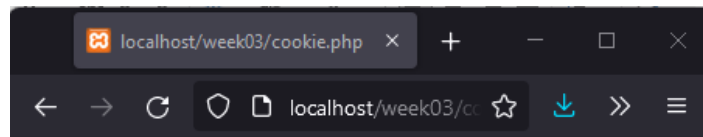
```
<?php
setcookie('nama', 'John Thor', time() + (86400 * 30), "/"); // 86400 = 1 day

if(!isset($_COOKIE['nama'])) {
    echo "COOKIE[nama] belum di-set.";
} else {
    echo "COOKIE[nama] sudah di-set.<br />";
    echo "COOKIE[nama]: " . $_COOKIE['nama'];
}
```

2. Coba buka melalui browser, maka akan muncul tampilan seperti berikut:



3. Coba refresh web browser, maka akan muncul tampilan seperti berikut:



COOKIE[nama] sudah di-set.
COOKIE[nama]: John Thor

Menyimpan data JSON pada Cookie

1. Buat sebuah file "cookie_set.php", kemudian tuliskan kode berikut:

```
<?php
if(!isset($_COOKIE['index_cookie'])){
    setcookie('index_cookie', 0);
    $index = 0;

    $data1 = array(
        "nama" => "John Thor",
        "prodi" => "Informatika",
        "info" => "First data"
    );

    setcookie("data[$index]", json_encode($data1));
    echo "log0";
}else{
    setcookie('index_cookie', $_COOKIE['index_cookie'] + 1);
    $index = $_COOKIE['index_cookie'] + 1;

    $data2 = array(
        "nama" => "John Wick",
        "prodi" => "Sistem Informasi",
        "info" => "Next data"
    );

    setcookie("data[$index]", json_encode($data2));
    echo "log1";
}
```

2. Buat file "cookie_read.php", kemudian tuliskan kode berikut:

```
<?php
echo "<pre>";

echo 'All $_COOKIE content: <br />';
print_r($_COOKIE);
echo '<br />';

echo 'All $_COOKIE[\'data\'] content: <br />';
var_dump($_COOKIE['data']);
echo '<br />';

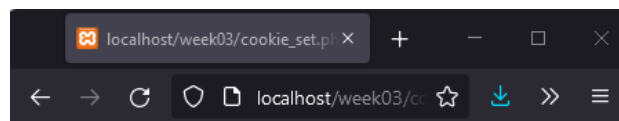
echo '$_COOKIE[\'data\'][0] content: <br />';
var_dump($_COOKIE['data'][0]);
echo '<br />';

$data_as_array = json_decode($_COOKIE['data'][0], associative: true);
echo $data_as_array['nama'], "<br />";
echo $data_as_array['prodi'], "<br />";
echo $data_as_array['info'], "<br /><br />";

$data_as_array = json_decode($_COOKIE['data'][1], associative: true);
echo $data_as_array['nama'], "<br />";
echo $data_as_array['prodi'], "<br />";
echo $data_as_array['info'], "<br /><br />";

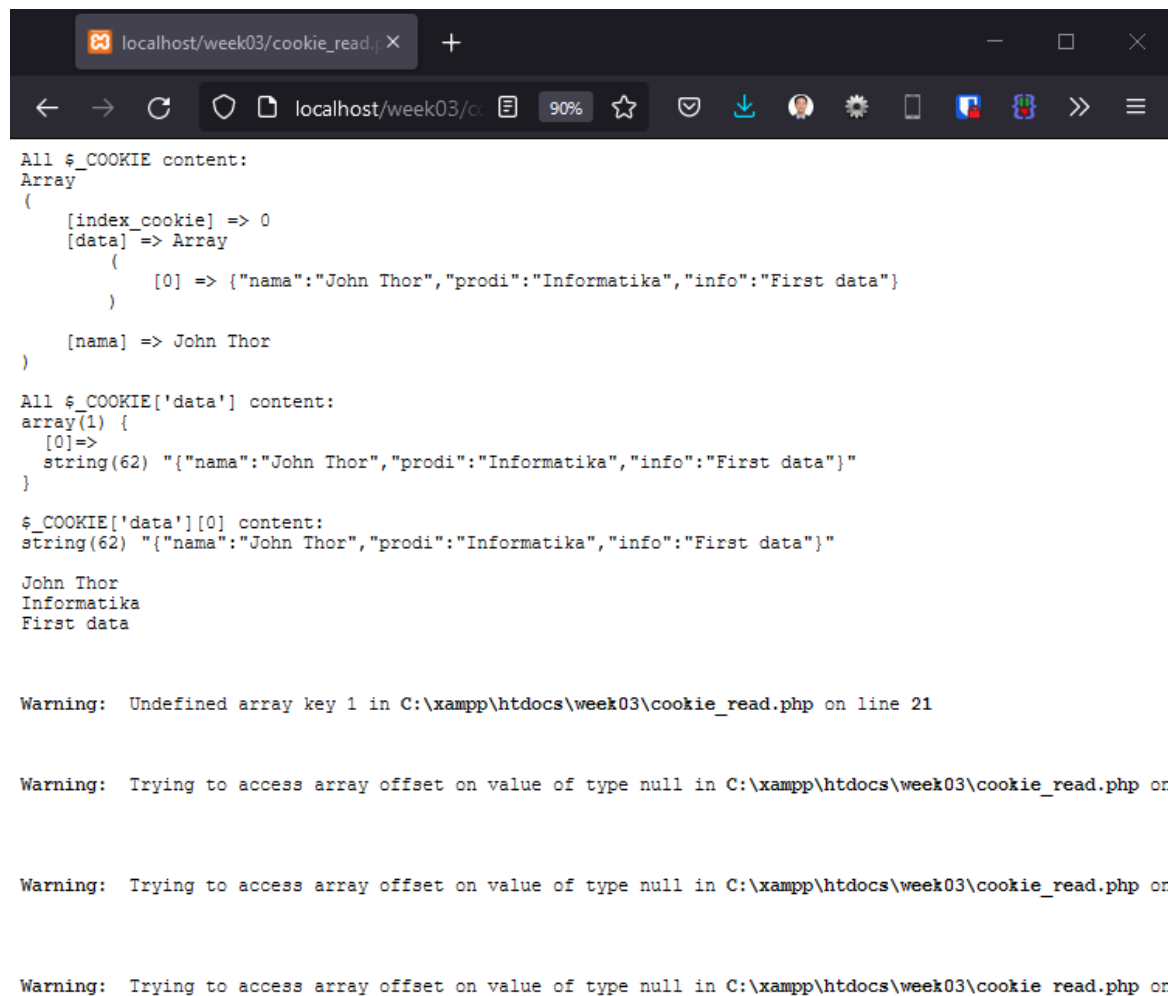
echo "</pre>";
```

3. Jalankan "cookie_set.php" pada web browser, maka akan muncul tampilan berikut:



log0

4. Jalankan "cookie_read.php" pada web browser, maka akan muncul tampilan berikut:



```

All $_COOKIE content:
Array
(
    [index_cookie] => 0
    [data] => Array
        (
            [0] => {"nama":"John Thor","prodi":"Informatika","info":"First data"}
        )
    [nama] => John Thor
)

All $_COOKIE['data'] content:
array(1) {
    [0] =>
        string(62) '{"nama":"John Thor","prodi":"Informatika","info":"First data"}'
}

$_COOKIE['data'][0] content:
string(62) '{"nama":"John Thor","prodi":"Informatika","info":"First data"}'

John Thor
Informatika
First data

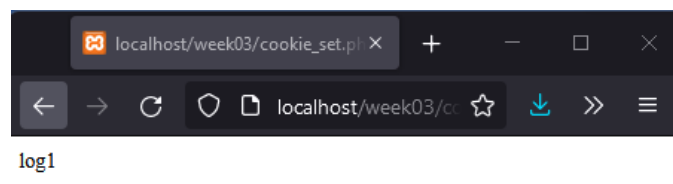
Warning: Undefined array key 1 in C:\xampp\htdocs\week03\cookie_read.php on line 21

Warning: Trying to access array offset on value of type null in C:\xampp\htdocs\week03\cookie_read.php on

Warning: Trying to access array offset on value of type null in C:\xampp\htdocs\week03\cookie_read.php on

Warning: Trying to access array offset on value of type null in C:\xampp\htdocs\week03\cookie_read.php on
  
```

5. Jalankan Kembali file “cookie_set.php” di browser.



```

log1
  
```

6. Jalankan Kembali file “cookie_read.php” melalui web browser, maka akan muncul tampilan seperti berikut:

```

localhost/week03/cookie_read . X +
localhost/week03/cookie_re. E ☆
All $_COOKIE content:
Array
(
    [index_cookie] => 1
    [data] => Array
        (
            [0] => {"nama":"John Thor","prodi":"Informatika","info":"First data"}
            [1] => {"nama":"John Wick","prodi":"Sistem Informasi","info":"Next data"}
        )
    [nama] => John Thor
)

All $_COOKIE['data'] content:
array(2) {
    [0] =>
        string(62) "{"nama":"John Thor","prodi":"Informatika","info":"First data"}"
    [1] =>
        string(66) "{"nama":"John Wick","prodi":"Sistem Informasi","info":"Next data"}"
}

$_COOKIE['data'][0] content:
string(62) "{"nama":"John Thor","prodi":"Informatika","info":"First data"}"

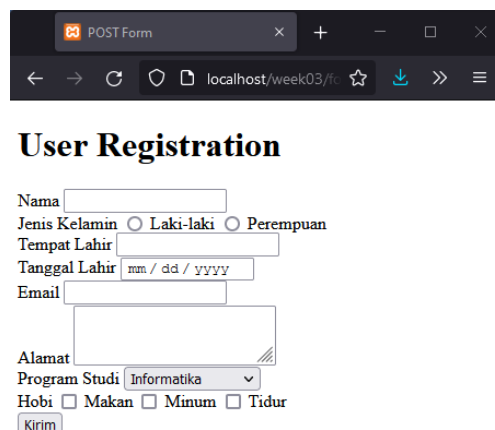
John Thor
Informatika
First data

John Wick
Sistem Informasi
Next data

```

TUGAS

1. Buat file "form_post.php" kemudian buatlah form menggunakan HTML dengan tampilan seperti berikut:



User Registration

Nama

Jenis Kelamin ☐ Laki-laki ☐ Perempuan

Tempat Lahir

Tanggal Lahir

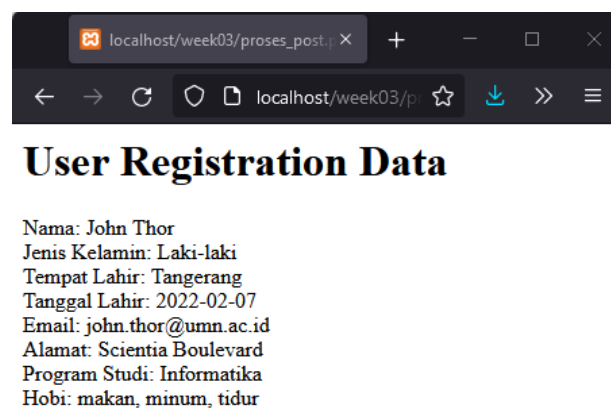
Email

Alamat

Program Studi

Hobi ☐ Makan ☐ Minum ☐ Tidur

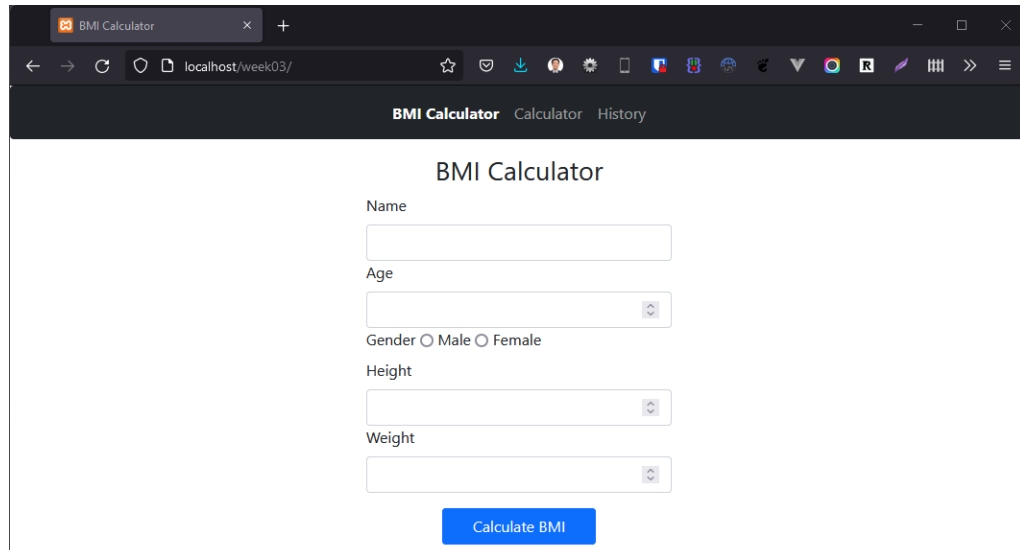
Kemudian buat file "proses_post.php" yang akan menerima data dari form_post dan menampilkannya seperti gambar berikut:



User Registration Data

Nama: John Thor
 Jenis Kelamin: Laki-laki
 Tempat Lahir: Tangerang
 Tanggal Lahir: 2022-02-07
 Email: john.thor@umn.ac.id
 Alamat: Scientia Boulevard
 Program Studi: Informatika
 Hobi: makan, minum, tidur

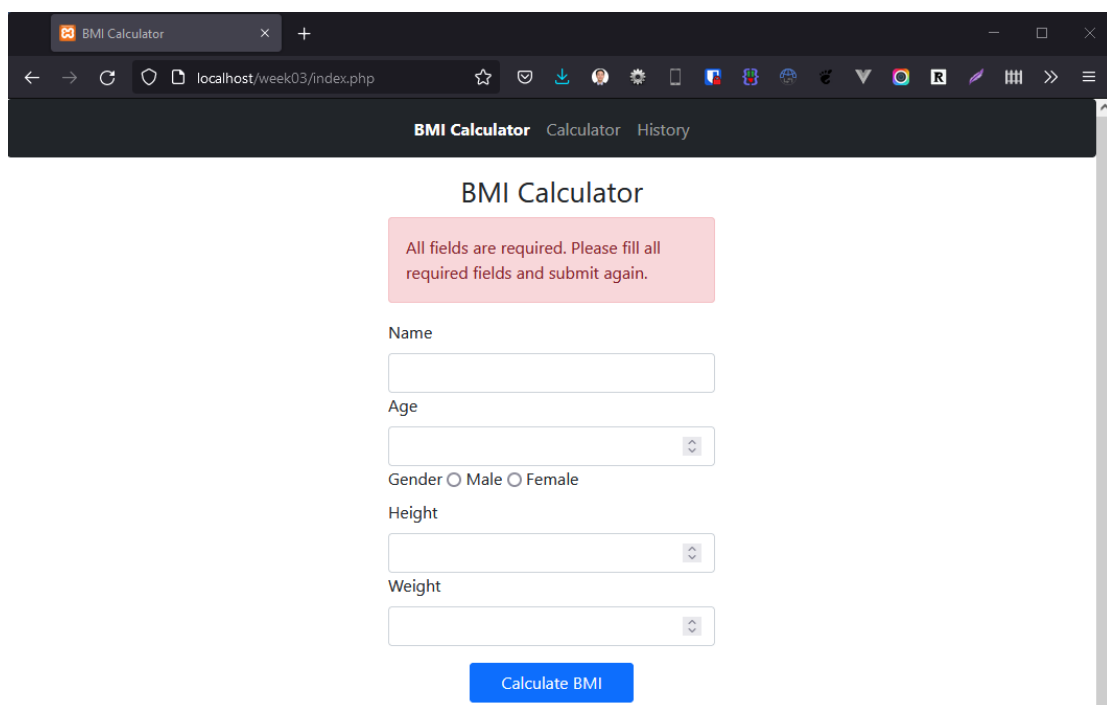
2. Buatlah sebuah file "index.php" untuk menampilkan formulir seperti gambar berikut, untuk melakukan perhitungan BMI:



The screenshot shows a web browser window with the address bar at localhost/week03/. The page title is "BMI Calculator". The form contains the following elements:

- Name:** A text input field.
- Age:** A dropdown menu.
- Gender:** Radio buttons for "Male" and "Female".
- Height:** A dropdown menu.
- Weight:** A dropdown menu.
- Calculate BMI:** A blue button.

Ketika tombol Calculate BMI di-click, maka data akan dikirimkan ke "proses_bmi.php" yang akan melakukan validasi, apakah semua field sudah diisi, jika belum, maka akan muncul pesan seperti berikut:

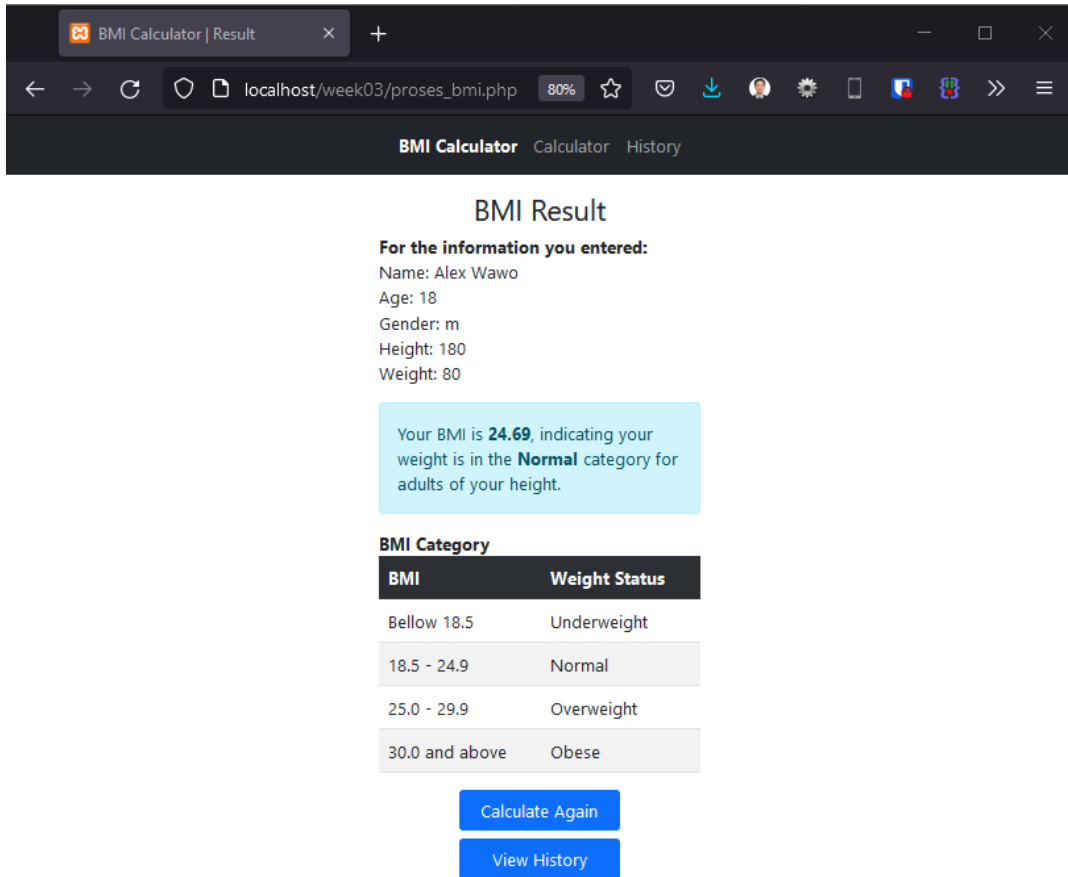


The screenshot shows the same BMI Calculator form, but with a red error message box at the top: "All fields are required. Please fill all required fields and submit again." The form fields and the "Calculate BMI" button are still visible below the message.

Jika pengisian formulir lengkap dan tombol Calculate BMI di-click, maka data akan diproses pada proses_bmi.php dengan rumus perhitungan BMI:

$$\text{BMI} = \text{berat badan} / (\text{tinggi badan} * \text{tinggi badan})$$

Hasil perhitungan BMI akan ditampilkan seperti gambar berikut:



BMI Calculator | Result

← → ↻ 🛡️ 📄 localhost/week03/proses_bmi.php 80% ☆ 📧 ⬇️ 👤 ⚙️ 📱 🌐 🏠 >> ☰

BMI Calculator Calculator History

BMI Result

For the information you entered:
 Name: Alex Wawo
 Age: 18
 Gender: m
 Height: 180
 Weight: 80

Your BMI is **24.69**, indicating your weight is in the **Normal** category for adults of your height.

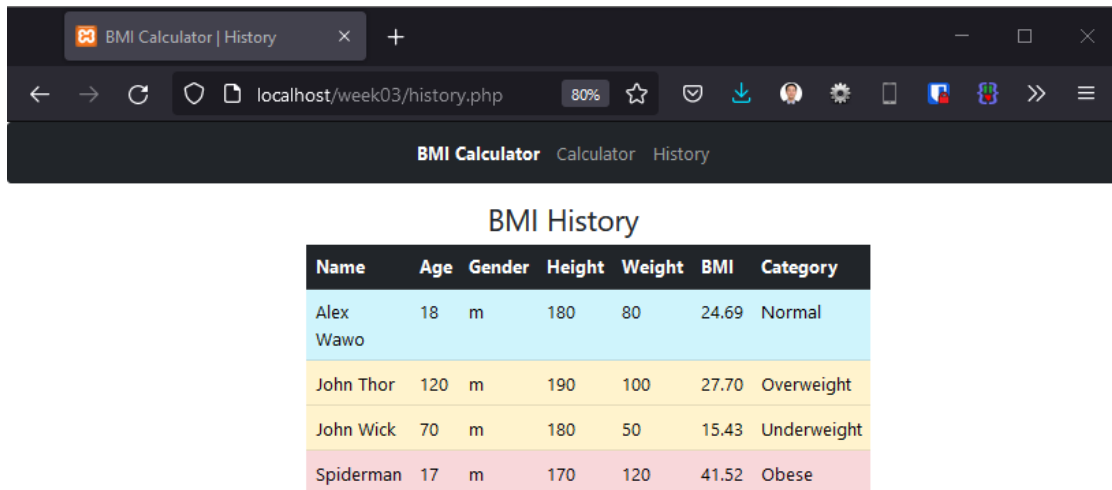
BMI Category

BMI	Weight Status
Bellow 18.5	Underweight
18.5 - 24.9	Normal
25.0 - 29.9	Overweight
30.0 and above	Obese

[Calculate Again](#)

[View History](#)

Setiap kali melakukan perhitungan BMI, data akan disimpan menggunakan Cookie, dan dapat dilihat pada menu History.



BMI Calculator | History

← → ↻ 🛡️ 📄 localhost/week03/history.php 80% ☆ 📧 ⬇️ 👤 ⚙️ 📱 🌐 🏠 >> ☰

BMI Calculator Calculator History

BMI History

Name	Age	Gender	Height	Weight	BMI	Category
Alex Wawo	18	m	180	80	24.69	Normal
John Thor	120	m	190	100	27.70	Overweight
John Wick	70	m	180	50	15.43	Underweight
Spiderman	17	m	170	120	41.52	Obese

Catatan: Perhatikan halaman BMI Result dan BMI History, terdapat warna yang ditentukan berdasarkan hasil perhitungan BMI.

Hijau – Normal, Kuning – Underweight / Overweight, Merah – Obese.

MINGGU 4

PHP OBJECT ORIENTED PROGRAMMING (OOP)

DESKRIPSI TEMA

Mahasiswa mempraktikkan bagaimana menerapkan konsep pemrograman berorientasi obyek menggunakan PHP

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menerapkan konsep pemrograman berbasis obyek pada PHP

PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor (VS Code, PHP Storm, etc.)
3. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

Buat sebuah folder Week04_NIM_Nama, simpan semua file yang akan dibuat pada praktikum ini pada folder tersebut.

Tugas 01: Class & Object

Buat sebuah file Tugas1.php, kemudian tuliskan kode berikut:

```
<?php
class Mobil {
    public $nama;
    public $merk;

    function getInfo() {
        echo "Nama Mobil: " . $this->nama . "<br />";
        echo "Merk Mobil: " . $this->merk;
    }
}

$avanza = new Mobil();
$avanza->nama = "Avanza Veloz";
$avanza->merk = "Toyota";
$avanza->getInfo();
```

Modifikasi Class Mobil, tambahkan constructor untuk menge-set property nama dan merk pada saat pembuatan object, dan tambahkan destructor yang akan menampilkan informasi produk. Berikut adalah hasil dari perubahan kode yang diharapkan.

```
1      <?php
2      class Mobil {...}
20     new Mobil('Avanza Veloz', 'Toyota');
```

Tugas 02: Access Modifier

Buat file Tugas2.php, kemudian tuliskan kode berikut

```
<?php
class Mahasiswa {
    public $nim;
    public $nama;
}

$john = new Mahasiswa();
$john->nim = '001';
$john->nama = 'John Thor';

echo $john->nim . " " . $john->nama;
```

Ubah access modifier dari property nim dan nama menjadi private. Jika program dijalankan maka akan mengalami error. Modifikasi program sehingga dapat berjalan seperti semula dengan tetap menggunakan access modifier private untuk nim dan nama (Tidak boleh menggunakan constructor)

Tugas 03: Inheritance

Buat file Tugas3.php kemudian tuliskan kode berikut:

```
<?php
class Manusia {
    protected $nama;

    function __construct($nama){
        $this->nama = $nama;
    }

    final public function get_nama(){
        return $this->nama;
    }
}

//...
$john = new Manusia( nama: 'John Thor');
$wick = new Bayi('John Wick', ' 3kg');

echo "Data 1 <br />Nama: " . $john->get_nama() . "<br />";
echo "<hr />";
echo "Data 2 <br />Nama: " . $wick->get_nama() . "<br />";
echo "Berat: " . $wick->get_berat_lahir();
```

Buat class Bayi yang merupakan turunan dari class Manusia, sehingga program menghasilkan output:

Data 1

Nama: John Thor

Data 2

Nama: Jowhn Wick

Bera: 3kg

Tugas 04: Abstract Class & Constant

Buat file Tugas4.php, kemudian tuliskan kode berikut:

```
<?php
abstract class Mahasiswa {
    abstract protected function getTugasAkhir();
    abstract protected function getProgram($postfix);

    public function tugasAkhir(){
        echo $this->getTugasAkhir() . "<br />";
    }
}

class Sarjana extends Mahasiswa {
    protected function getTugasAkhir(){
        return "Skripsi";
    }

    public function getProgram($postfix){
        echo "{$postfix} S1";
    }
}
```

```
echo Mahasiswa::KAMPUS . "<br />";
$s = new Sarjana();
$s->getProgram('Mahasiswa') . '<br />';
$s->tugasAkhir();

$m = new Magister();
$m->getProgram('Mahasiswa') . '<br />';
$m->tugasAkhir();
```

Lengkapi kode tersebut sehingga menghasilkan output seperti berikut:

Universitas Multimedia Nusantara

Mahasiswa S1Skripsi

Mahasiswa S2Tesis

Tugas 05: Interface

Buat file Tugas5.php, tuliskan kode berikut dan lengkapi kode pada Class KacangPolong sehingga program tidak error.

```
<?php
interface Buah {
    public function setJumlahBiji();
}

interface Sayur {
    public function setWarna();
}

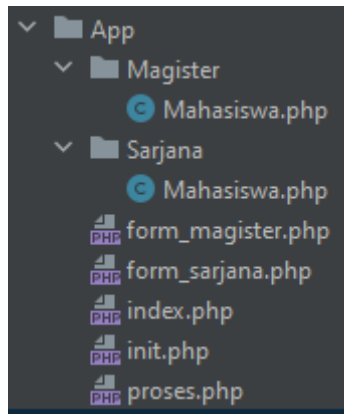
interface SayurBuah extends Sayur, Buah {
    public function setCaraMakan();
}

class KacangPolong implements SayurBuah {
    function __destruct(){
        echo "Kacang Polong";
    }
}

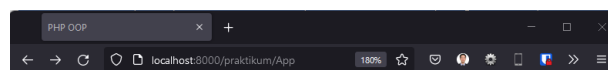
new KacangPolong();
```

Tugas 06: Namespace & Autoloading

1. Buatlah sebuah folder App beserta dengan file-file dengan struktur seperti berikut:



2. Buatlah class Mahasiswa pada file Magister/Mahasiswa.php dan Sarjana/Mahasiswa.php.
3. Class Sarjana/Mahasiswa memiliki property NIM, Nama, dan Prodi, sedangkan class Magister/Mahasiswa memiliki property NIM, Nama, Prodi dan GelarSarjana.
4. Lengkapi kedua class Mahasiswa dengan constructor dan method getter.
5. Pada file index.php, buat tampilan seperti berikut:



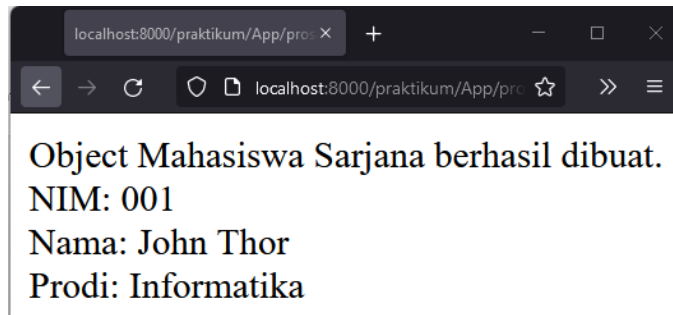
Pilih Object yang mau dibuat

[Mahasiswa Sarjana](#)
[Mahasiswa Magister](#)

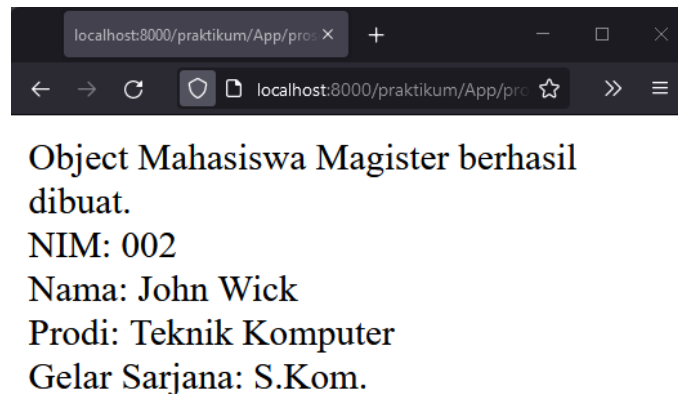
6. Berikut adalah tampilan Form Sarjana dan Form Magister

Form Sarjana	Form Magister
NIM Nama Prodi <input type="submit" value="Submit"/>	NIM Nama Prodi Gelar Sarjana <input type="submit" value="Submit"/>

7. Ketika form sarjana disubmit, maka akan tampil seperti berikut:



8. Ketika form magister disubmit, maka akan tampil seperti berikut:



9. Berikut adalah potongan kode pada file proses.php yang menampilkan hasil seperti pada gambar poin 7 dan 8.

```
echo "Object Mahasiswa " . $mhs::LEVEL . " berhasil dibuat.<br />";  
if($mhs::LEVEL == "Sarjana"){  
    echo "NIM: " . $mhs->getNim() . "<br />";  
    echo "Nama: " . $mhs->getNama() . "<br />";  
    echo "Prodi: " . $mhs->getProdi();  
}else{  
    echo "NIM: " . $mhs->getNim() . "<br />";  
    echo "Nama: " . $mhs->getNama() . "<br />";  
    echo "Prodi: " . $mhs->getProdi() . "<br />";  
    echo "Gelar Sarjana: " . $mhs->getSarjana();  
}
```

10. Gunakan init.php untuk melakukan autoloading class Mahasiswa dari folder Sarjana dan Magister.

MINGGU 5

UPLOAD DAN MANIPULASI FILE

DESKRIPSI TEMA

Mahasiswa mempraktikkan bagaimana melakukan upload file dan membuat file (PDF dan Excel) menggunakan PHP

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu merancang dan membangun aplikasi web untuk melakukan upload dan manipulasi file menggunakan PHP

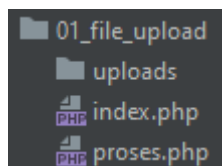
PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor (VS Code, PHPStorm, Atom, etc.)
3. Composer
4. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

File Upload Sederhana

1. Buatlah sebuah folder "01_file_upload", kemudian buat file index.php dan proses.php



2. Tambahkan kode berikut pada file index.php

```
<!doctype html>
<html>
<head>
  <title>Simple File Upload</title>
</head>
<body>

<form action="01_file_upload/proses.php" method="post" enctype="multipart/form-data">
  <input type="file" name="foto" />
  <button type="submit">Upload</button>
</form>

</body>
</html>
```

3. Tambahkan kode berikut pada file proses.php

```
<?php
echo "<pre>";
var_dump($_FILES);
```

4. Coba upload file dan lihat hasilnya

```
array(1) {
  ["foto"]=>
  array(6) {
    ["name"]=>
    string(11) "ironman.jpg"
    ["full_path"]=>
    string(11) "ironman.jpg"
    ["type"]=>
    string(10) "image/jpeg"
    ["tmp_name"]=>
    string(24) "C:\xampp\tmp\php1BF5.tmp"
    ["error"]=>
    int(0)
    ["size"]=>
    int(221578)
  }
}
```

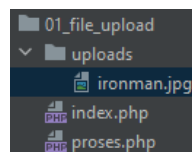
5. Ubah kode pada file proses.php untuk menyimpan file yang diupload secara permanen pada folder uploads

```
<?php

$filename = $_FILES['foto']['name'];
$temp_file = $_FILES['foto']['tmp_name'];

move_uploaded_file($temp_file, "uploads/{$filename}");
```

6. Lakukan upload file Kembali, apakah file berhasil tersimpan di folder uploads?



7. Tambahkan kode berikut untuk melakukan validasi upload file hanya untuk file gambar saja.

```
<?php
$filename = $_FILES['foto']['name'];
$temp_file = $_FILES['foto']['tmp_name'];

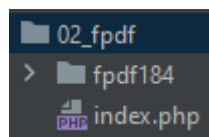
$file_ext = explode( separator: ".", $filename);
$file_ext = end( &array: $file_ext);
$file_ext = strtolower($file_ext);

switch($file_ext){
    case 'jpg':
    case 'jpeg':
    case 'png':
    case 'svg':
    case 'webp':
    case 'bmp':
    case 'gif':
        move_uploaded_file($temp_file, "uploads/{$filename}");
        echo "File berhasil di upload.";
        break;
    default: echo "Anda hanya bisa upload file gambar.";
}
```

8. Coba Kembali, apakah berhasil proses upload hanya untuk file gambar saja?

Generate PDF menggunakan FPDF

1. Download library FPDF dari <http://www.fpdf.org/>
2. Buat folder o2_fpdf, kemudian extrack library FPDF yang telah di download ke folder tersebut dan buat sebuah file index.php



3. Tambahkan kode berikut pada file index.php

```
<?php
require('fpdf184/fpdf.php');

$pdf = new FPDF();
$pdf->AddPage();
$pdf->SetFont( family: 'Arial', style: 'B', size: 16);
$pdf->Cell( w: 40, h: 10, txt: 'Hello World!');
$pdf->Output();
```

4. Lihat hasilnya, apakah berhasil membuat file PDF?

Hello World!

5. Kerjakan Tugas 1

Composer Hello World

1. Buat folder `o3_composer`, kemudian ketikkan command berikut

```
o3_composer> composer init
```

2. Ikuti panduan dari command seperti berikut:

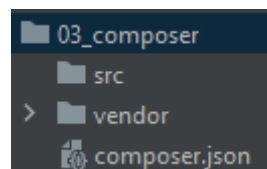
```
This command will guide you through creating your composer.json config.

Package name (<vendor>/<name>) [alexwawo/o3_composer]:
Description []: Praktik menggunakan Composer
Author [alexwawo <alex.wawo@gmail.com>, n to skip]:
Minimum Stability []: dev
Package Type (e.g. library, project, metapackage, composer-plugin) []:
License []:

Define your dependencies.

Would you like to define your dependencies (require) interactively [yes]? n
Would you like to define your dev dependencies (require-dev) interactively [yes]? n
Add PSR-4 autoload mapping? Maps namespace "Alexwawo\O3Composer" to the entered relative path. [src/, n to skip]:
```

3. Akan muncul file dengan struktur seperti berikut:



4. Buat file `Greetings.php` di dalam folder `src/`, dan tuliskan kode berikut

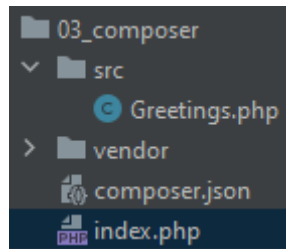
```
<?php
namespace HelloWorld;

class Greetings {
    public static function sayHelloWorld() {
        return 'Hello World';
    }
}
```

5. Ubah kode pada `composer.json` menjadi seperti berikut

```
"autoload": {
    "psr-4": {
        "HelloWorld\\": "src/"
    }
}
```

6. Buat file `index.php` pada di dalam folder `o3_composer` (pada level yang sama dengan file `composer.json`)



7. Tuliskan kode berikut pada file index.php

```
<?php
require_once __DIR__ . '/vendor/autoload.php';

use HelloWorld\Greetings;
echo Greetings::sayHelloWorld();
```

8. Jalankan di browser, apakah muncul error seperti berikut?

Fatal error: Uncaught Error: Class "HelloWorld\Greetings" not found in C:\Insync\alexander.waworuntu@lecturer.umn.ac.id\DELL-PC\Documents\CODE\PemWeb-Genap-21-22\Week-05\03_composer\index.php:5 Stack trace: #0 {main} thrown in C:\Insync\alexander.waworuntu@lecturer.umn.ac.id\DELL-PC\Documents\CODE\PemWeb-Genap-21-22\Week-05\03_composer\index.php on line 5

9. Jika ya, jalankan command: `composer dump-autoload`, kemudian coba jalankan Kembali di browser, apakah berhasil?



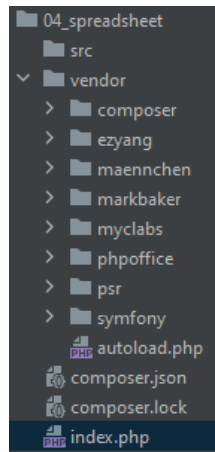
Hello World

Generate File Excel dengan PhpSpreadsheet

1. Buat sebuah folder `o4_spreadsheet`, kemudian jalankan command "composer init"
2. Baca dokumentasi library PhpSpreadsheet:
<https://packagist.org/packages/phpoffice/phpspreadsheet>
3. Jalankan command berikut untuk meng-install PhpSpreadsheet

```
composer require phpoffice/phpspreadsheet --prefer-source
```

4. Buat sebuah file `index.php`



5. Tuliskan kode berikut pada file index.php

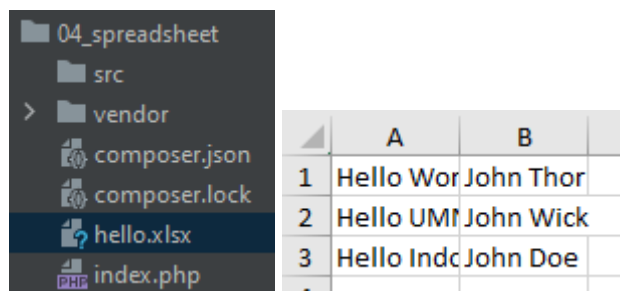
```
<?php
require_once __DIR__ . '/vendor/autoload.php';

use PhpOffice\PhpSpreadsheet\Spreadsheet;
use PhpOffice\PhpSpreadsheet\Writer\Xlsx;
use PhpOffice\PhpSpreadsheet\IOFactory;

$spreadsheet = new Spreadsheet();
$sheet = $spreadsheet->getActiveSheet();
$sheet->setCellValue( pCoordinate: 'A1', pValue: 'Hello World!');
$sheet->setCellValue( pCoordinate: 'A2', pValue: 'Hello UMN');
$sheet->setCellValue( pCoordinate: 'A3', pValue: 'Hello Indonesia');
$sheet->setCellValue( pCoordinate: 'B1', pValue: 'John Thor');
$sheet->setCellValue( pCoordinate: 'B2', pValue: 'John Wick');
$sheet->setCellValue( pCoordinate: 'B3', pValue: 'John Doe');

$writer = new Xlsx($spreadsheet);
$writer->save( pFilename: 'hello.xlsx');
```

6. Coba jalankan di browser, apakah berhasil membuat file Excel?



7. Tambahkan kode berikut pada file index.php

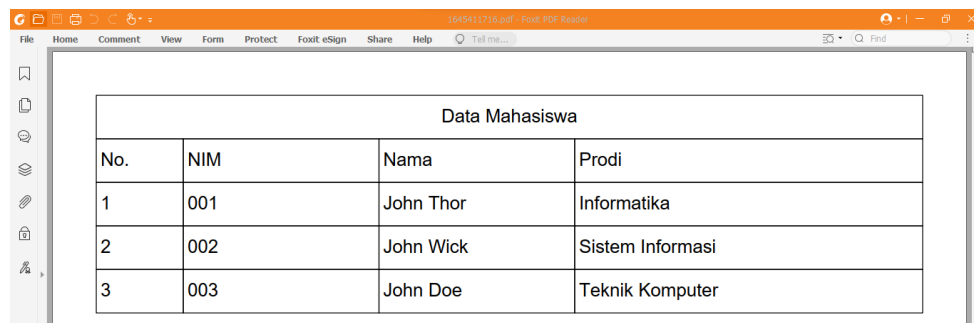
```
$inputFileType = 'Xlsx';
$inputFileName = __DIR__ . '/hello.xlsx';
$reader = IOFactory::createReader($inputFileType);
$reader->setReadDataOnly( pValue: true);
$spreadsheet = $reader->load($inputFileName);
$sheetData = $spreadsheet->getActiveSheet()->toArray( nullValue: null, calculateFormulas: true,
    formatData: true, returnCellRef: true);
echo "<pre>";
var_dump($sheetData);
```

8. Coba jalankan di browser, data file excel akan ditampilkan dalam bentuk array.

```
array(3) {
  [1]=>
  array(2) {
    ["A"]=>
    string(12) "Hello World!"
    ["B"]=>
    string(9) "John Thor"
  }
  [2]=>
  array(2) {
    ["A"]=>
    string(9) "Hello UMN"
    ["B"]=>
    string(9) "John Wick"
  }
  [3]=>
  array(2) {
    ["A"]=>
    string(15) "Hello Indonesia"
    ["B"]=>
    string(8) "John Doe"
  }
}
```

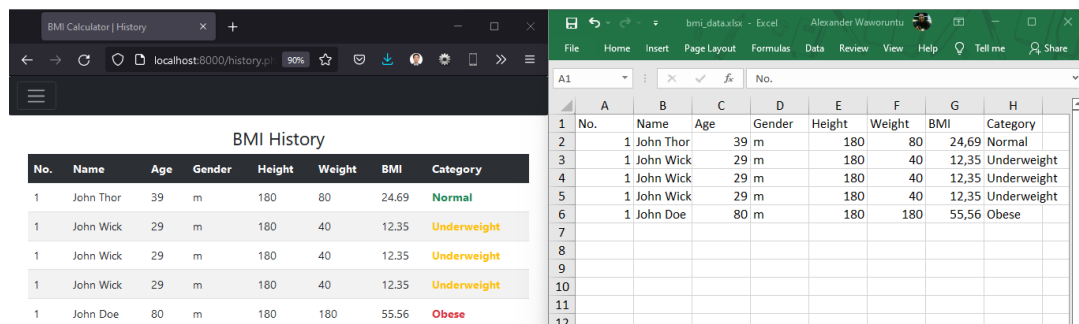
TUGAS

1. Modifikasi file index.php untuk menghasilkan file PDF dengan tampilan seperti berikut



Data Mahasiswa			
No.	NIM	Nama	Prodi
1	001	John Thor	Informatika
2	002	John Wick	Sistem Informasi
3	003	John Doe	Teknik Komputer

2. Modifikasi aplikasi perhitungan BMI yang Anda kerjakan pada pertemuan ke-3 sehingga data hasil perhitungan BMI disimpan ke dalam file Excel dan data history diambil dari file Excel yang sama.



No.	Name	Age	Gender	Height	Weight	BMI	Category
1	John Thor	39	m	180	80	24.69	Normal
1	John Wick	29	m	180	40	12.35	Underweight
1	John Wick	29	m	180	40	12.35	Underweight
1	John Wick	29	m	180	40	12.35	Underweight
1	John Doe	80	m	180	180	55.56	Obese

MINGGU 6

PHP & MYSQL

DESKRIPSI TEMA

Mahasiswa mempraktikkan proses manipulasi data pada database menggunakan PHP dan MySQL.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu merancang dan membangun aplikasi web yang terkoneksi dengan basis data MySQL dan melakukan proses CRUD pada basis data tersebut

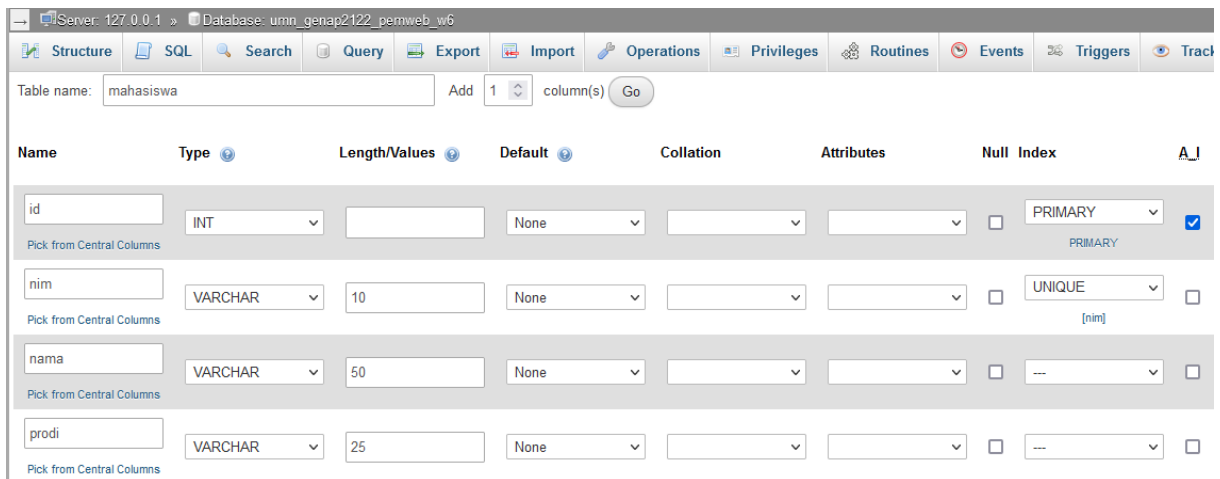
PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor
3. Web Browser
4. Composer

LANGKAH-LANGKAH PRAKTIKUM

MySQLi Extension

1. Buka phpMyAdmin dan buat sebuah database baru (nama database bebas, contoh ini menggunakan nama database umn_genap2122_pemweb_w6), buat sebuah table mahasiswa dengan struktur seperti berikut



The screenshot shows the phpMyAdmin interface for a database named 'umn_genap2122_pemweb_w6'. The 'Structure' tab is selected, and a table named 'mahasiswa' is being created. The table structure is as follows:

Name	Type	Length/Values	Default	Collation	Attributes	Null	Index
id	INT		None			☐	PRIMARY
nim	VARCHAR	10	None			☐	UNIQUE
nama	VARCHAR	50	None			☐	
prodi	VARCHAR	25	None			☐	

2. Insert data dummy melalui phpMyAdmin

+ Options

					id	nim	nama	prodi
☐	Edit	Copy	Delete	1	001		John Thor	Informatika
☐	Edit	Copy	Delete	2	002		John Doel	Sistem Informasi

3. Buat sebuah file latihan_mysql.php, kemudian ketikkan kode berikut

```
<?php

// 1. Koneksi DB
$kunci = mysqli_connect("localhost", "root", "", "umn_genap2122_pemweb_w6");

// 2. Query SQL
$sql = "SELECT * FROM mahasiswa";

// 3. Eksekusi SQL
$keranjang = mysqli_query($kunci, $sql);

//4 Menampilkan hasil Query
$data = mysqli_fetch_assoc($keranjang);
echo "<pre>";
var_dump($data);
```

4. Jalankan di browser, maka akan tampil seperti berikut

```
array(4) {
    ["id"]=>
    string(1) "1"
    ["nim"]=>
    string(3) "001"
    ["nama"]=>
    string(9) "John Thor"
    ["prodi"]=>
    string(11) "Informatika"
}
```

5. Modifikasi kode php menjadi seperti berikut untuk menampilkan 2 data dari database. Perhatikan bahwa setiap kali function `mysqli_fetch_assoc()` dijalankan, maka akan menarik baris data berikutnya dari database.

```
//4 Menampilkan hasil Query
echo "<pre>";

$data = mysqli_fetch_assoc($keranjang);
var_dump($data);

$data = mysqli_fetch_assoc($keranjang);
var_dump($data);
```

6. Berikut adalah tampilan di browser

```

array(4) {
  ["id"]=>
  string(1) "1"
  ["nim"]=>
  string(3) "001"
  ["nama"]=>
  string(9) "John Thor"
  ["prodi"]=>
  string(11) "Informatika"
}
array(4) {
  ["id"]=>
  string(1) "2"
  ["nim"]=>
  string(3) "002"
  ["nama"]=>
  string(9) "John Doel"
  ["prodi"]=>
  string(16) "Sistem Informasi"
}

```

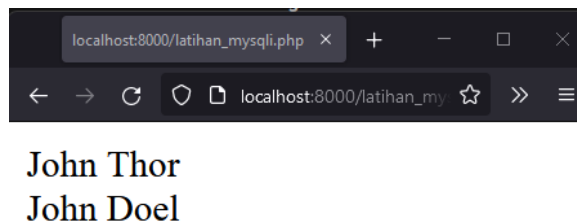
7. Modifikasi kode PHP menjadi seperti berikut untuk melakukan perulangan sebanyak jumlah data pada hasil query SQL.

```

//4 Menampilkan hasil Query
while($data = mysqli_fetch_assoc($keranjang)){
    echo $data['nama'];
    echo "<br />";
}

```

8. Tampilan di browser



9. Untuk melakukan INSERT, UPDATE, dan DELETE Anda dapat mengubah kode SQL yang akan dijalankan. Berikut contoh kode untuk melakukan INSERT data mahasiswa.

```

<?php
// 1. Koneksi DB
$kunci = mysqli_connect("localhost", "root", "", "
    umn_genap2122_pemweb_w6");

// 2. Query SQL
$sql = "INSERT INTO mahasiswa (nim, nama, prodi)
    VALUES ('003', 'John Doel', 'Teknik Komputer')";

// 3. Eksekusi SQL
mysqli_query($kunci, $sql);

```

PHP Data Object (PDO)

1. Buat sebuah file Latihan_pdo.php, kemudian tuliskan kode php berikut ini.

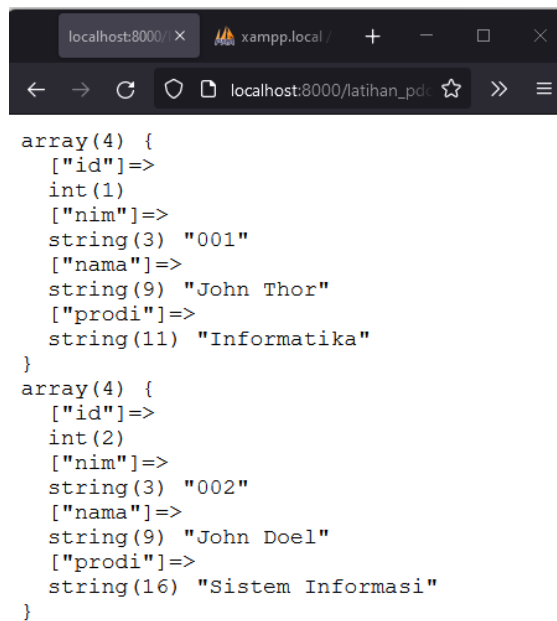
```
<?php
// 1
$dsn = "mysql:host=localhost;dbname=umn_genap2122_pemweb_w6";
$kunci = new PDO($dsn, "root", "");

// 2
$sql = "SELECT * FROM mahasiswa";

// 3
$hasil = $kunci->query($sql);

// 4
echo "<pre>";
$row = $hasil->fetch(PDO::FETCH_ASSOC);
var_dump($row);
$row = $hasil->fetch(PDO::FETCH_ASSOC);
var_dump($row);
```

2. Lihat hasilnya di browser



```
array(4) {
  ["id"]=>
  int(1)
  ["nim"]=>
  string(3) "001"
  ["nama"]=>
  string(9) "John Thor"
  ["prodi"]=>
  string(11) "Informatika"
}
array(4) {
  ["id"]=>
  int(2)
  ["nim"]=>
  string(3) "002"
  ["nama"]=>
  string(9) "John Doel"
  ["prodi"]=>
  string(16) "Sistem Informasi"
}
```

3. Untuk melakukan INSERT, UPDATE, dan DELETE dapat dilakukan dengan mengubah kode SQL. Berikut adalah contoh kode untuk melakukan UPDATE.

```
<?php
// 1
$dsn = "mysql:host=localhost;dbname=umn_genap2122_pemweb_w6";
$kunci = new PDO($dsn, "root", "");

// 2
$sql = "UPDATE mahasiswa
        SET nama = 'John Wick'
        WHERE nim = '002'";

// 3
$kunci->query($sql);
```

INSERT Data menggunakan Form

1. Buat sebuah file dengan nama insert_form.php dan tuliskan kode berikut

```
<form action="insert_proses.php" method="post">
    <input type="text" name="nim" placeholder="NIM" /><br />
    <input type="text" name="nama" placeholder="Nama" /><br />
    <input type="text" name="prodi" placeholder="Prodi" /><br />
    <input type="submit" />
</form>
```

2. Buat sebuah file dengan nama insert_proses.php dan tuliskan kode berikut

```
<?php

// Form Data:
$nim = $_POST['nim'];
$nama = $_POST['nama'];
$prodi = $_POST['prodi'];

//1
$dsn = "mysql:host=localhost;dbname=umn_genap2122_pemweb_w6";
$kunci = new PDO($dsn, "root", "");

//2
$sql = "INSERT INTO mahasiswa (nim, nama, prodi)
      VALUES ('{$nim}', '{$nama}', '{$prodi}')";

//3
$kunci->query($sql);
```

3. Coba submit data dari form, apakah data berhasil tersimpan di database?

PDO Prepared Statement

1. PDO Prepared Statement digunakan untuk menghindari SQL Injection. Modifikasi kode pada file insert_proses.php menjadi seperti berikut:

```
<?php

// Form Data:
$nim = $_POST['nim'];
$nama = $_POST['nama'];
$prodi = $_POST['prodi'];

//1
$dsn = "mysql:host=localhost;dbname=umn_genap2122_pemweb_w6";
$kunci = new PDO($dsn, "root", "");

//2
$sql = "INSERT INTO mahasiswa (nim, nama, prodi)
      VALUES (?, ?, ?)";

//3
$stmt = $kunci->prepare($sql);
$data = [$nim, $nama, $prodi];
$stmt->execute($data);
```

Menampilkan Hasil Select dan Membuat Link dengan Data

1. Buat sebuah file dengan nama select.php, tuliskan kode berikut:

```
<?php
// 1
$dsn = "mysql:host=localhost;dbname=umn_genap2122_pemweb_w6";
$kunci = new PDO($dsn, "root", "");

// 2
$sql = "SELECT * FROM mahasiswa";

// 3
$hasil = $kunci->query($sql);
?>
```

```
<body>
<h1>Daftar Mahasiswa</h1>
<table border="1">
  <tr>
    <th>NIM</th>
    <th>Nama</th>
    <th>Prodi</th>
    <th>Tindakan</th>
  </tr>
  <?php
  while($row = $hasil->fetch(PDO::FETCH_ASSOC)){
    ?>

    <tr>
      <td><?= $row['nim'] ?></td>
      <td><?= $row['nama'] ?></td>
      <td><?= $row['prodi'] ?></td>
      <td>
        <a href="individu.php?id=<?= $row['id'] ?>">Detail</a>
      </td>
    </tr>

    <?php
    }
  ?>
</table>
</body>
```

2. Lihat hasilnya di browser

NIM	Nama	Prodi	Tindakan
001	John Thor	Informatika	Detail
002	John Wick	Sistem Informasi	Detail
003	John Doel	Teknik Komputer	Detail
004	John DOe	IF	Detail

3. Buat sebuah file individu.php, tuliskan kode berikut

```
<?php
// Get data from URL:
$id = $_GET['id'];

// 1
$dsn = "mysql:host=localhost;dbname=umn_genap2122_pemweb_w6";
$kunci = new PDO($dsn, "root", "");

// 2
$sql = "SELECT * FROM mahasiswa WHERE id = ?";

//3
$stmt = $kunci->prepare($sql);
$data = [$id];
$stmt->execute($data);
$row = $stmt->fetch(PDO::FETCH_ASSOC)
?>

NIM: <?= $row['nim'] ?><br />
Nama: <?= $row['nama'] ?><br />
Prodi: <?= $row['prodi'] ?>
```

4. Ketika link "Detail" di-klik maka akan menampilkan data dari masing-masing mahasiswa.

localhost:8000/	xampp.local	+	-	□	×
←	→	↻	🔒	📄	localhost:8000/individu.php?id=1 ☆ >> ☰
NIM: 001 Nama: John Thor Prodi: Informatika					

TUGAS

1. Buat aplikasi yang dapat menyimpan data Mahasiswa, yaitu NIM, Nama, Prodi, dan Foto ke dalam database. Foto disimpan dalam folder khusus dan yang tersimpan pada database adalah path atau nama file dari foto mahasiswa.
2. Terdapat halaman index yang menampilkan semua data mahasiswa yang ada pada database beserta thumbnail foto masing-masing mahasiswa. Terdapat tombol untuk melakukan edit dan delete data mahasiswa.
3. Ketika tombol edit di-klik, maka akan menampilkan form edit yang telah berisi data lama.

MINGGU 7

PHP SECURITY & AUTHENTICATION

DESKRIPSI TEMA

Mahasiswa mempraktikkan bagaimana membangun aplikasi yang aman dengan PHP

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu merancang dan membangun aplikasi web yang aman dengan fitur autentikasi pengguna

PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor
3. Web Browser

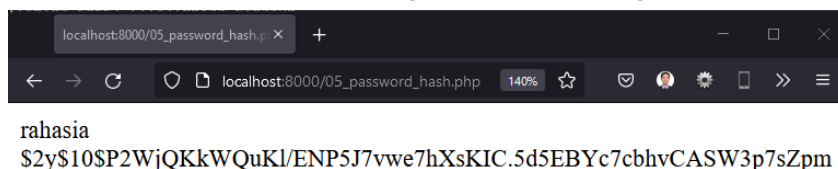
LANGKAH-LANGKAH PRAKTIKUM

Password Hash, Password Verify & Random Token

1. Buatlah sebuah file password_hash.php, kemudian tuliskan kode berikut:

```
<?php
echo "rahasia";
echo "<br />";
echo password_hash("rahasia", PASSWORD_BCRYPT);
```

2. Ketika dijalankan di browser, akan menampilkan 60 karakter string hasil enkripsi. Jika browser di-refresh, maka akan dihasilkan 60 karakter string hasil enkripsi yang berbeda.



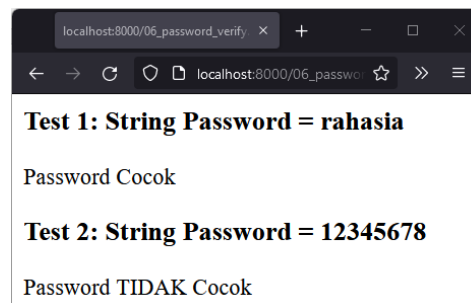
3. Buatlah sebuah file password_verify.php, kemudian tuliskan kode berikut:

```
<?php
$input = "rahasia";
$hashedPwd = password_hash($input, PASSWORD_BCRYPT);

echo "<h3>Test 1: String Password = rahasia</h3>";
if(password_verify("rahasia", $hashedPwd)){
    echo "Password Cocok";
}else{
    echo "Password TIDAK Cocok";
}

echo "<h3>Test 2: String Password = 12345678</h3>";
if(password_verify("12345678", $hashedPwd)){
    echo "Password Cocok";
}else{
    echo "Password TIDAK Cocok";
}
}
```

4. Ketika di jalankan, password verify akan mencocokkan string password yang diberikan sebagai argument pertama dengan string hasil enkripsi dari string "rahasia".

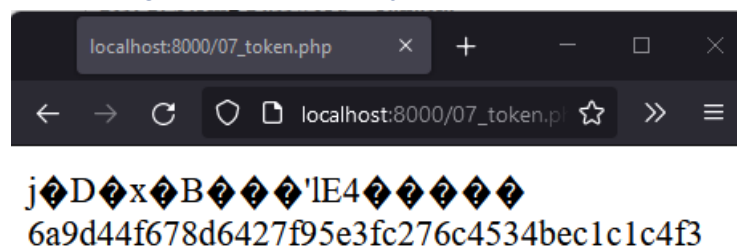


5. Buat file token.php, kemudian tuliskan kode berikut

```
<?php
$random_bytes = random_bytes(20);
echo $random_bytes;
echo "<br />";

$token = bin2hex($random_bytes);
echo $token;
```

6. Ketika di jalankan, function random_bytes akan menghasilkan random bytes sebanyak 20 karakter dan function bin2hex akan mengkonversi bytes menjadi hexadecimal.



Authentication System

1. Buatlah sebuah database dengan table usersz dengan struktur seperti berikut

Server: 127.0.0.1 Database: umn_genap2122_pemweb_w7

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers Trac

Table name: users2 Add 1 column(s) Go

Name	Type	Length/Values	Default	Collation	Attributes	Null	Index
id	INT		None			☐	PRIMARY ☒
username	VARCHAR	50	None			☐	UNIQUE ☐ [username]
password	VARCHAR	100	None			☐	--- ☐
role	VARCHAR	25	None			☐	--- ☐

Structure

2. Buatlah sebuah file db.php dan tuliskan kode berikut

```
<?php
// db.php
// DB Credentials
define('DSN', 'mysql:host=localhost;dbname=umn_genap2122_pemweb_w7');
define('DBUSER', 'root');
define('DBPASS', '');

// 1. Connect to DB
$db = new PDO(DSN, DBUSER, DBPASS);
```

3. Buat file register.php dan tuliskan kode berikut

```
<h1>Register</h1>
<form action="register_process.php" method="post">
  <input type="text" name="username" placeholder="Username" /><br />
  <input type="password" name="password" placeholder="Password" /><br />
  <select name="role" required>
    <option value="">-- Role --</option>
    <option value="student">Student</option>
    <option value="lecturer">Lecturer</option>
  </select>
  <button type="submit">Register</button>
</form>
```

4. Buat file register_process.php, kemudian tuliskan kode berikut

```
<?php
require_once('db.php');

// Data from Form
$username = $_POST['username'];
$password = $_POST['password'];
$role = $_POST['role'];

// Encrypt the Password
$en_pass = password_hash($password, PASSWORD_BCRYPT);

// 2. SQL Query
$sql = "INSERT INTO users2 (username, password, role)
      VALUES(?, ?, ?)";

// 3 Execute Query
$result = $db->prepare($sql);
$result->execute([$username, $en_pass, $role]);

header('location: login.php');
```

5. Coba jalankan di browser dan register beberapa user dengan role yang berbeda. Apakah berhasil tersimpan ke database dengan password yang ter-enkripsi?

id	username	password	role
3	alexwawo	\$2y\$10\$WqYrnDY9qik.w50VDZ4upuCwPRbJhzkF5RyAOEGUwV...	lecturer
4	john	\$2y\$10\$d2dvgZnidzJHKmzy6lJklel1ZzXUpw7QDkGSBBFIJQY...	student

6. Buat file login.php, kemudian tuliskan kode berikut

```
<h1>Login</h1>
<form action="login_process.php" method="post">
  <input type="text" name="username" placeholder="Username" /><br />
  <input type="password" name="password" placeholder="Password" /><br />
  <button type="submit">Login</button>
</form>
```

7. Buat file login_process.php, kemudian tuliskan kode berikut

```
<?php
session_start();
require_once('db.php');

// Data from Form
$username = $_POST['username'];
$password = $_POST['password'];

// Check if user exists
$sql = "SELECT * FROM users2
        WHERE username = ?";
$stmt = $db->prepare($sql);
$stmt->execute([$username]);
$row = $stmt->fetch(PDO::FETCH_ASSOC);

if(!$row){
    echo "User not found";
}else{
    // Check if password correct
    echo "USERNAME ada di database<br />";
    echo "Password yang diinput di form login: " . $password;
    echo "<br />Password yang tersimpan di database: " . $row['password'];
    if(!password_verify($password, $row['password'])){
        echo "Wrong password";
    }else{
        // Login success, set SESSION DATA
        $_SESSION['user_id'] = $row['id'];
        $_SESSION['user_role'] = $row['role'];
        $_SESSION['username'] = $row['username'];
        header('location: internal.php');
    }
}
}
```

8. Buatlah file internal.php, kemudian tuliskan kode berikut

```
<?php
session_start();
// require_once('db.php');

if(!isset($_SESSION['username']) &&
    !isset($_SESSION['user_id']) &&
    !isset($_SESSION['user_role'])){
    echo "You don't have access to this page";
}else{
    if($_SESSION['user_role'] == "lecturer"){
        echo "THIS IS A LECTURER AREA<br />";
        echo "WELCOME " . $_SESSION['username'];
    }elseif($_SESSION['user_role'] == "student"){
        echo "THIS IS A STUDENT AREA<br />";
        echo "WELCOME " . $_SESSION['username'];
    }
}
?>

<?php
if(!isset($_SESSION['user_id'])){
    ?>
    <a href="login.php">Login</a>
    <?php
}else{
    ?>
    <a href="logout.php">Logout</a>
    <?php
}
?>
```

9. Coba lakukan login, apakah berhasil masuk ke halaman internal.php? Kemudian logout, dan coba buka halaman internal.php tanpa login. Jika dilakukan dengan benar, maka halaman internal.php hanya bisa diakses jika user telah login.

10. Tambahkan kode berikut pada login.php untuk me-redirect user jika telah melakukan login langsung ke halaman internal.php

```
<?php
session_start();
if(isset($_SESSION['user_id'])){
    header('location: internal.php');
}
?>

<h1>Login</h1>
<form action="login_process.php" method="post">
    <input type="text" name="username" placeholder="Username" /><br />
    <input type="password" name="password" placeholder="Password" /><br />
    <button type="submit">Login</button>
</form>
```

TUGAS

Modifikasi tugas pada pertemuan sebelumnya (minggu ke-6), dimana manajemen (CRUD) data mahasiswa hanya bisa dilakukan oleh user yang telah login.

MINGGU 8

DASAR-DASAR FRAMEWORK LARAVEL



DESKRIPSI TEMA

Mahasiswa mempraktikkan dasar-dasar pengembangan web dengan konsep MVC menggunakan framework Laravel.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menjelaskan dasar-dasar pengembangan web menggunakan Laravel Framework.

PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor
3. Composer
4. Web browser

LANGKAH-LANGKAH PRAKTIKUM

Instalasi Laravel

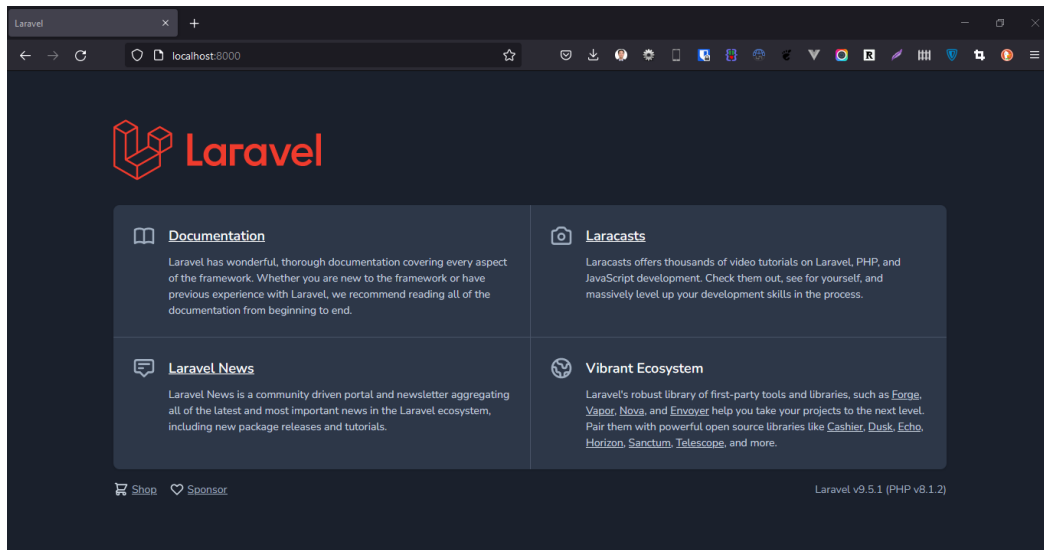
1. Arahkan directory command prompt/terminal pada folder yang diinginkan sebagai tempat instalasi project, kemudian jalankan command berikut:

```
\Week-08>composer create-project laravel/laravel my-app
```

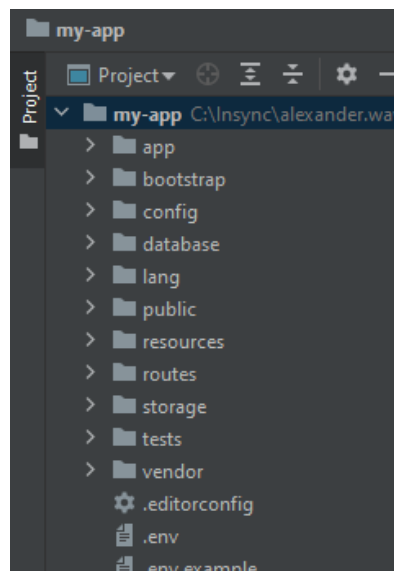
2. Masuk ke folder project, kemudian jalankan command **php artisan serve** untuk menjalankan local web server.

```
\Week-08>cd my-app
\Week-08\my-app>php artisan serve
```

3. Jalankan app di browser melalui URL <http://localhost:8000> (default).



4. Buka project pada code Editor (VS Code/PHPStorm/Sublime Text)

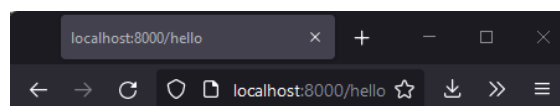


Routing (File: "routes/web.php")

1. [Basic Routing] Buka file "routes/web.php", kemudian tambahkan kode berikut:

```
Route::get( uri: '/hello', function(){
    return "Hello World!";
});
```

2. Arahkan URL web browser pada <http://localhost:8000/hello>, lihat hasilnya



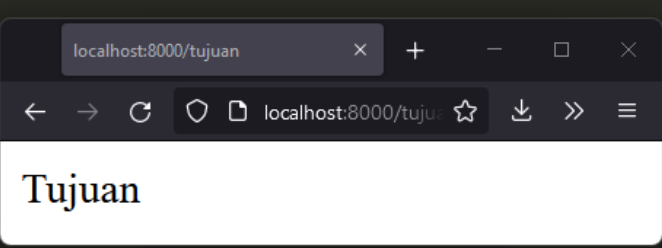
Hello World!

3. Tambahkan kode berikut untuk membuat Redirect Route

```
Route::get( uri: '/awal', function(){
    return 'Awal';
});

Route::get( uri: '/tujuan', function(){
    return 'Tujuan';
});

Route::redirect( uri: '/awal', destination: '/tujuan');
```

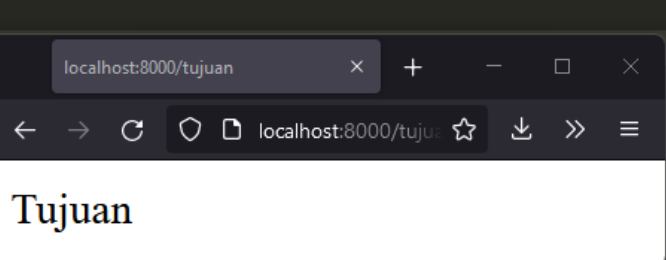


4. Modifikasi kode sebelumnya untuk membuat Named Route dan menggunakannya pada Redirect

```
Route::get( uri: '/awal', function(){
    return redirect()->route( route: 'destination');
});

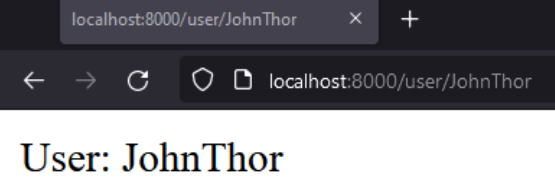
Route::get( uri: '/tujuan', function(){
    return 'Tujuan';
})->name( name: 'destination');
```

```
//Route::redirect('/awal', '/tujuan');
```

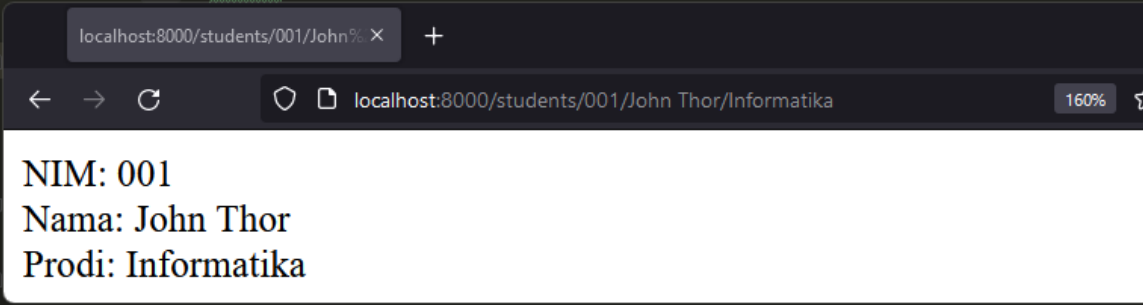


5. Tambahkan kode berikut untuk membuat Route dengan Parameter

```
Route::get( uri: '/user/{name}', function($theName){
    return "User: " . $theName;
});
```

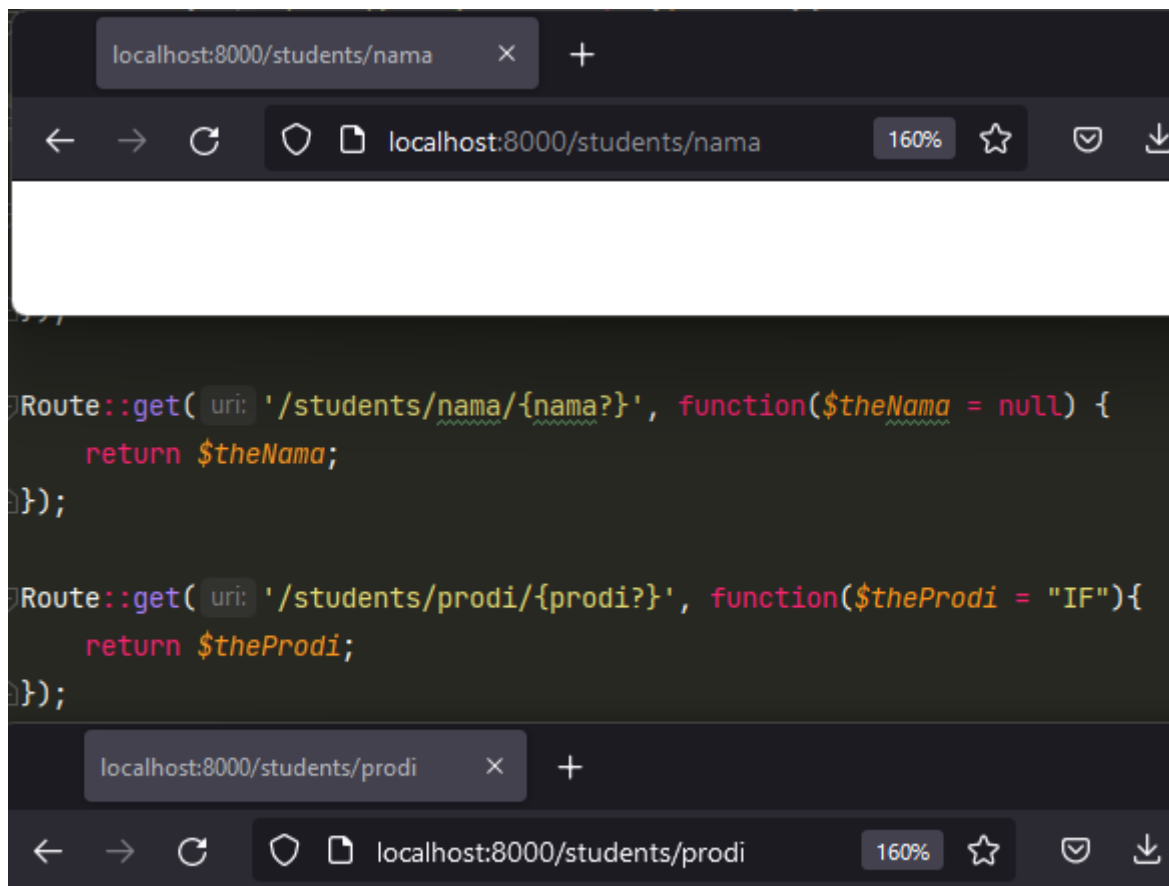


6. Tambahkan kode berikut untuk membuat Route dengan Parameter lebih dari satu



```
Route::get( uri: '/students/{nim}/{nama}/{prodi}', function($theNim, $theNama, $theProdi){
    return "NIM: " . $theNim . "<br /> Nama: " . $theNama . "<br /> " . "Prodi: " . $theProdi;
});
```

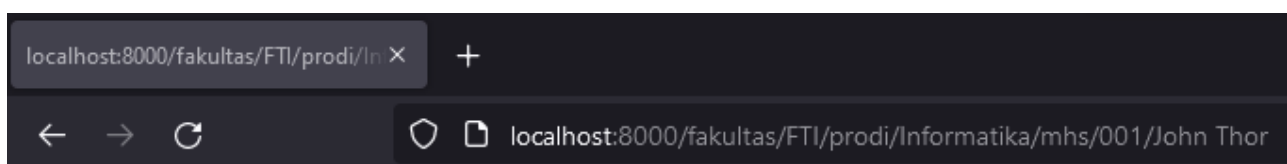
7. Tambahkan kode berikut untuk membuat Route dengan Parameter Opsional



IF

LATIHAN

Buatlah sebuah Route yang memerlukan 4 (empat) buah parameter, yaitu Fakultas, Prodi, NIM dan Nama. Ketika URL Route tersebut dibuka di browser, akan menampilkan data-data parameternya seperti berikut:

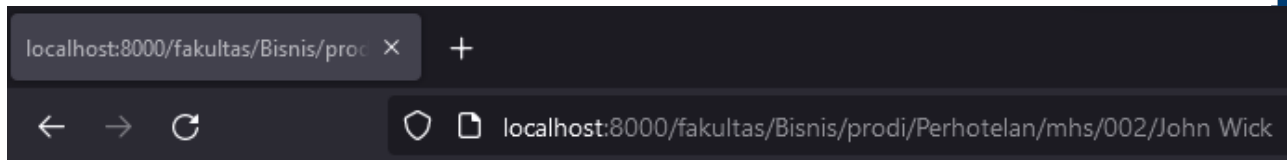


NIM: 001

Nama: John Thor

Prodi: Informatika

Fakultas: FTI

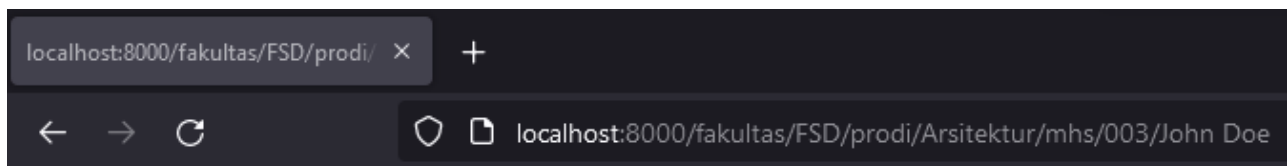


NIM: 002

Nama: John Wick

Prodi: Perhotelan

Fakultas: Bisnis



NIM: 003

Nama: John Doe

Prodi: Arsitektur

Fakultas: FSD

MINGGU 9

CONTROLLER & VIEW

DESKRIPSI TEMA

Mahasiswa mempraktikkan penggunaan Controller dan View pada Framework Laravel

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menerapkan Controller dan View menggunakan Framework Laravel.

PERALATAN YANG DIGUNAKAN

1. Code Editor
2. XAMPP
3. Composer

LANGKAH-LANGKAH PRAKTIKUM

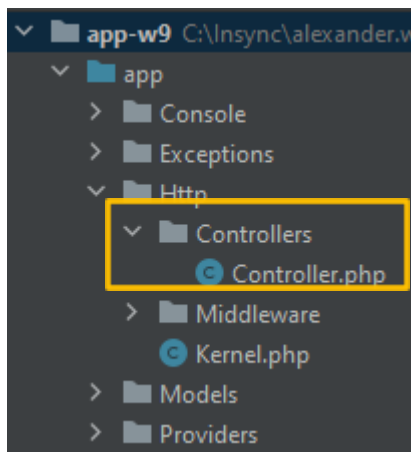
Memulai Proyek Laravel Baru

Buatlah sebuah proyek Laravel baru dengan nama "app-w9".

#composer create-project Laravel/Laravel app-w9

Membuat Controller

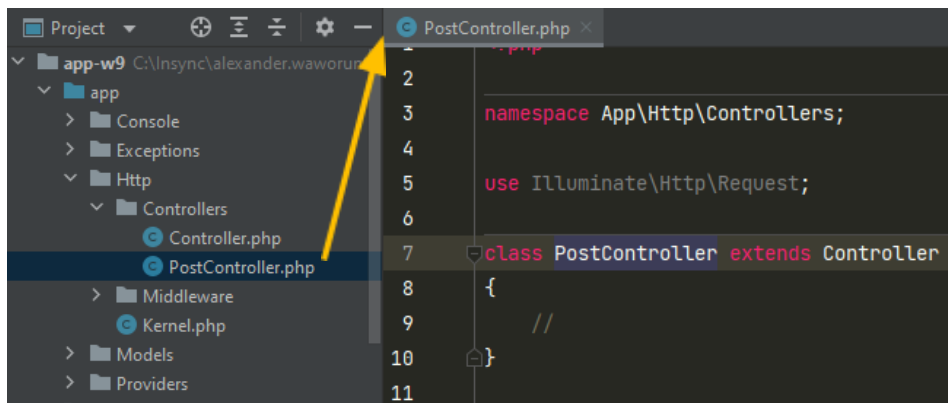
1. Controller berada pada direktori app/Http/Controllers



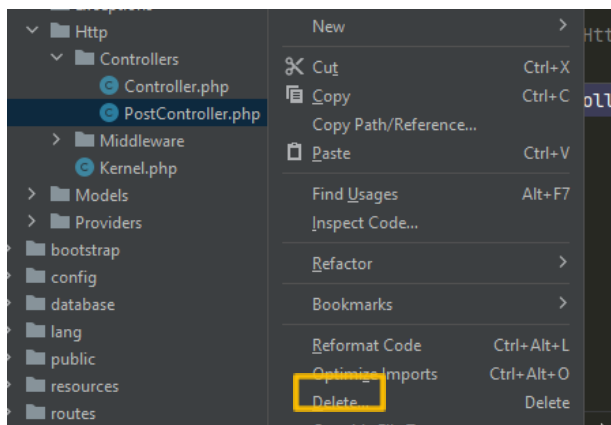
2. Terdapat dua cara untuk membuat Controller, yaitu secara manual dengan membuat file php pada folder Controller dan menulis kode secara manual, atau bisa dilakukan secara otomatis menggunakan php artisan. Untuk membuat Controller menggunakan php artisan, gunakan terminal dan ketikkan command berikut: **php artisan make:controller PostController**

```
app-w9> php artisan make:controller PostController
```

3. File PostController akan otomatis dibuatkan.



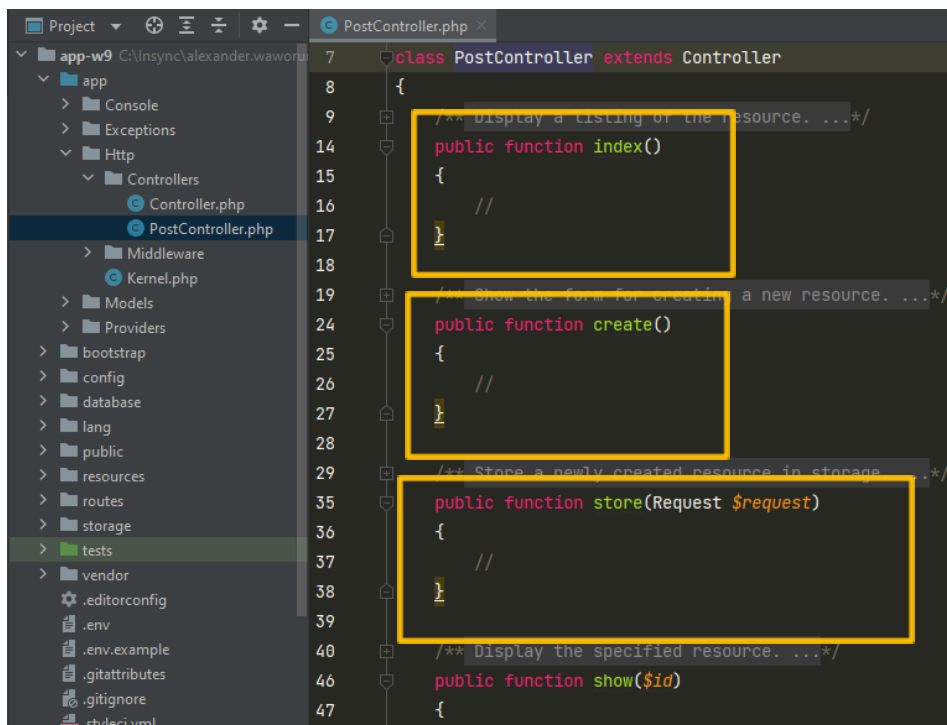
4. Kita akan mencoba membuat file PostController dengan beberapa fungsi dasar. Sebelumnya, hapus terlebih dahulu file PostController yang baru saja dibuat.



5. Ketikkan command berikut pada terminal: **php artisan make:controller PostController --resource**

```
PS C:\Insync\alexander.waworuntu@lecturer.umn.ac.id\DELL-PC\Documents\CODE\Genap2021-22\Pemrograman-Web\Week-09\app-w9> php artisan make:controller PostController --resource
```

6. Beberapa fungsi dasar otomatis sudah dibuatkan pada file PostController.php



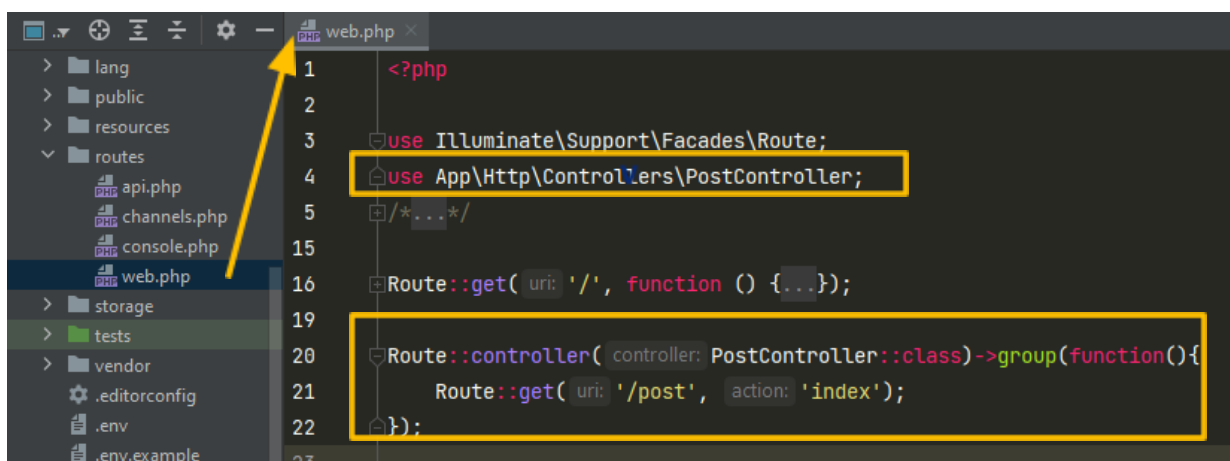
```

7  class PostController extends Controller
8  {
9      /** Display a listing of the resource. ...*/
14     public function index()
15     {
16         //
17     }
18
19     /** Show the form for creating a new resource. ...*/
24     public function create()
25     {
26         //
27     }
28
29     /** Store a newly created resource in storage. ...*/
35     public function store(Request $request)
36     {
37         //
38     }
39
40     /** Display the specified resource. ...*/
46     public function show($id)
47     {

```

Routing Controller

1. Buka file routes/web.php, kemudian ketikkan kode berikut:



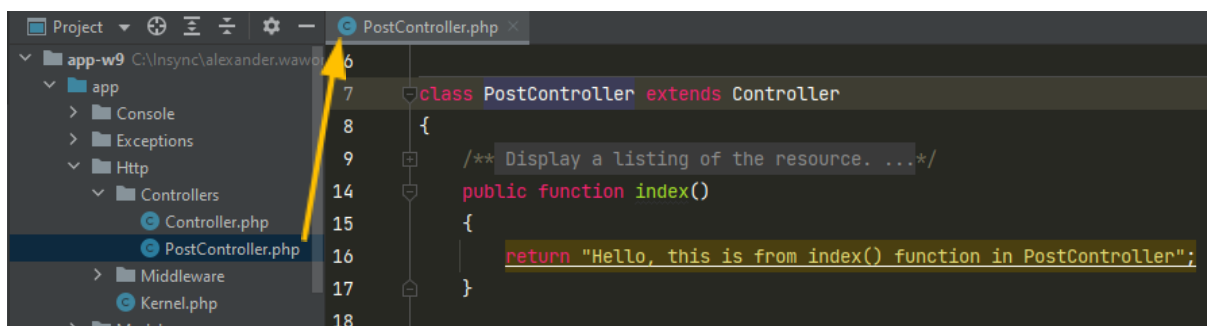
```

1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\PostController;
5
6  /* ... */
15
16  Route::get(uri: '/', function () {...});
19
20  Route::controller(controller: PostController::class)->group(function(){
21      Route::get(uri: '/post', action: 'index');
22  });
23

```

Kode tersebut akan membuat route `/post` untuk memanggil function `index()` yang ada di `PostController`.

2. Kemudian buka file `PostController.php` dan tambahkan kode berikut:



```

6
7  class PostController extends Controller
8  {
9      /** Display a listing of the resource. ...*/
14     public function index()
15     {
16         return "Hello, this is from index() function in PostController";
17     }
18

```

3. Jalankan command `php artisan serve`

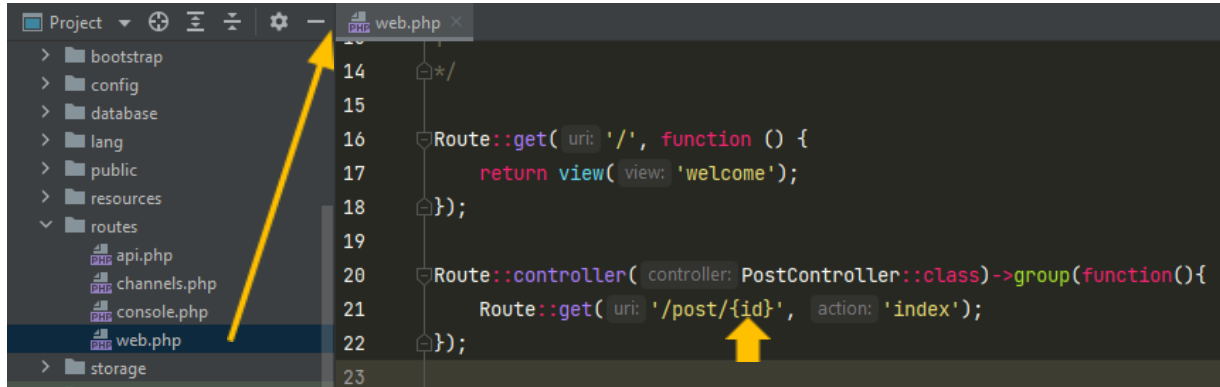
```
app-w9> php artisan serve
```

4. Buka browser dan ketikkan URL: <http://localhost:8000/post>



Passing Data

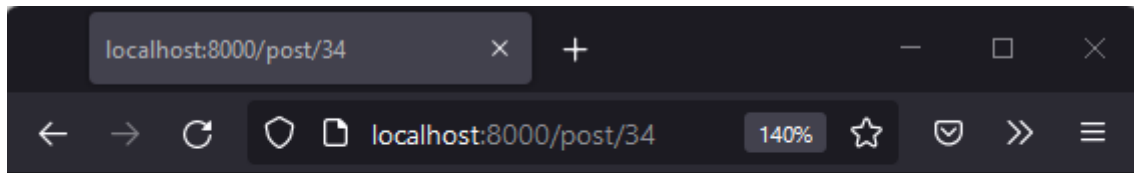
1. Buka file routes/web.php, kemudian tambahkan {id} pada parameter '/post/{id}'.



2. Kemudian pada PostController.php, ubah kode pada function index() menjadi seperti berikut:

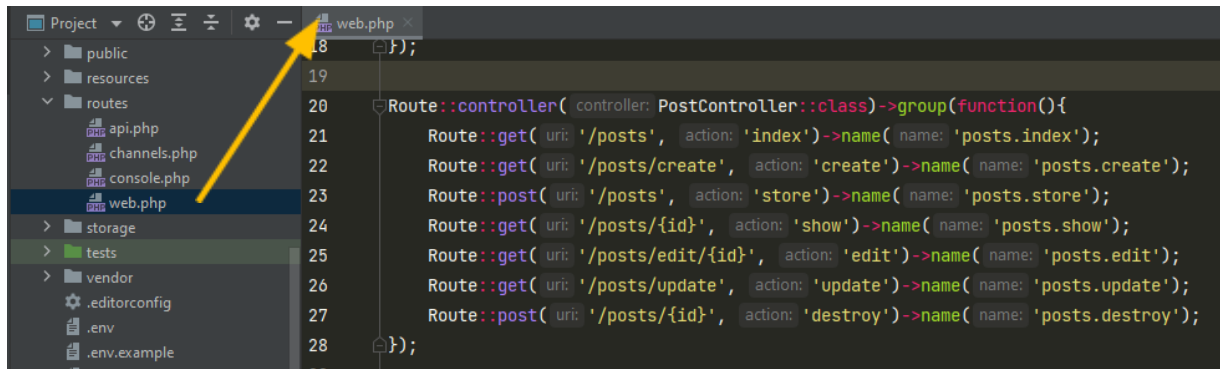


3. Buka browser dan ketikkan URL: <http://localhost:8000/post/34>

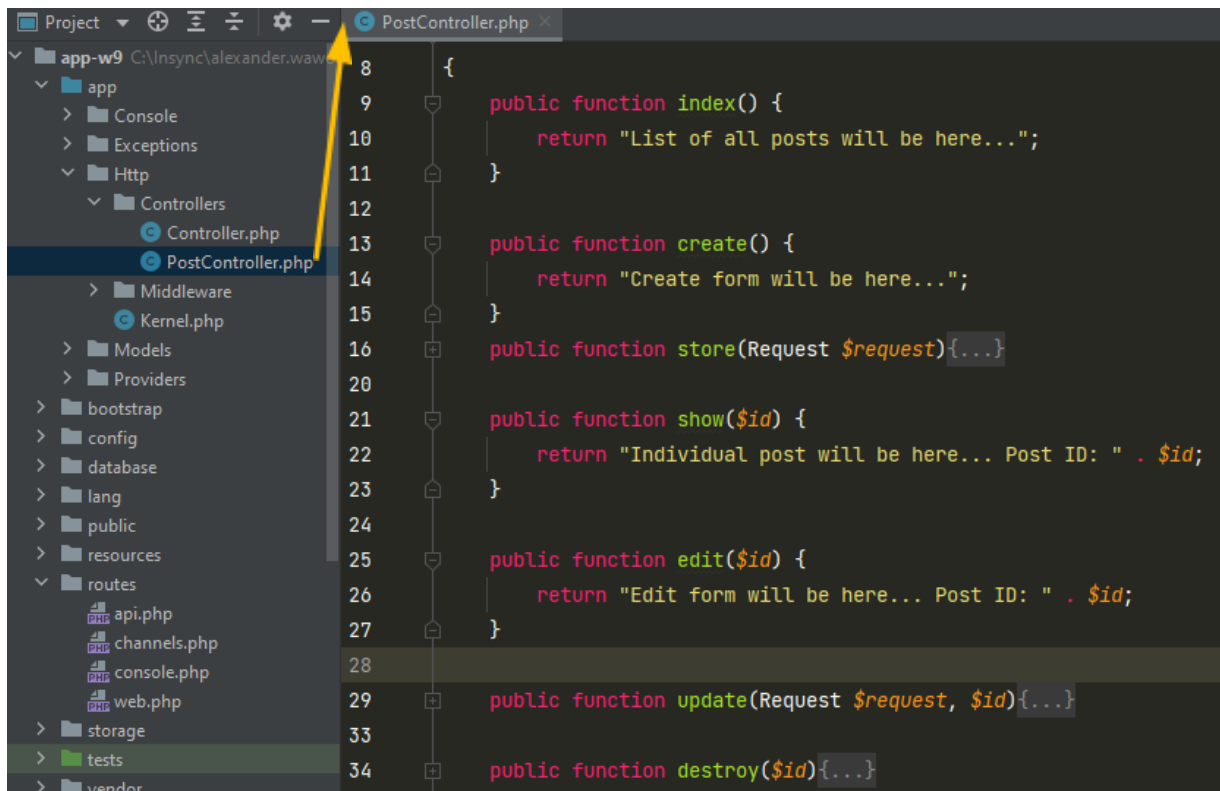


Hello, this is from index() function in PostController, this is post no: 34

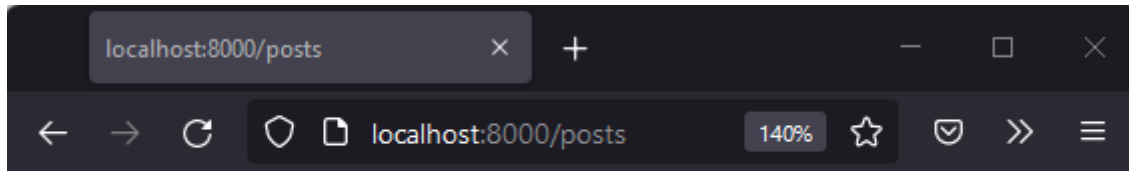
4. Ubah kode pada routes/web.php menjadi seperti berikut. Routes berikut ini merupakan routes standar yang biasa digunakan untuk melakukan operasi CRUD.



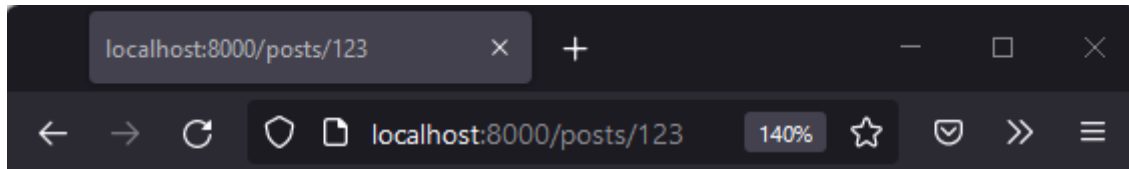
5. Ubah kode pada PostController.php menjadi seperti berikut:



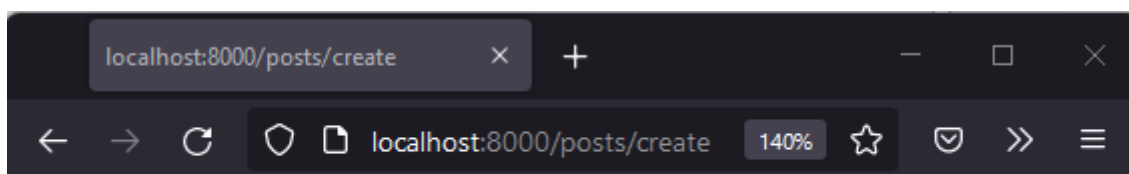
6. Cobalah di browser semua route get yang sudah di definisikan.



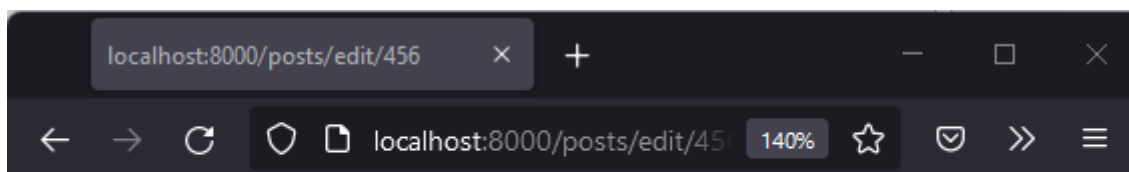
List of all posts will be here...



Individual post will be here... Post ID: 123



Create form will be here...



Edit form will be here... Post ID: 456

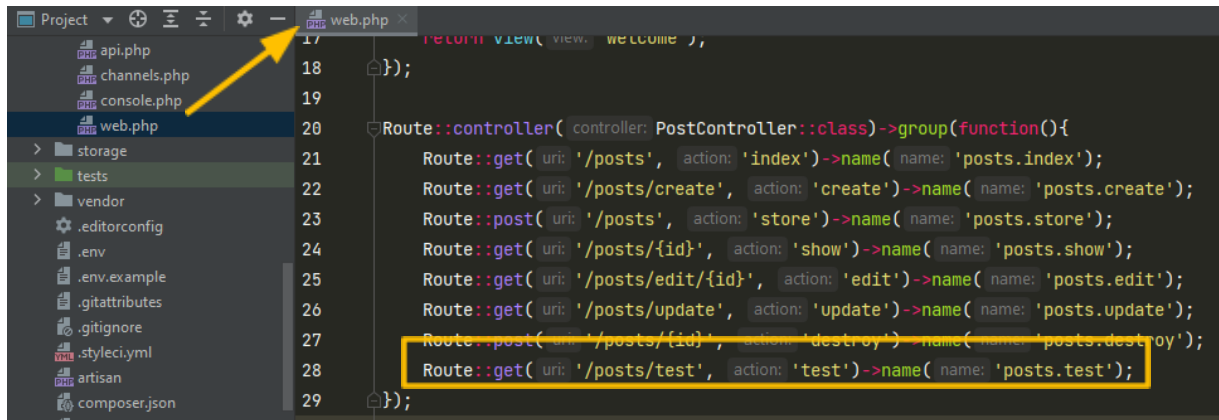
7. Jalankan command berikut untuk melihat semua list yang sudah didefinisikan:

```
PS C:\Insync\alexander.waworuntu\Lecturer.umn.ac.id\DELL-PC\Documents\CODE\Genap2021-22\Pemrograman-Web\Week-09\app-w9> php artisan route:list

GET|HEAD / ..... ignition.executeSolution > Spatie\LaravelIgnition > ExecuteSolutionController
POST _ignition/execute-solution ..... ignition.executeSolution > Spatie\LaravelIgnition > ExecuteSolutionController
GET|HEAD _ignition/health-check ..... ignition.healthCheck > Spatie\LaravelIgnition > HealthCheckController
POST _ignition/health-check ..... ignition.healthCheck > Spatie\LaravelIgnition > HealthCheckController
POST _ignition/update-config ..... ignition.updateConfig > Spatie\LaravelIgnition > UpdateConfigController
GET|HEAD /posts ..... posts.index > PostController@index
POST /posts ..... posts.store > PostController@store
GET|HEAD /posts/create ..... posts.create > PostController@create
GET|HEAD /posts/edit/{id} ..... posts.edit > PostController@edit
GET|HEAD /posts/update ..... posts.update > PostController@update
GET|HEAD /posts/{id} ..... posts.show > PostController@show
POST /posts/{id} ..... posts.destroy > PostController@destroy
GET|HEAD /sanctum/csrf-cookie ..... Laravel\Sanctum > CsrfCookieController@show
```

Membuat Views dan Custom Method

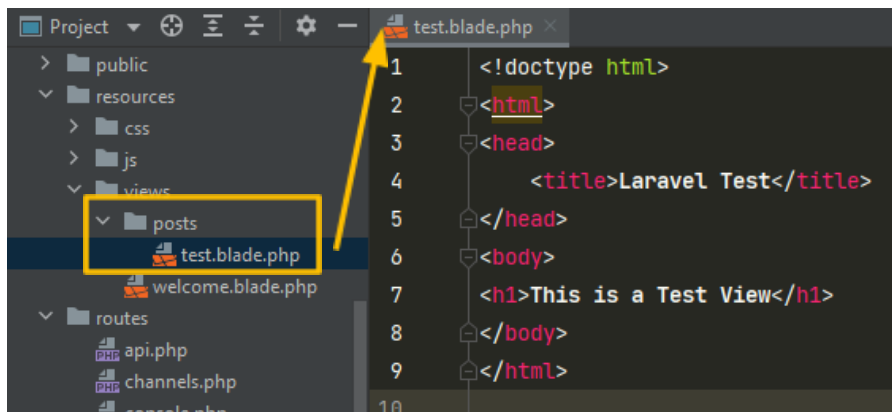
1. Sebagai percobaan awal, tambahkan sebuah route baru sebagai berikut:



```

17 return view( view: 'welcome' );
18 });
19
20 Route::controller( controller: PostController::class )->group(function(){
21     Route::get( uri: '/posts', action: 'index' )->name( name: 'posts.index' );
22     Route::get( uri: '/posts/create', action: 'create' )->name( name: 'posts.create' );
23     Route::post( uri: '/posts', action: 'store' )->name( name: 'posts.store' );
24     Route::get( uri: '/posts/{id}', action: 'show' )->name( name: 'posts.show' );
25     Route::get( uri: '/posts/edit/{id}', action: 'edit' )->name( name: 'posts.edit' );
26     Route::get( uri: '/posts/update', action: 'update' )->name( name: 'posts.update' );
27     Route::post( uri: '/posts/{id}', action: 'destroy' )->name( name: 'posts.destroy' );
28     Route::get( uri: '/posts/test', action: 'test' )->name( name: 'posts.test' );
29 });
  
```

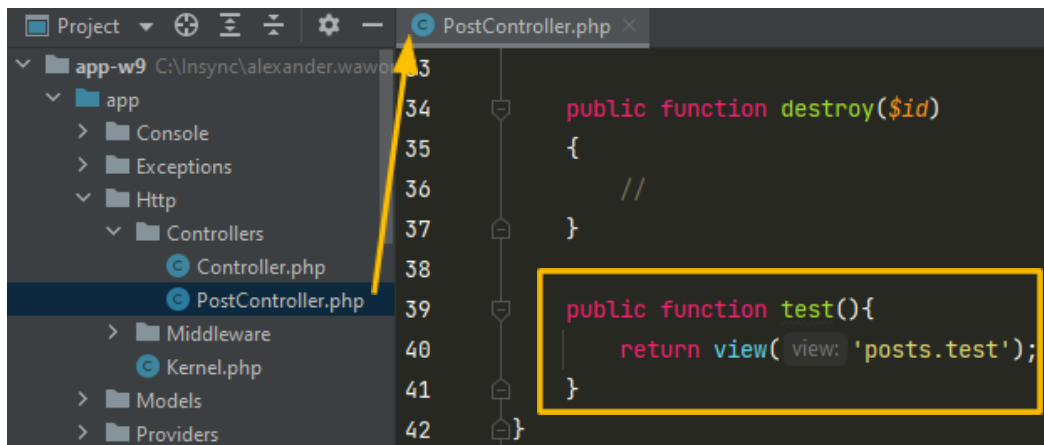
2. Buatlah sebuah file posts/test.blade.php pada folder resources/views



```

1 <!doctype html>
2 <html>
3 <head>
4     <title>Laravel Test</title>
5 </head>
6 <body>
7     <h1>This is a Test View</h1>
8 </body>
9 </html>
10
  
```

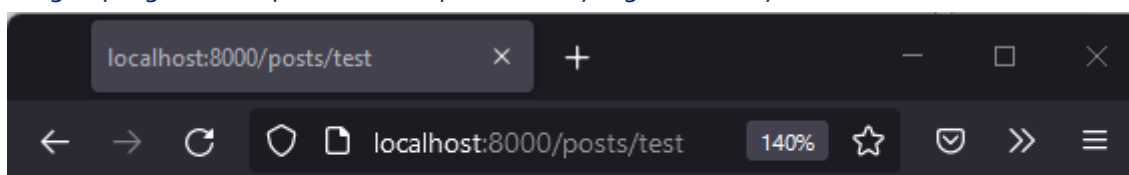
3. Tambahkan kode berikut pada PostController.php



```

33
34 public function destroy($id)
35 {
36     //
37 }
38
39 public function test(){
40     return view( view: 'posts.test' );
41 }
42
  
```

4. Coba hasilnya pada browser. (<http://localhost/posts/test>) Perhatikan bahwa hasilnya tidak sesuai dengan yang kita harapkan (menampilkan view yang sebelumnya dibuat).



Individual post will be here... Post ID: test

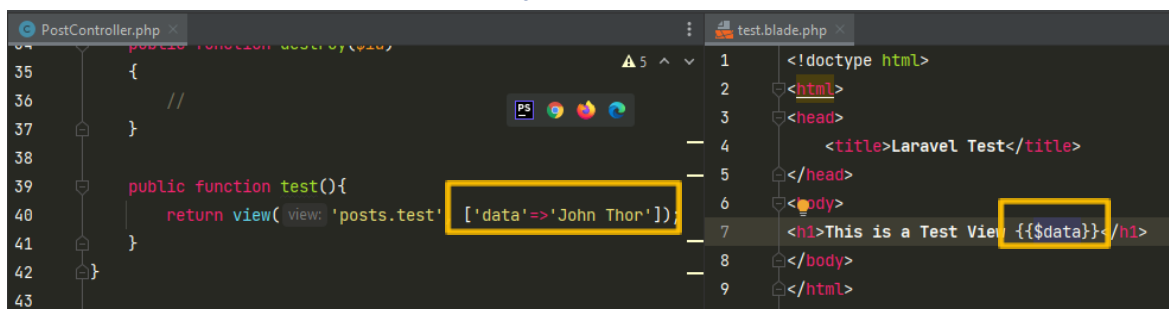
5. Untuk memperbaikinya, pindahkan posisi route `/posts/test` pada baris sebelum `/posts/{id}`.



```

18 });
19
20 Route::controller( controller: PostController::class )->group(function(){
21     Route::get( uri: '/posts', action: 'index')->name( name: 'posts.index');
22     Route::get( uri: '/posts/create', action: 'create')->name( name: 'posts.create');
23     Route::post( uri: '/posts', action: 'store')->name( name: 'posts.store');
24
25     Route::get( uri: '/posts/test', action: 'test')->name( name: 'posts.test');
26     Route::get( uri: '/posts/{id}', action: 'show')->name( name: 'posts.show');
27
28     Route::get( uri: '/posts/edit/{id}', action: 'edit')->name( name: 'posts.edit');
29     Route::get( uri: '/posts/update', action: 'update')->name( name: 'posts.update');
30     Route::post( uri: '/posts/{id}', action: 'destroy')->name( name: 'posts.destroy');
31 });
  
```

6. Untuk mengirimkan data dari Controller ke View, digunakan array asosiatif. Modifikasi kode `PostController.php` dan `test.blade.php` menjadi seperti berikut:



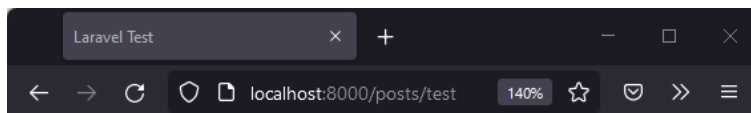
```

35 {
36     //
37 }
38
39 public function test(){
40     return view( view: 'posts.test' , [ 'data'=>'John Thor' ] );
41 }
42
43
  
```

```

1 <!doctype html>
2 <html>
3 <head>
4     <title>Laravel Test</title>
5 </head>
6 <body>
7     <h1>This is a Test View {{ $data }}</h1>
8 </body>
9 </html>
  
```

7. Coba hasilnya di browser



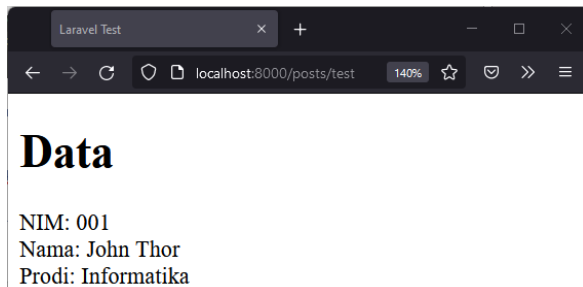
This is a Test View John Thor

8. Untuk mengirimkan lebih dari satu data:

```
PostController.php | test.blade.php
37 | }
38 |
39 | public function test()
40 | {
41 |     return view('posts.test',
42 |         [
43 |             'nim' => '001',
44 |             'nama' => 'John Thor',
45 |             'prodi' => 'Informatika'
46 |         ]
47 |     );
48 | }

4 | <title>Laravel Test</title>
5 | </head>
6 | <body>
7 |
8 | <h1>Data</h1>
9 | NIM: {{$nim}} <br />
10 | Nama: {{$nama}} <br />
11 | Prodi: {{$prodi}}
12 |
13 | </body>
```

9. Lihat hasilnya di browser

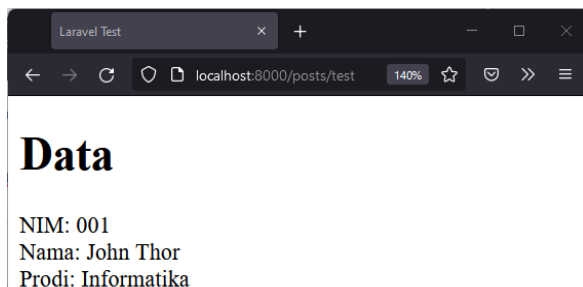


10. Cara penulisan lain untuk mengirimkan data:

```
PostController.php | test.blade.php
37 | }
38 |
39 | public function test(){
40 |     return view('posts.test')
41 |         ->with('nim', '001')
42 |         ->with('nama', 'John Thor')
43 |         ->with('prodi', 'Informatika');
44 | }
45 | }

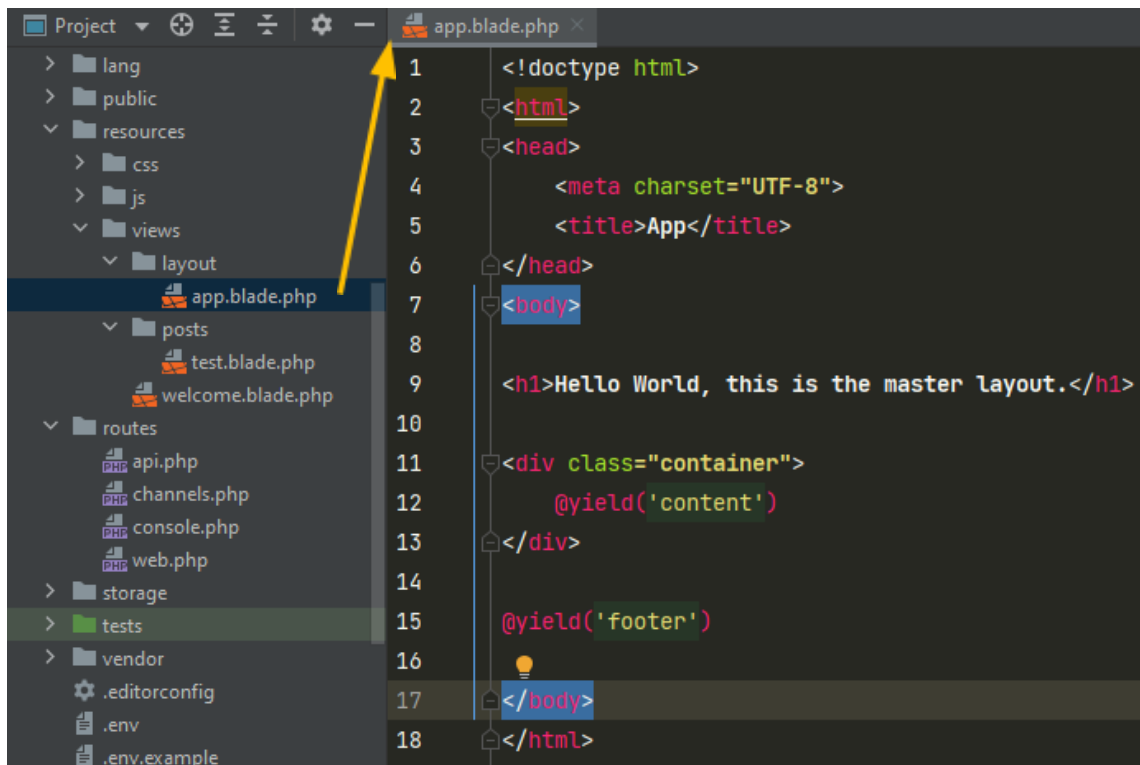
6 | <body>
7 |
8 | <h1>Data</h1>
9 | NIM: {{$nim}} <br />
10 | Nama: {{$nama}} <br />
11 | Prodi: {{$prodi}}
12 |
13 | </body>
14 | </html>
```

11. Lihat hasilnya di browser:



Blade Templating Engine: Master Layout Setup

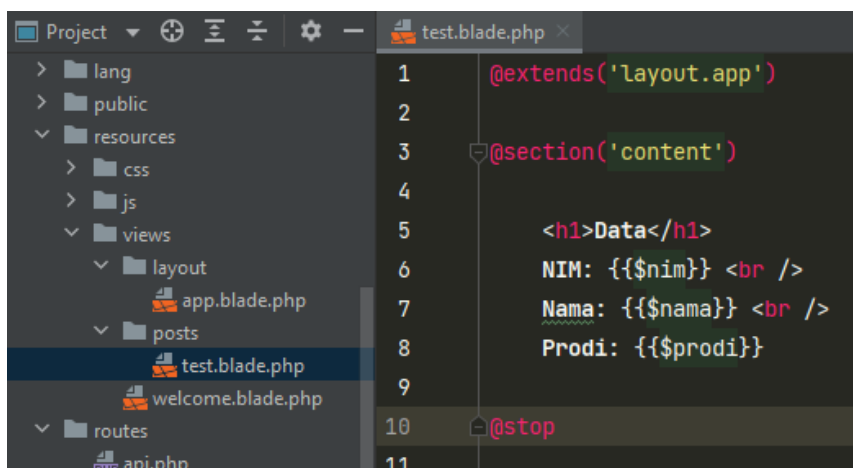
1. Buat folder file layout/app.blade.php dalam folder views, dan tuliskan kode berikut:



```

1 <!doctype html>
2 <html>
3 <head>
4     <meta charset="UTF-8">
5     <title>App</title>
6 </head>
7 <body>
8
9     <h1>Hello World, this is the master layout.</h1>
10
11     <div class="container">
12         @yield('content')
13     </div>
14
15     @yield('footer')
16
17 </body>
18 </html>
  
```

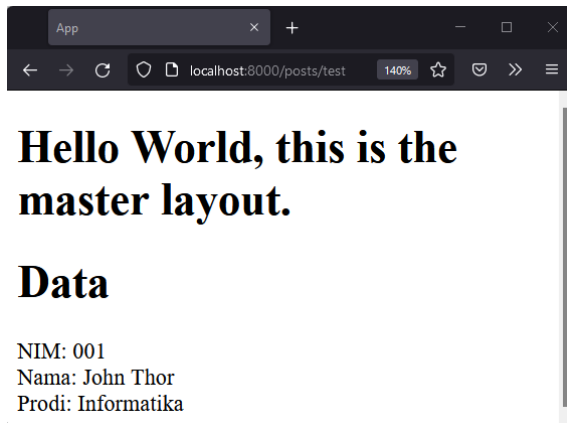
2. Ubah kode pada posts/test.blade.php menjadi seperti berikut



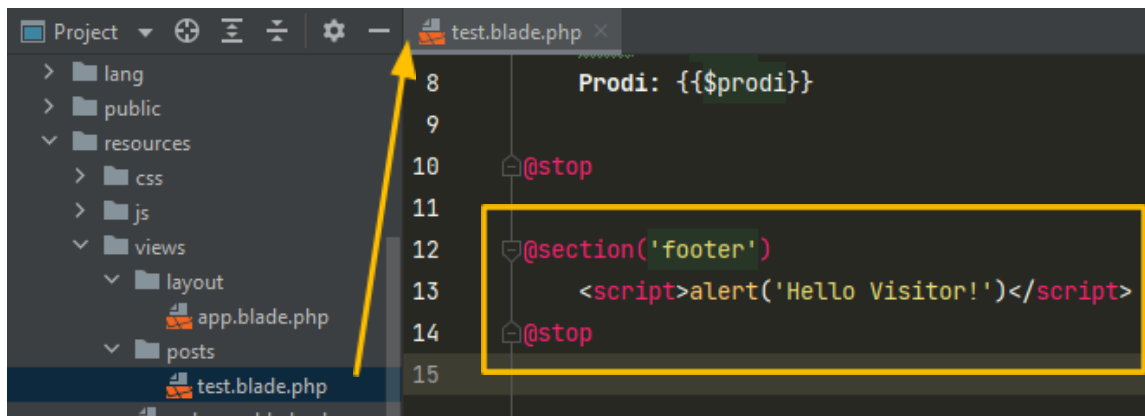
```

1 @extends('layout.app')
2
3 @section('content')
4
5     <h1>Data</h1>
6     NIM: {{$nim}} <br />
7     Nama: {{$nama}} <br />
8     Prodi: {{$prodi}}
9
10 @stop
11
  
```

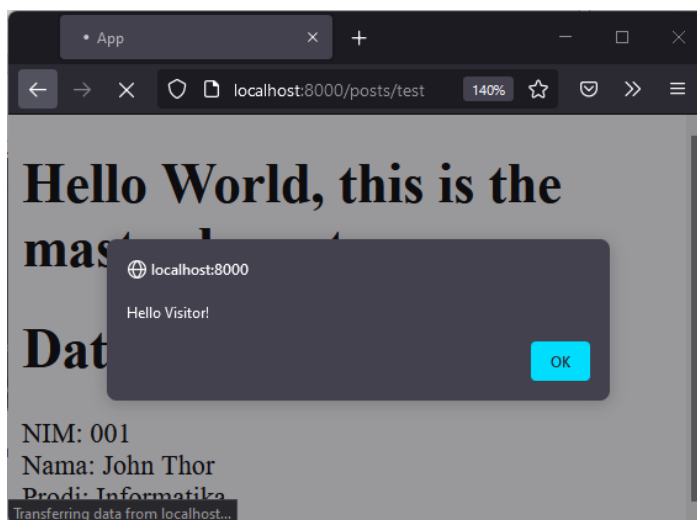
3. Coba hasilnya di browser



4. Tambahkan kode berikut pada test.blade.php

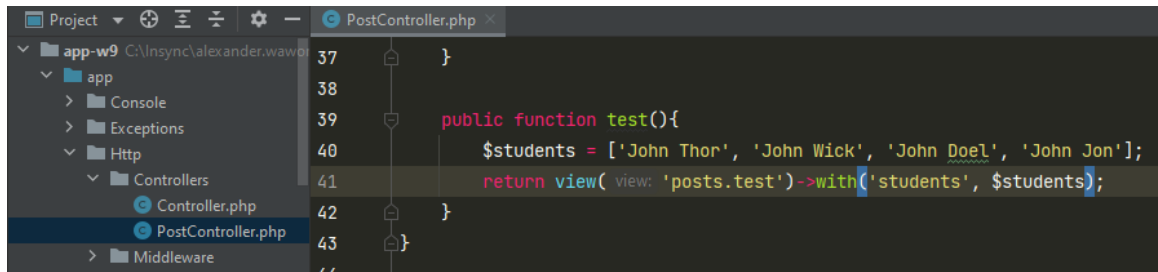


5. Lihat hasilnya di browser



Blade Features

1. Tambahkan kode berikut pada PostController.php

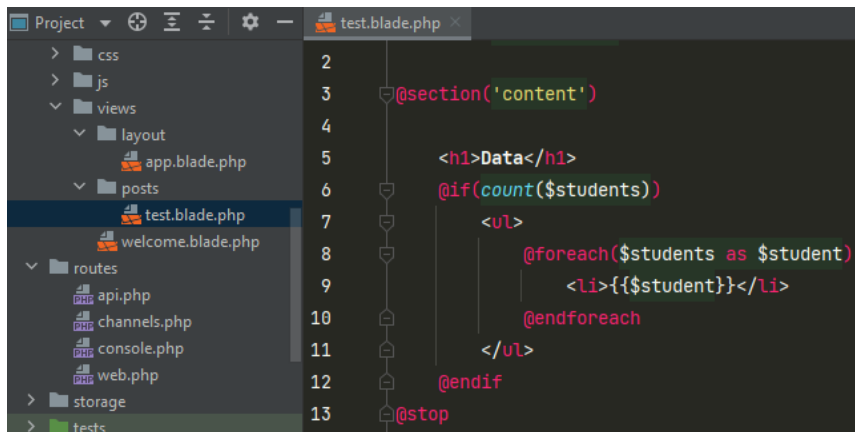


```

37     }
38
39     public function test(){
40         $students = ['John Thor', 'John Wick', 'John Doel', 'John Jon'];
41         return view('posts.test')->with('students', $students);
42     }
43 }

```

2. Modifikasi test.blade.php menjadi seperti berikut

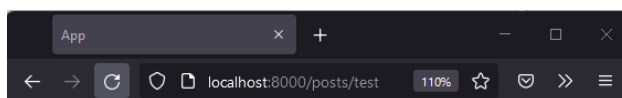


```

2
3 @section('content')
4
5 <h1>Data</h1>
6 @if(count($students))
7     <ul>
8         @foreach($students as $student)
9             <li>{{$student}}</li>
10        @endforeach
11    </ul>
12 @endif
13 @stop

```

3. Lihat hasilnya di browser



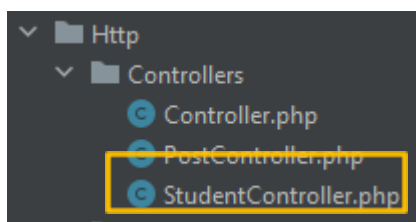
Hello World, this is the master layout.

Data

- John Thor
- John Wick
- John Doel
- John Jon

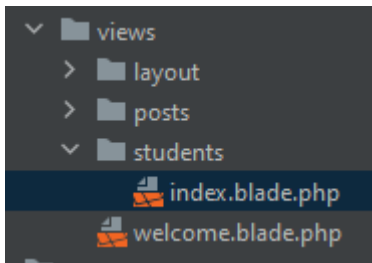
TUGAS

1. Buat sebuah controller dengan nama StudentController.

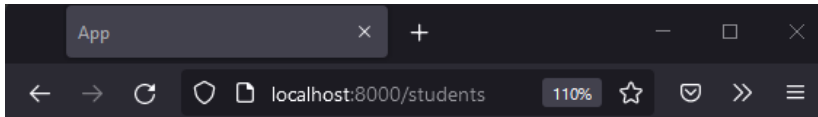


2. Buat sebuah route '/students' dengan method GET yang menggunakan function index() pada StudentController.

3. Buat file views/students/index.blade.php



4. Tuliskan kode pada function index() pada StudentController.php untuk menampilkan view students/index.blade.php. Kemudian buat variable data yang berisi data beberapa mahasiswa (lebih dari 1) yang berisi nim, nama dan prodi untuk dikirimkan ke view.
5. Tuliskan kode pada students/index.blade.php untuk menampilkan data mahasiswa tersebut, seperti berikut:



Hello World, this is the master layout.

Data Mahasiswa

NIM	Nama	Prodi
001	John Thor	Informatika
002	John Wick	Sistem Informasi
003	John Does	Teknik Komputer

MINGGU 10

MODEL & MIGRATION

DESKRIPSI TEMA

Mahasiswa mempraktikkan proses CRUD menggunakan framework Laravel dengan memanfaatkan Model dan Migration.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menerapkan Model dan Migration menggunakan Framework Laravel

PERALATAN YANG DIGUNAKAN

1. XAMPP
2. Code Editor
3. Browser
4. Composer

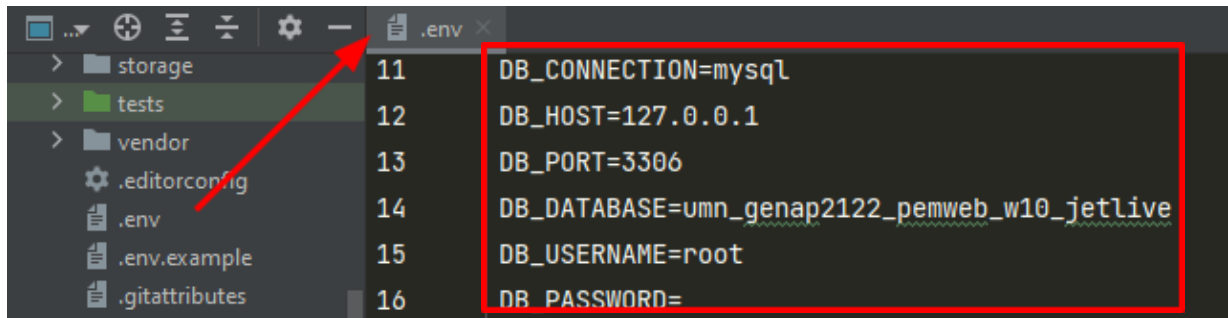
LANGKAH-LANGKAH PRAKTIKUM

Database – Laravel Migration

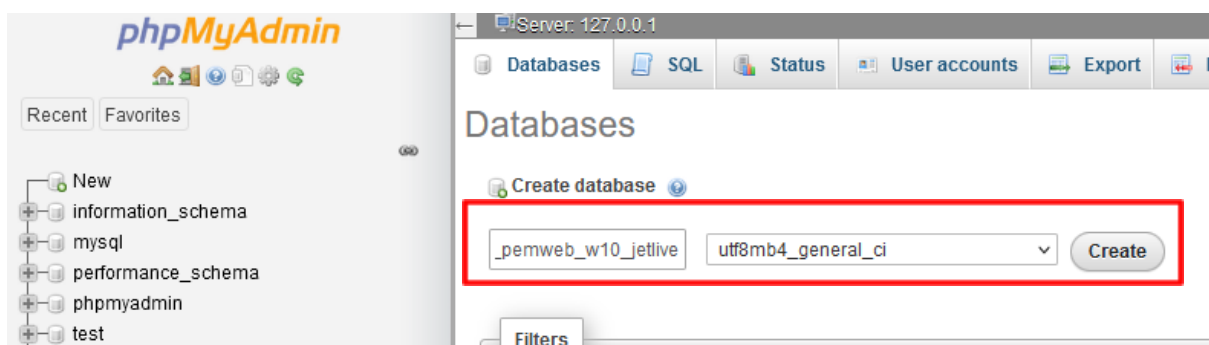
Kita akan membuat table database melalui migration, dimana table dan kolom database dideklarasikan pada kode program yang kemudian dengan menggunakan command migration untuk membuat table tersebut pada database MySQL.

Migrations

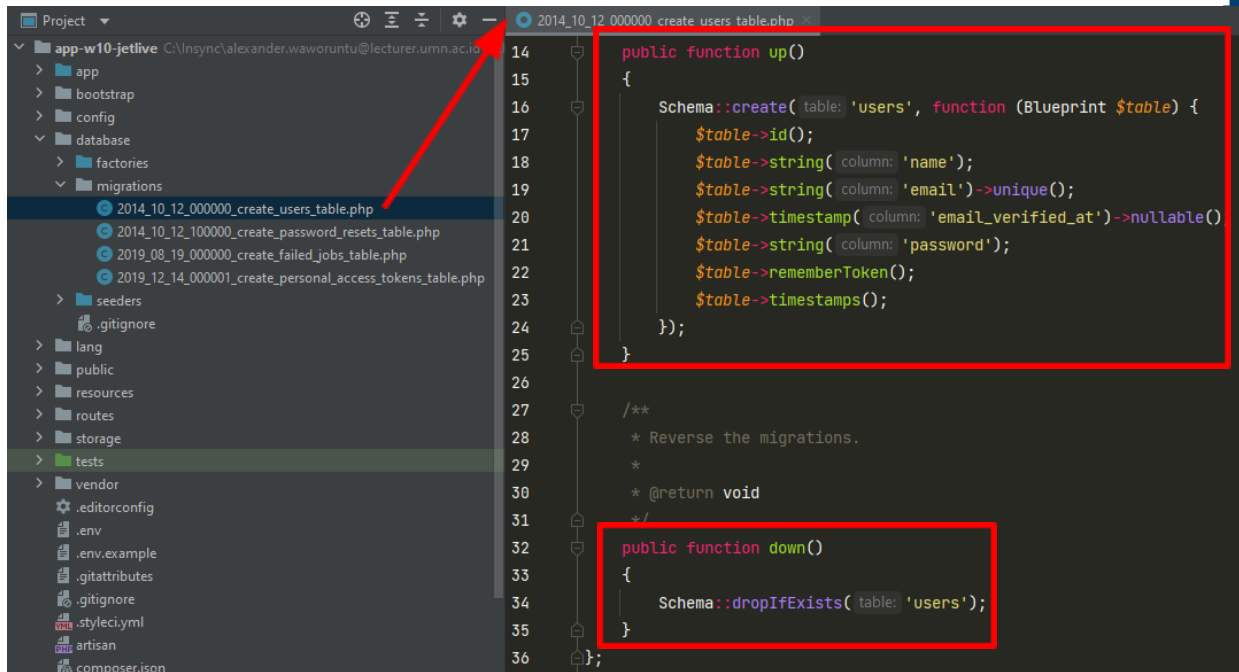
1. Sebelum menggunakan database, kita lakukan setting terlebih dulu informasi database yang akan digunakan pada file .env. Ubah kode .env seperti contoh pada gambar berikut:



2. Buka phpMyAdmin dan buat database baru dengan nama sesuai dengan yang dituliskan pada file .env.



3. Perhatikan folder database/migrations, terdapat empat file migrations yang sudah ada secara default.



```
2014_10_12_000000_create_users_table.php
14 public function up()
15 {
16     Schema::create( table: 'users', function (Blueprint $table) {
17         $table->id();
18         $table->string( column: 'name');
19         $table->string( column: 'email')->unique();
20         $table->timestamp( column: 'email_verified_at')->nullable();
21         $table->string( column: 'password');
22         $table->rememberToken();
23         $table->timestamps();
24     });
25 }
26
27 /**
28  * Reverse the migrations.
29  *
30  * @return void
31  */
32 public function down()
33 {
34     Schema::dropIfExists( table: 'users');
35 }
36 };
```

Sebagai contoh, coba buka file ..._create_users_table.php. Terdapat dua fungsi, yaitu up() dan down(). Fungsi up() untuk membuat table dan fungsi down() untuk menghapus table.

4. Buka terminal, kemudian jalankan command php artisan migrate

```
PS C:\Insync\alexander.waworuntu@Lecturer.umh.ac.id\DELL-PC\Documents\CODE\Genap2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan migrate
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (61.77ms)
Migrating: 2019_12_14_000001_create_personal_access_tokens_table
Migrated: 2019_12_14_000001_create_personal_access_tokens_table (83.72ms)
```

5. Buka phpMyAdmin, perhatikan database yang sebelumnya dibuat. Setelah migration, kini terdapat 5 tabel.

Server: 127.0.0.1 Database: umn_genap2122_pemweb_w10_jetlive

Structure SQL Search Query Export Import

Filters

Containing the word:

Table	Action
☐ failed_jobs	★ Browse Structure Search Insert
☐ migrations	★ Browse Structure Search Insert
☐ password_resets	★ Browse Structure Search Insert
☐ personal_access_tokens	★ Browse Structure Search Insert
☐ users	★ Browse Structure Search Insert
5 tables	Sum

6. Coba perhatikan struktur table users dan bandingkan dengan kode yang terdapat pada fungsi up() yang tadi kita lihat.

#	Name	Type	Collation
1	id	bigint(20)	
2	name	varchar(255) utf8mb4_unicode_ci	
3	email	varchar(255) utf8mb4_unicode_ci	
4	email_verified_at	timestamp	
5	password	varchar(255) utf8mb4_unicode_ci	
6	remember_token	varchar(100) utf8mb4_unicode_ci	
7	created_at	timestamp	
8	updated_at	timestamp	


```

14 public function up()
15 {
16     Schema::create( table: 'users', function (Blueprint $table) {
17         $table->id();
18         $table->string( column: 'name');
19         $table->string( column: 'email')->unique();
20         $table->timestamp( column: 'email_verified_at')->nullable();
21         $table->string( column: 'password');
22         $table->rememberToken();
23         $table->timestamps();
24     });
  
```

Method timestamp() akan membuat dua kolom pada table database, yaitu created_at dan updated_at.

Create Migration dan Drop Migration

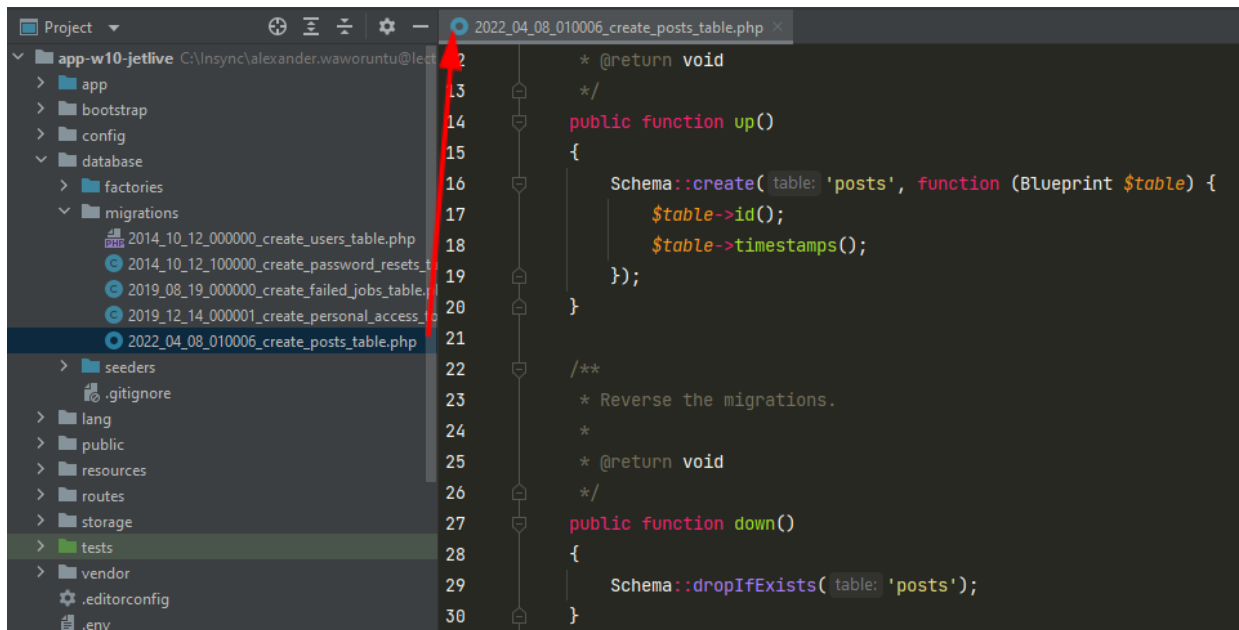
1. Melalui terminal, ketikkan command berikut untuk membuat migration file.

```

ap2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan make:migration
create_posts_table --create="posts"
Created Migration: 2022_04_08_010006_create_posts_table
  
```

nama table

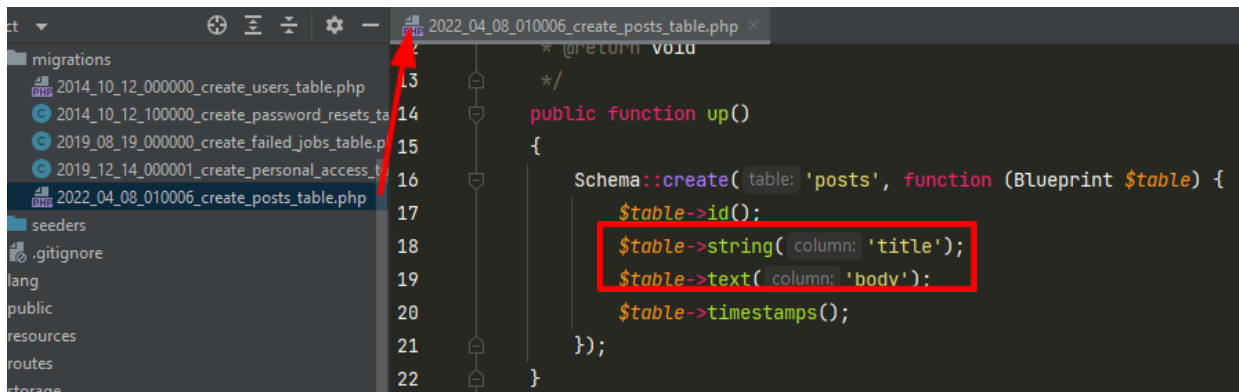
2. Migration file otomatis dibuatkan.



```

1  * @return void
2  */
3
4  public function up()
5  {
6      Schema::create( table: 'posts', function (Blueprint $table) {
7          $table->id();
8          $table->timestamps();
9      });
10 }
11
12 /**
13  * Reverse the migrations.
14  *
15  * @return void
16  */
17 public function down()
18 {
19     Schema::dropIfExists( table: 'posts');
20 }
  
```

3. Tambahkan kode berikut untuk memodifikasi struktur table dasar yang sudah dibuat secara default.



```

13  * @return void
14  */
15
16 public function up()
17 {
18     Schema::create( table: 'posts', function (Blueprint $table) {
19         $table->id();
20         $table->string( column: 'title');
21         $table->text( column: 'body');
22         $table->timestamps();
23     });
24 }
  
```

Kode tersebut akan membuat table posts dengan struktur:

id	INT	AUTO_INCREMENT
title	VARCHAR	
body	VARCHAR	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

4. Lakukan migrate dengan menjalankan command berikut:

```

ograman-Web\Week-10\app-w10-jetlive> php artisan migrate
Migrating: 2022_04_08_010006_create_posts_table
Migrated: 2022_04_08_010006_create_posts_table (36.57ms)
  
```

5. Buka phpMyAdmin dan cek apakah table posts sudah berhasil dibuat:

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetlive » Table: posts

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)

[Table structure](#)
[Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT
☐ 2	title	varchar(255)	utf8mb4_unicode_ci		No	None		
☐ 3	body	text	utf8mb4_unicode_ci		No	None		
☐ 4	created_at	timestamp			Yes	NULL		
☐ 5	updated_at	timestamp			Yes	NULL		

6. Misalnya jika kita ingin membatalkan migration yang baru saja dilakukan, ketikkan command berikut:

```

ograman-Web\Week-10\app-w10-jetlive> php artisan migrate:rollback
Rolling back: 2022_04_08_010006_create_posts_table
Rolled back: 2022_04_08_010006_create_posts_table (20.03ms)
  
```

7. Migration terakhir yang kita lakukan adalah membuat table posts, ketika di-rollback, maka akan menghapus table posts. Buka phpMyAdmin dan cek apakah table posts sudah dihapus.

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetlive

[Structure](#)
[SQL](#)
[Search](#)
[Query](#)
[Export](#)
[Import](#)

Filters

Containing the word:

Table	Action
☐ failed_jobs	★ Browse Structure Search Ins
☐ migrations	★ Browse Structure Search Ins
☐ password_resets	★ Browse Structure Search Ins
☐ personal_access_tokens	★ Browse Structure Search Ins
☐ users	★ Browse Structure Search Ins
5 tables	Sum

8. Misalnya kita ingin mengubah salah satu nama kolom pada tabel posts, dari "body" menjadi "content". Ubah file migration posts menjadi seperti berikut:



```

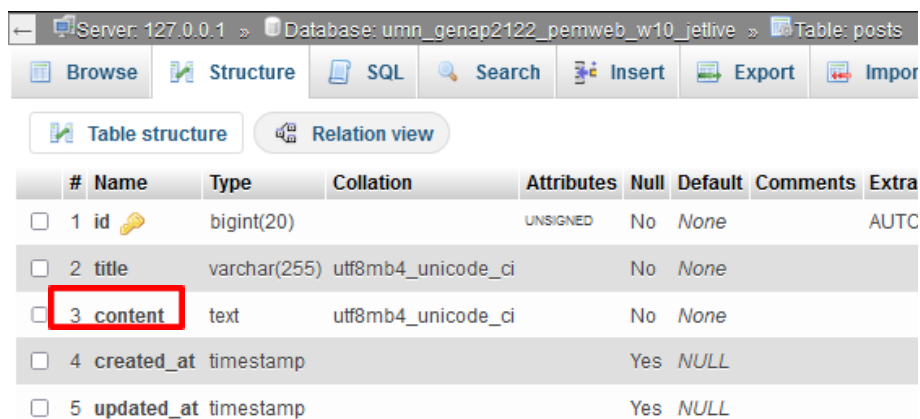
2022_04_08_010006_create_posts_table.php
13
14
15
16 Schema::create( table: 'posts', function (Blueprint $table) {
17     $table->id();
18     $table->string( column: 'title');
19     $table->text( column: 'content');
20     $table->timestamps();
21 });
22
  
```

9. Lakukan migrate dengan command php artisan

```

ograman-Web\Week-10\app-w10-jetlive> php artisan migrate
Migrating: 2022_04_08_010006_create_posts_table
Migrated: 2022_04_08_010006_create_posts_table (33.30ms)
  
```

10. Buka phpMyAdmin untuk melihat hasilnya.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	bigint(20)		UNSIGNED	No	None		AUTC
2	title	varchar(255)	utf8mb4_unicode_ci		No	None		
3	content	text	utf8mb4_unicode_ci		No	None		
4	created_at	timestamp			Yes	NULL		
5	updated_at	timestamp			Yes	NULL		

Menambahkan Kolom pada Tabel yang Sudah Ada

1. Seringkali kita ingin mengubah struktur tabel database yang sudah ada tanpa harus menghapus tabel tersebut, misalnya ketika sudah ada data pada tabel tersebut. Sebagai contoh, kita akan membuat migration file untuk menambahkan kolom is_admin pada tabel posts. Ketikkan command berikut pada terminal:

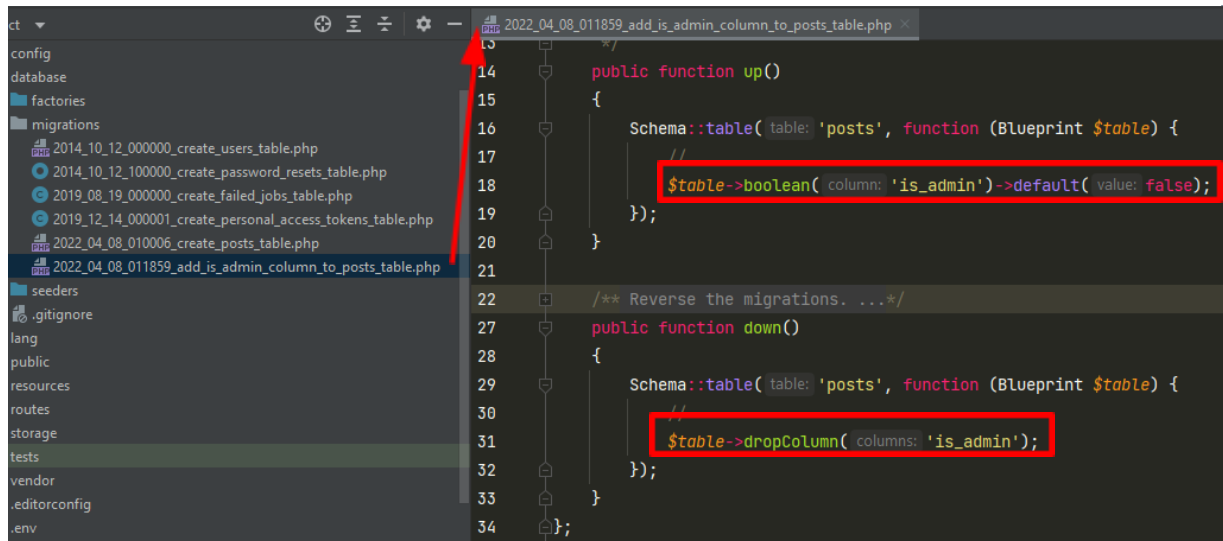
```

ap2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan make:migration
add_is_admin_column_to_posts_table --table=posts
Created Migration: 2022_04_08_011859_add_is_admin_column_to_posts_table
  
```

Nama migration file

Nama tabel

2. Migration file otomatis dibuat. Ubah kode pada migration file tersebut menjadi seperti berikut:



```

14 public function up()
15 {
16     Schema::table('posts', function (Blueprint $table) {
17         //
18         $table->boolean('is_admin')->default(value: false);
19     });
20 }
21
22 /** Reverse the migrations. ... */
23
24 public function down()
25 {
26     Schema::table('posts', function (Blueprint $table) {
27         //
28         $table->dropColumn('is_admin');
29     });
30 }
31
32 }
33
34 }

```

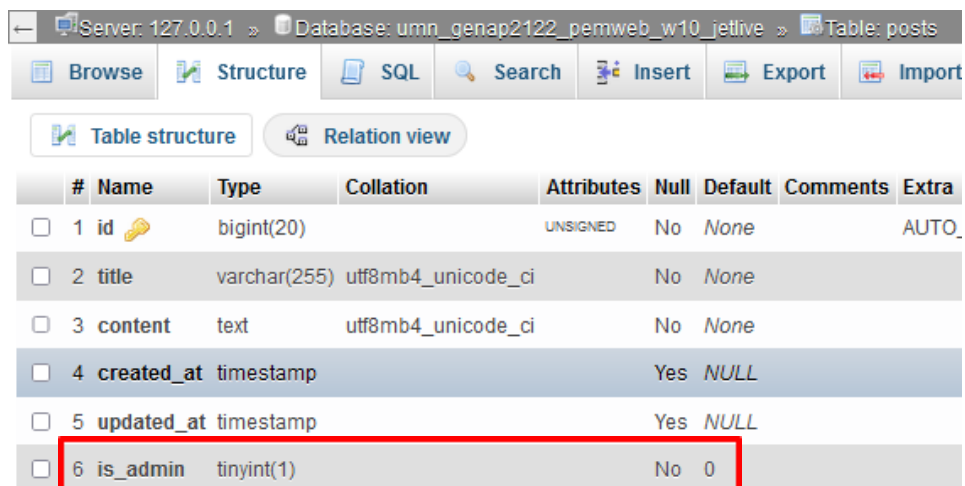
3. Lakukan migrate melalui command php artisan

```

ap2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan migrate
Migrating: 2022_04_08_011859_add_is_admin_column_to_posts_table
Migrated: 2022_04_08_011859_add_is_admin_column_to_posts_table (19.22ms)

```

4. Lihat hasilnya melalui phpMyAdmin



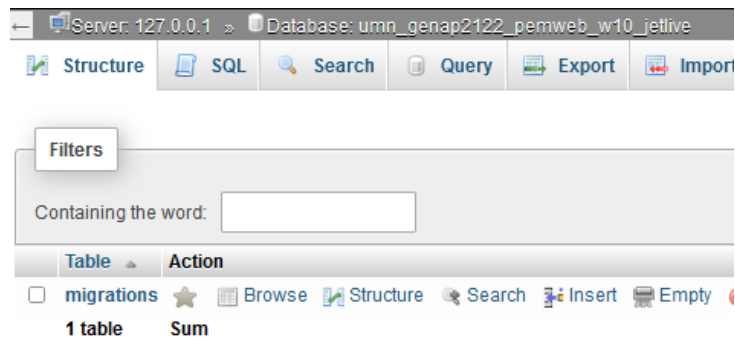
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT
2	title	varchar(255)	utf8mb4_unicode_ci		No	None		
3	content	text	utf8mb4_unicode_ci		No	None		
4	created_at	timestamp			Yes	NULL		
5	updated_at	timestamp			Yes	NULL		
6	is_admin	tinyint(1)			No	0		

Beberapa Command Migrations Lainnya

1. php artisan migrate:reset
untuk menghapus semua tabel pada database (drop all migrations)

```
roggraman-Web\Week-10\app-w10-jetlive> php artisan migrate:reset
Rolling back: 2022_04_08_011859_add_is_admin_column_to_posts_table
Rolled back: 2022_04_08_011859_add_is_admin_column_to_posts_table (12.72ms)
Rolling back: 2022_04_08_010006_create_posts_table
Rolled back: 2022_04_08_010006_create_posts_table (18.08ms)
Rolling back: 2019_12_14_000001_create_personal_access_tokens_table
Rolled back: 2019_12_14_000001_create_personal_access_tokens_table (14.38ms)
Rolling back: 2019_08_19_000000_create_failed_jobs_table
Rolled back: 2019_08_19_000000_create_failed_jobs_table (16.89ms)
Rolling back: 2014_10_12_100000_create_password_resets_table
Rolled back: 2014_10_12_100000_create_password_resets_table (16.95ms)
Rolling back: 2014_10_12_000000_create_users_table
Rolled back: 2014_10_12_000000_create_users_table (14.49ms)
```

2. Buka phpMyAdmin untuk melihat hasilnya



3. php artisan migrate:status

Untuk melihat status dari setiap file migration apakah sudah dijalankan atau belum

```
rpgman-Web\Week-10\app-w10-jetlive> php artisan migrate:status
```

Ran?	Migration	Batch
No	2014_10_12_000000_create_users_table	
No	2014_10_12_100000_create_password_resets_table	
No	2019_08_19_000000_create_failed_jobs_table	
No	2019_12_14_000001_create_personal_access_tokens_table	
No	2022_04_08_010006_create_posts_table	
No	2022_04_08_011859_add_is_admin_column_to_posts_table	

Semua status Ran?=No, artinya belum ada file migration yang dijalankan, karena baru saja kita reset.

4. Lakukan migrate lagi dan cek status

```

rograman-Web\Week-10\app-w10-jetlive> php artisan migrate
Migrating: 2014_10_12_000000_create_users_table
Migrated: 2014_10_12_000000_create_users_table (71.76ms)
Migrating: 2014_10_12_100000_create_password_resets_table
Migrated: 2014_10_12_100000_create_password_resets_table (92.20ms)
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (62.45ms)
Migrating: 2019_12_14_000001_create_personal_access_tokens_table
Migrated: 2019_12_14_000001_create_personal_access_tokens_table (87.51ms)
Migrating: 2022_04_08_010006_create_posts_table
Migrated: 2022_04_08_010006_create_posts_table (32.00ms)
Migrating: 2022_04_08_011859_add_is_admin_column_to_posts_table
Migrated: 2022_04_08_011859_add_is_admin_column_to_posts_table (11.07ms)

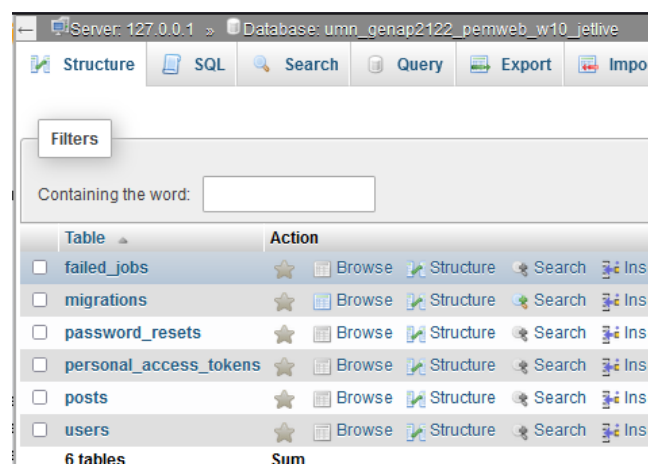
```

```

rograman-Web\Week-10\app-w10-jetlive> php artisan migrate:status
+-----+-----+-----+
| Ran? | Migration                                          | Batch |
+-----+-----+-----+
| Yes  | 2014_10_12_000000_create_users_table              | 1      |
| Yes  | 2014_10_12_100000_create_password_resets_table    | 1      |
| Yes  | 2019_08_19_000000_create_failed_jobs_table        | 1      |
| Yes  | 2019_12_14_000001_create_personal_access_tokens_table | 1      |
| Yes  | 2022_04_08_010006_create_posts_table              | 1      |
| Yes  | 2022_04_08_011859_add_is_admin_column_to_posts_table | 1      |
+-----+-----+-----+

```

5. Lihat hasilnya di phpMyAdmin



6. php artisan migrate:refresh

Untuk melakukan reset dan migrate dalam satu command. Digunakan ketika, misalnya kita melakukan beberapa perubahan pada file migration, daripada melakukan reset dan migrate secara terpisah, dapat dilakukan dengan 1 command saja.

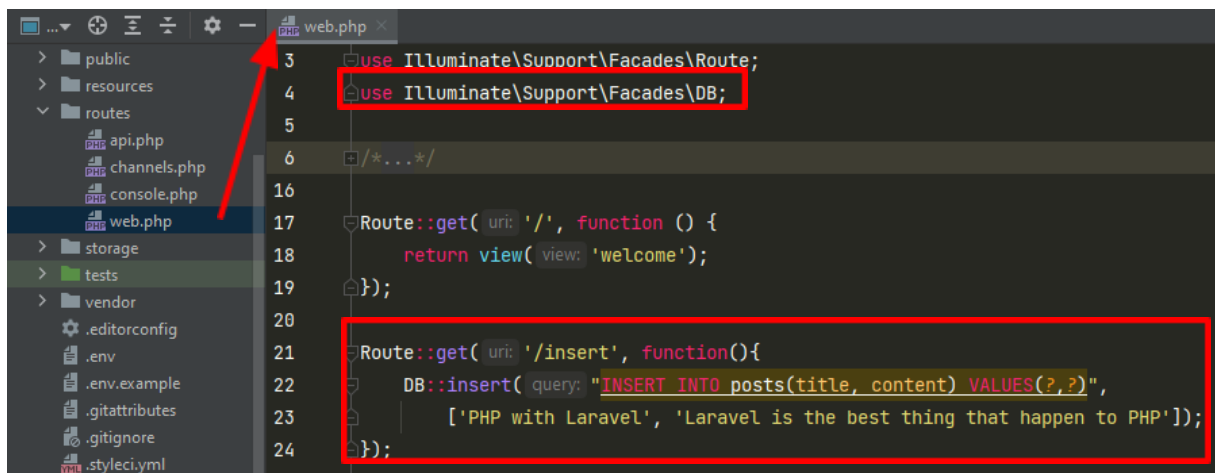
```
rograman-Web\Week-10\app-w10-jetlive> php artisan migrate:refresh
Rolling back: 2022_04_08_011859_add_is_admin_column_to_posts_table
Rolled back: 2022_04_08_011859_add_is_admin_column_to_posts_table (13.03ms)
Rolling back: 2022_04_08_010006_create_posts_table
Rolling back: 2014_10_12_100000_create_password_resets_table
Rolled back: 2014_10_12_100000_create_password_resets_table (13.91ms)
Rolling back: 2014_10_12_000000_create_users_table
Rolled back: 2014_10_12_000000_create_users_table (13.59ms)
Migrating: 2014_10_12_000000_create_users_table
Migrated: 2014_10_12_000000_create_users_table (63.56ms)
Migrating: 2014_10_12_100000_create_password_resets_table
Migrated: 2014_10_12_100000_create_password_resets_table (58.27ms)
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (53.19ms)
Migrating: 2019_12_14_000001_create_personal_access_tokens_table
Migrated: 2019_12_14_000001_create_personal_access_tokens_table (76.15ms)
Migrating: 2022_04_08_010006_create_posts_table
Migrated: 2022_04_08_010006_create_posts_table (31.61ms)
Migrating: 2022_04_08_011859_add_is_admin_column_to_posts_table
Migrated: 2022_04_08_011859_add_is_admin_column_to_posts_table (11.61ms)
```

Raw SQL Queries

Kita akan melakukan manipulasi database menggunakan Raw SQL Statement.

Inserting Data

1. Ketikkan kode berikut pada file routes/web.php

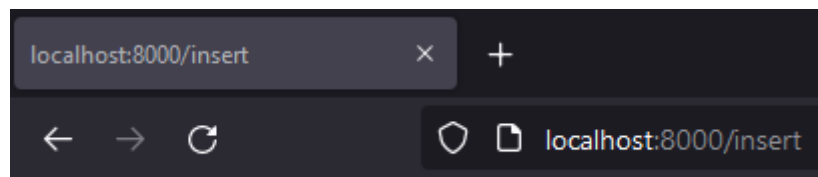


```

3 use Illuminate\Support\Facades\Route;
4 use Illuminate\Support\Facades\DB;
5
6 /*...*/
16
17 Route::get('/', function () {
18     return view('welcome');
19 });
20
21 Route::get('/insert', function(){
22     DB::insert( query: "INSERT INTO posts(title, content) VALUES(?,?)",
23         ['PHP with Laravel', 'Laravel is the best thing that happen to PHP']);
24 });

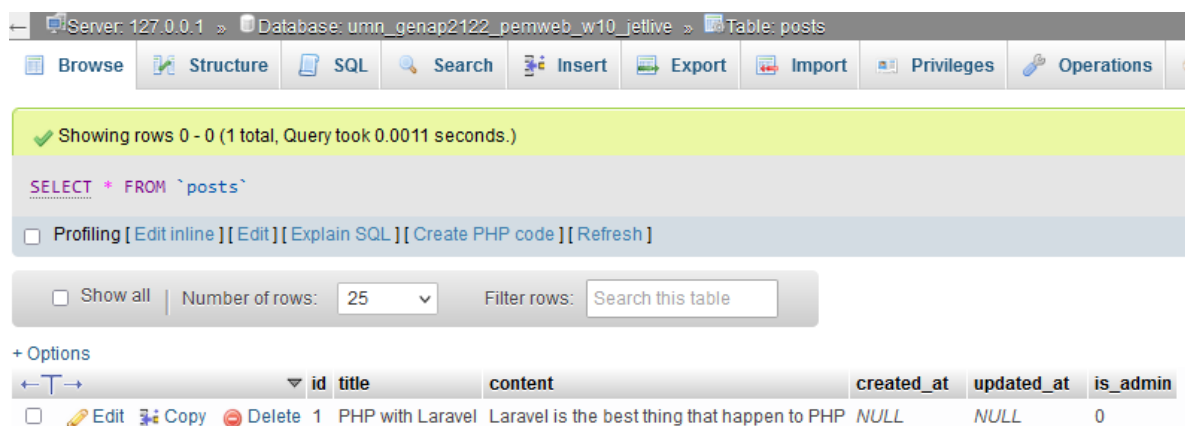
```

2. Jalankan command serve kemudian buka browser dan ketikkan URL: <http://localhost:8000/insert>



Jika tidak ada error, akan tampil halaman kosong.

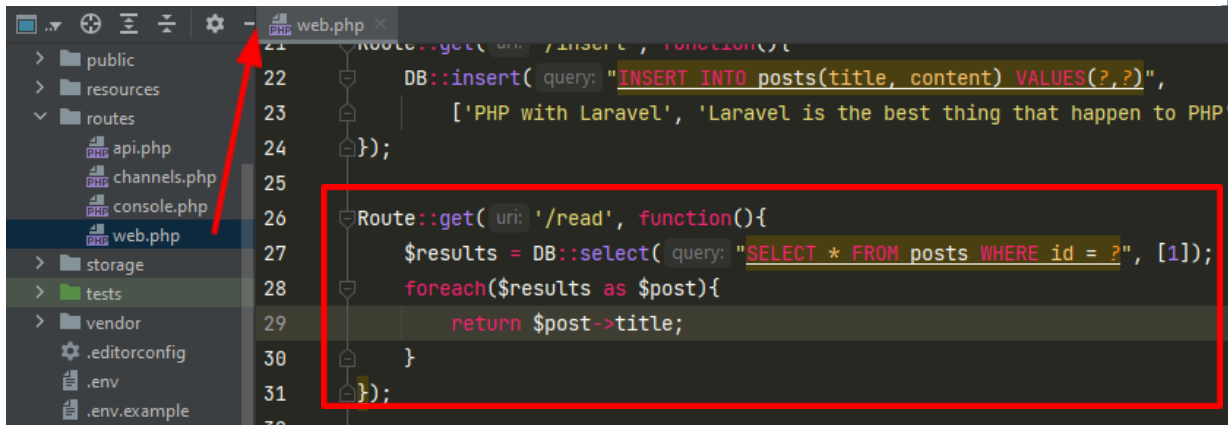
3. Buka phpMyAdmin, dan cek apakah tabel post sudah berisi data.



id	title	content	created_at	updated_at	is_admin
1	PHP with Laravel	Laravel is the best thing that happen to PHP	NULL	NULL	0

Reading Data

1. Ketikkan kode berikut pada file routes/web.php

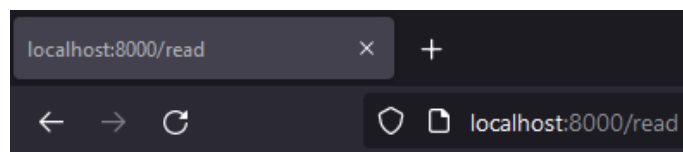


```

21 Route::get('uri', function() {
22     DB::insert( query: "INSERT INTO posts(title, content) VALUES(?,?)",
23         ['PHP with Laravel', 'Laravel is the best thing that happen to PHP']
24     );
25
26     Route::get( uri: '/read', function(){
27         $results = DB::select( query: "SELECT * FROM posts WHERE id = ?", [1]);
28         foreach($results as $post){
29             return $post->title;
30         }
31     });
32
33 }

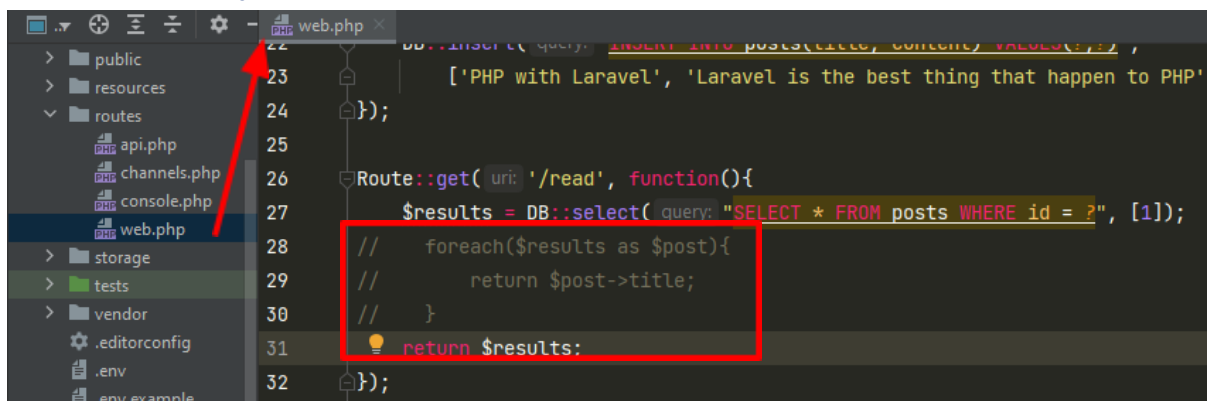
```

2. Buka browser dan jalankan URL: <http://localhost:8000/read>



PHP with Laravel

3. Variable \$results merupakan variable array yang menampung hasil SELECT. Coba ubah kode pada routes/web.php menjadi seperti berikut:

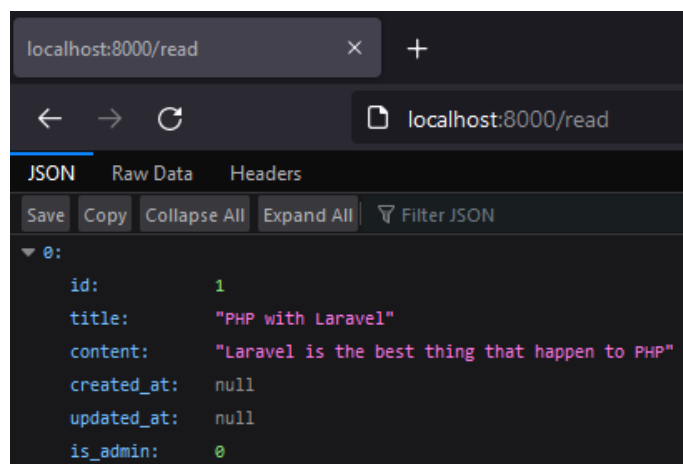


```

22 DB::insert( query: "INSERT INTO posts(title, content) VALUES(?,?)",
23     ['PHP with Laravel', 'Laravel is the best thing that happen to PHP']
24 );
25
26 Route::get( uri: '/read', function(){
27     $results = DB::select( query: "SELECT * FROM posts WHERE id = ?", [1]);
28     // foreach($results as $post){
29     //     return $post->title;
30     // }
31     return $results;
32 }

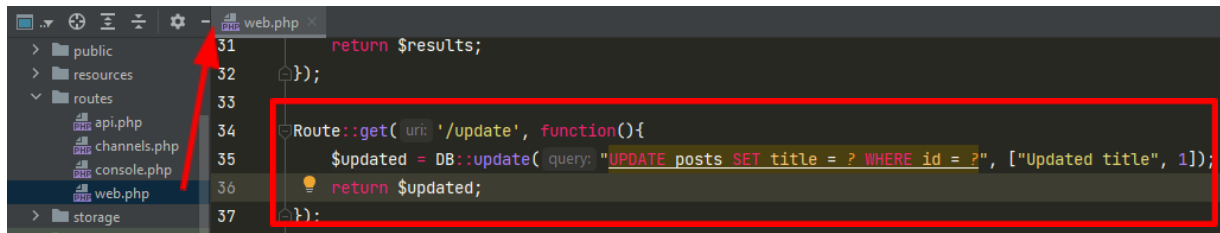
```

4. Lihat hasilnya pada browser



Updating Data

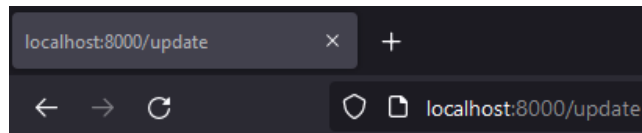
1. Ketikkan kode berikut pada routes/web.php



```

31 return $results;
32 });
33
34 Route::get( uri: '/update', function(){
35     $updated = DB::update( query: "UPDATE posts SET title = ? WHERE id = ?", ["Updated title", 1]);
36     return $updated;
37 });
  
```

2. Jalankan pada browser



1

3. Lihat hasilnya pada phpMyAdmin

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetlive » Table: posts

Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)

SELECT * FROM `posts`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

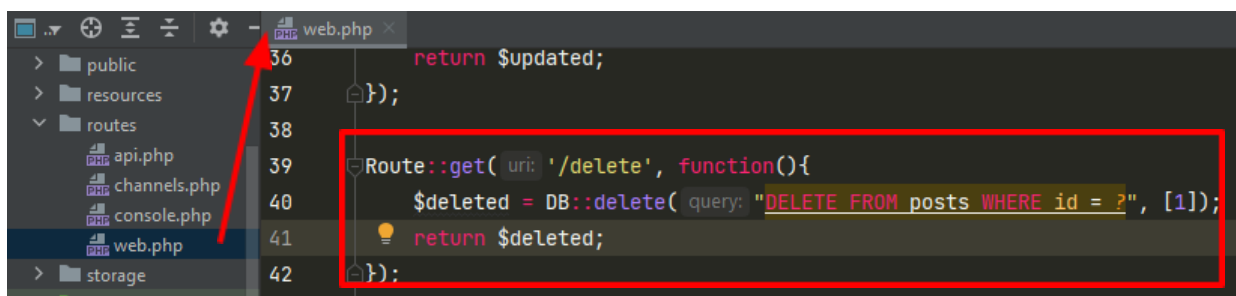
Show all | Number of rows: 25 | Filter rows: Search this table

+ Options

	id	title	content	created_at	updated_at	is_admin
☐ Edit ☐ Copy ☐ Delete	1	Updated title	Laravel is the best thing that happen to PHP	NULL	NULL	0

Deleting Data

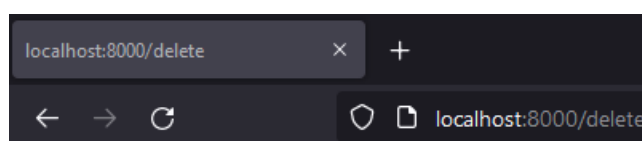
1. Ketikkan kode berikut pada routes/web.php



```

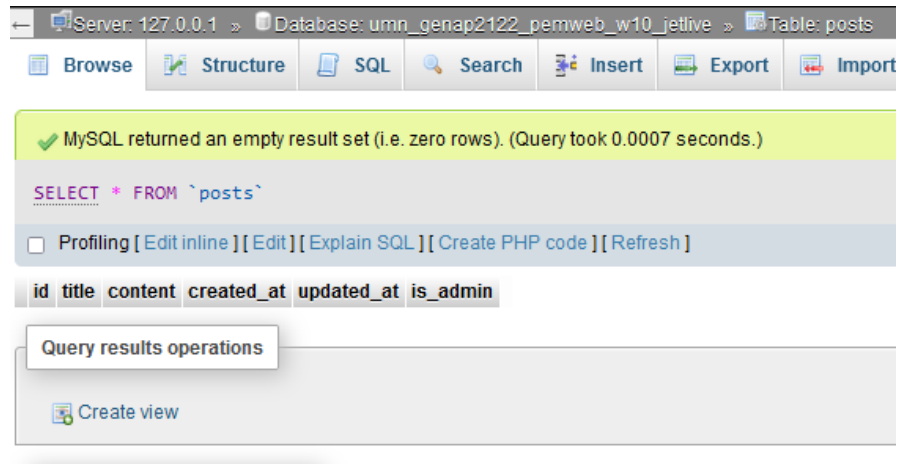
36 return $updated;
37 });
38
39 Route::get( uri: '/delete', function(){
40     $deleted = DB::delete( query: "DELETE FROM posts WHERE id = ?", [1]);
41     return $deleted;
42 });
  
```

2. Jalankan pada browser



1

3. Lihat pada phpMyAdmin, data sudah di hapus.



Latihan:

Coba insert beberapa data dengan cara ini, kemudian tampilkan data yang sudah di-insert tersebut. Update dan delete beberapa data di antaranya.

Database Eloquent/ORM

Kita akan melakukan manipulasi database menggunakan method yang sudah disediakan oleh Laravel.

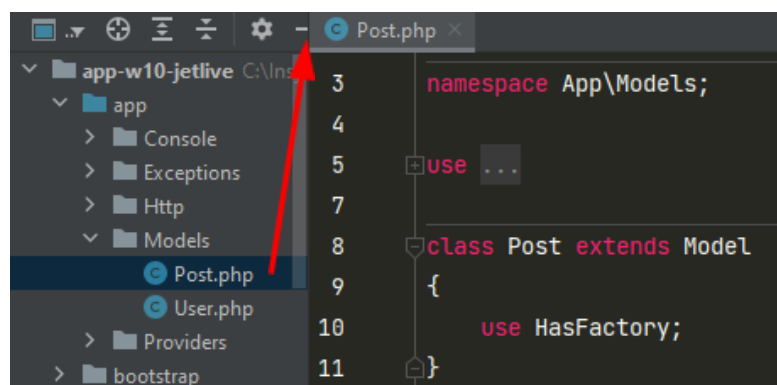
Reading Data

1. Kita mulai dengan membuat Model dengan command php artisan

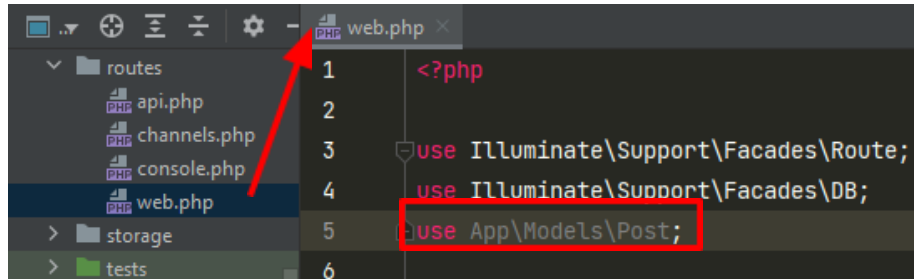
```
rograman-Web\Week-10\app-w10-jetlive> php artisan make:model Post
Model created successfully.
```

Nama model, harus diawali huruf kapital.

2. File model akan otomatis dibuatkan. Secara default, Model Post akan mengakses tabel posts pada database (bentuk jamak dari Post – lowercase) dan menganggap tabel database memiliki primary key dengan nama id.



3. Pada file routes/web.php, tambahkan kode berikut untuk menandakan bahwa kita akan menggunakan model Post.

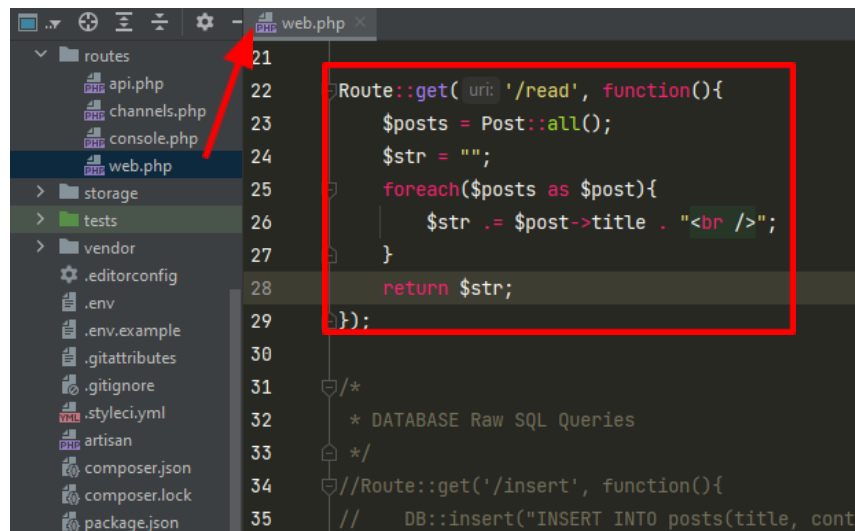


```

1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use Illuminate\Support\Facades\DB;
5 use App\Models\Post;
6

```

4. Tambahkan kode berikut pada routes/web.php (komentar bagian Raw SQL Queries) sehingga Anda dapat membandingkan cara kerja Raw SQL dengan Eloquent.

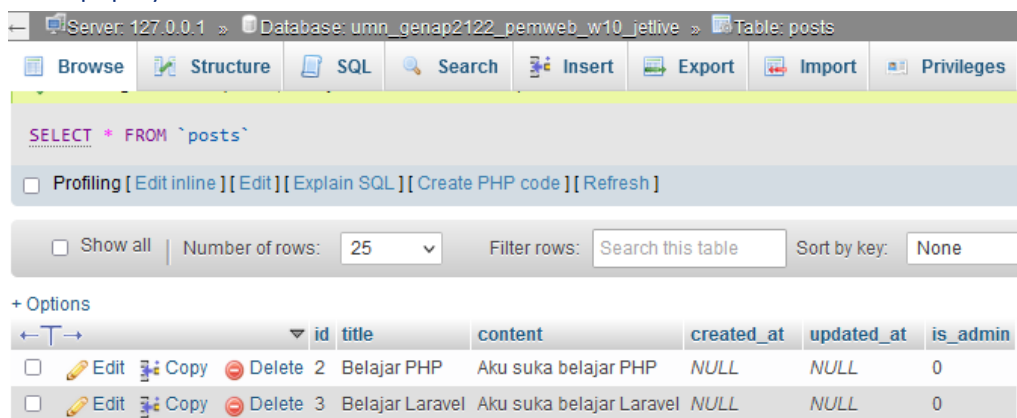


```

21
22 Route::get( uri: '/read', function(){
23     $posts = Post::all();
24     $str = "";
25     foreach($posts as $post){
26         $str .= $post->title . "<br />";
27     }
28     return $str;
29 });
30
31 /*
32  * DATABASE Raw SQL Queries
33 */
34 //Route::get('/insert', function(){
35     // DB::insert("INSERT INTO posts(title, cont

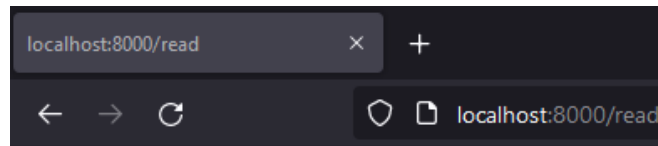
```

5. Pastikan terdapat data pada tabel posts. Jika belum, Anda dapat melakukan insert secara manual menggunakan phpMyAdmin.



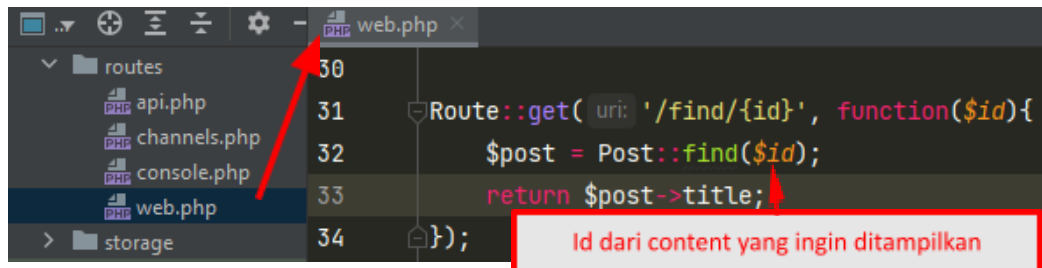
id	title	content	created_at	updated_at	is_admin
2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0

6. Lihat hasilnya di browser

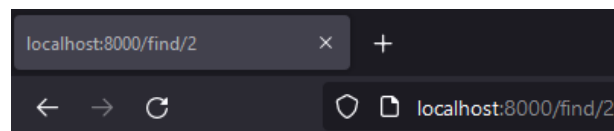


Belajar PHP
Belajar Laravel

7. Tambahkan kode berikut pada routes/web.php. Fungsi find() berguna digunakan ketika kita mau menampilkan hasil pencarian data berdasarkan id.



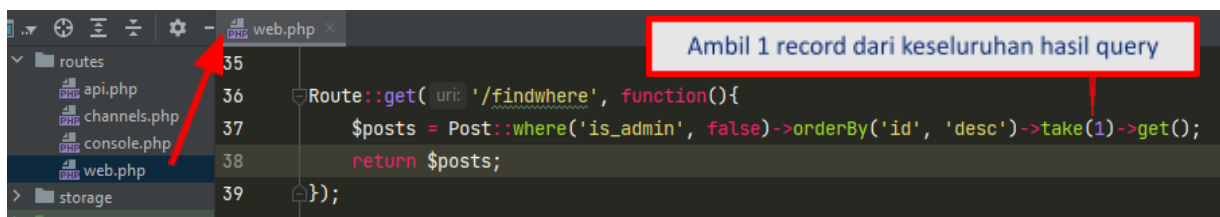
8. Lihat hasilnya di browser



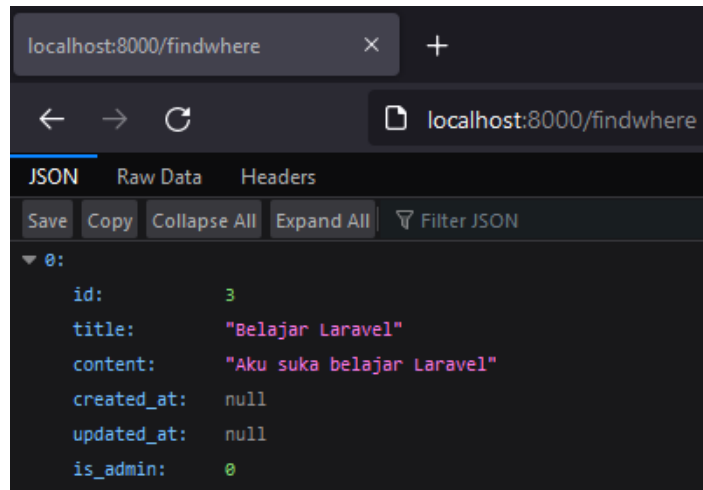
Belajar PHP

Reading/Finding with Constraints

1. Tambahkan kode berikut pada routes/web.php untuk membaca data dari database dengan batasan tertentu (WHERE).

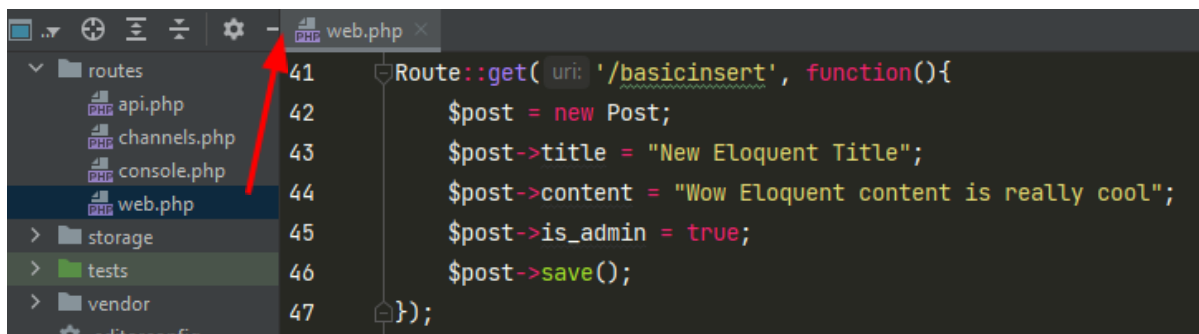


2. Lihat hasilnya di browser

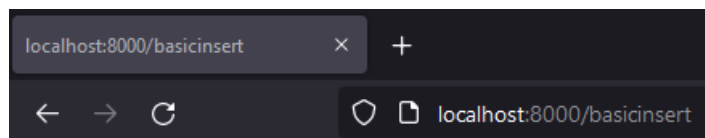


Inserting/Saving Data (INSERT CARA 1)

1. Untuk melakukan insert data, ketikkan kode berikut pada routes/web.php



2. Jalankan di browser



3. Lihat hasilnya melalui phpMyAdmin

Server: 127.0.0.1 Database: umn_genap2122_pemweb_w10_jettive Table: posts

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds.)

SELECT * FROM `posts`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

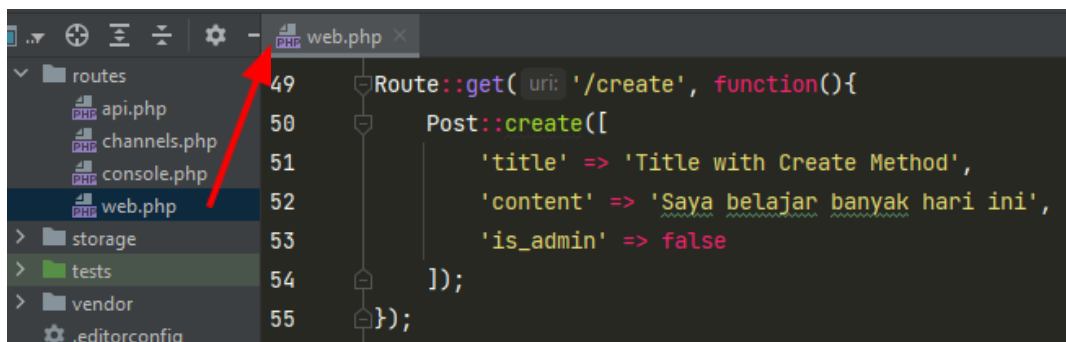
Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

+ Options

	id	title	content	created_at	updated_at	is_admin
☐ Edit ☐ Copy ☐ Delete	2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	4	New Eloquent Title	Wow Eloquent content is really cool	2022-04-08 03:40:37	2022-04-08 03:40:37	1

Creating Data and Configuring Mass Assignment (INSERT DARA 2)

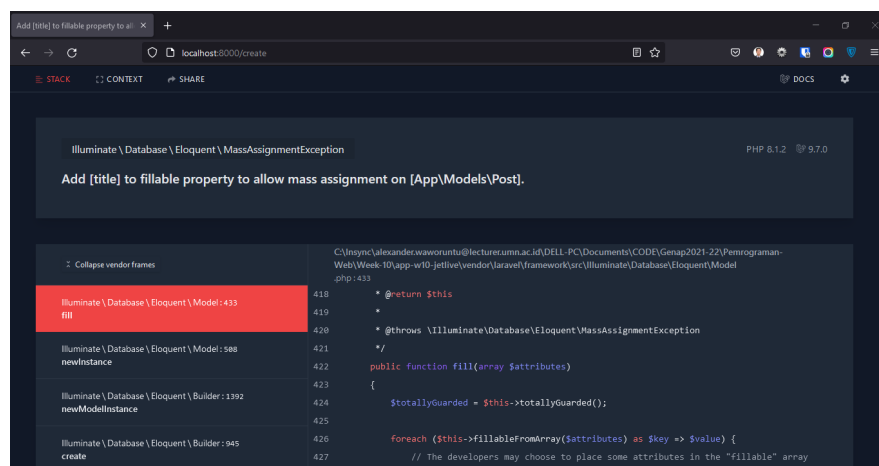
- Selain dengan metode yang sudah kita coba sebelumnya, terdapat fungsi/metode create() yang digunakan untuk mengisi data pada tabel database. Ketikkan kode berikut pada routes/web.php



```

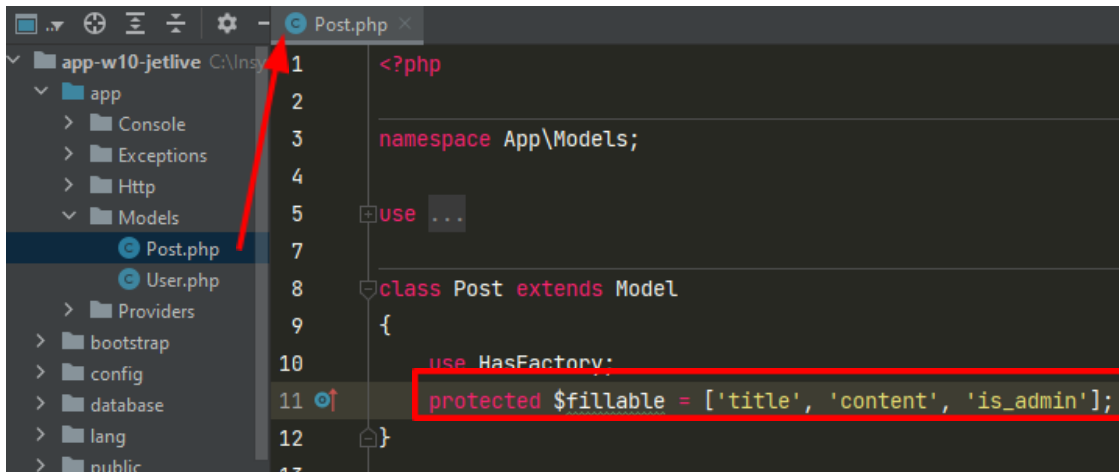
49 Route::get( uri: '/create', function(){
50     Post::create([
51         'title' => 'Title with Create Method',
52         'content' => 'Saya belajar banyak hari ini',
53         'is_admin' => false
54     ]);
55 });
  
```

- Coba jalankan di browser



Muncul pesan kesalahan (MassAssignmentException) karena dengan metode create, Laravel melakukan insert data pada beberapa kolom sekaligus, secara default Laravel tidak mengizinkan hal tersebut, kecuali kita melakukan konfigurasi pada Model.

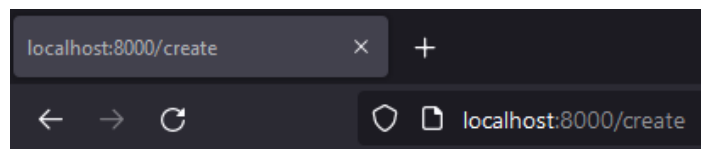
- Buka model Post.php, tuliskan kode berikut sehingga memungkinkan untuk mengisi data pada kolom title, content dan is_admin.



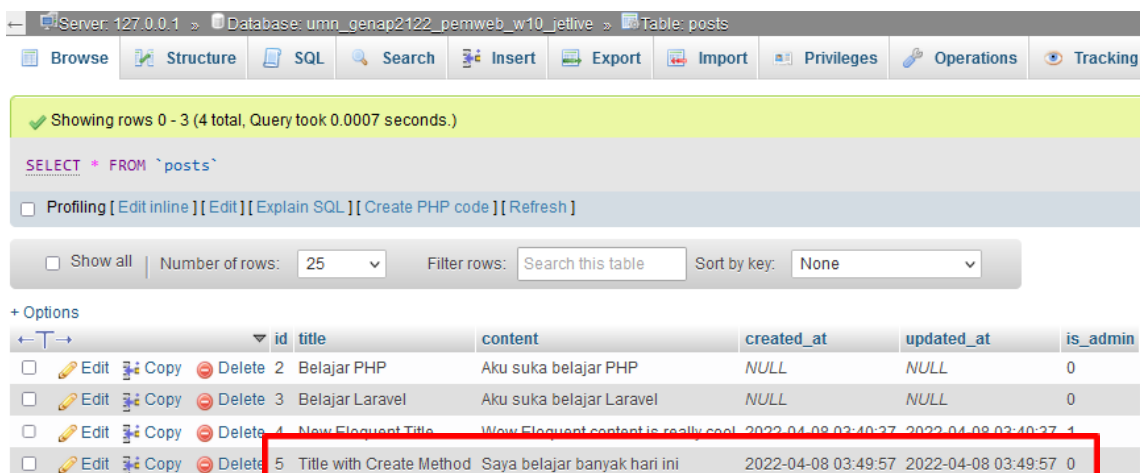
```

1 <?php
2
3 namespace App\Models;
4
5 use ...
6
7
8 class Post extends Model
9 {
10     use HasFactory;
11     protected $fillable = ['title', 'content', 'is_admin'];
12 }
13
  
```

- Jalankan kembali di browser, error yang tadi muncul sudah hilang.



- Lihat hasilnya di phpMyAdmin



id	title	content	created_at	updated_at	is_admin
2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0
4	New Eloquent Title	New Eloquent content is really cool	2022-04-08 03:49:57	2022-04-08 03:49:57	1
5	Title with Create Method	Saya belajar banyak hari ini	2022-04-08 03:49:57	2022-04-08 03:49:57	0

Update Data (UPDATE CARA 1)

- Tuliskan kode berikut pada routes/web.php



```

57 Route::get( uri: '/basicupdate', function(){
58     $post = Post::find(5);
59     $post->title = 'Updated Eloquent Title';
60     $post->content = 'Updated Eloquent Content';
61     $post->save();
62 });
63
  
```

- Sebelum kita jalankan, coba lihat data saat ini melalui phpMyAdmin, data yang ingin di-update adalah yang memiliki id=5

Server: 127.0.0.1 → Database: umn_genap2122_pemweb_w10_jettive → Table: posts

Showing rows 0 - 3 (4 total, Query took 0.0007 seconds.)

SELECT * FROM `posts`

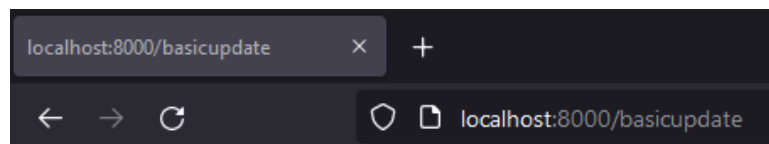
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

+ Options

	id	title	content	created_at	updated_at	is_admin
☐ Edit ☐ Copy ☐ Delete	2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	4	New Eloquent Title	Wow Eloquent content is really cool	2022-04-08 03:40:37	2022-04-08 03:40:37	1
☐ Edit ☐ Copy ☐ Delete	5	Title with Create Method	Saya belajar banyak hari ini	2022-04-08 03:49:57	2022-04-08 03:49:57	0

- Jalankan di browser



- Lihat hasilnya di phpMyAdmin

Server: 127.0.0.1 → Database: umn_genap2122_pemweb_w10_jettive → Table: posts

Showing rows 0 - 3 (4 total, Query took 0.0008 seconds.)

SELECT * FROM `posts`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

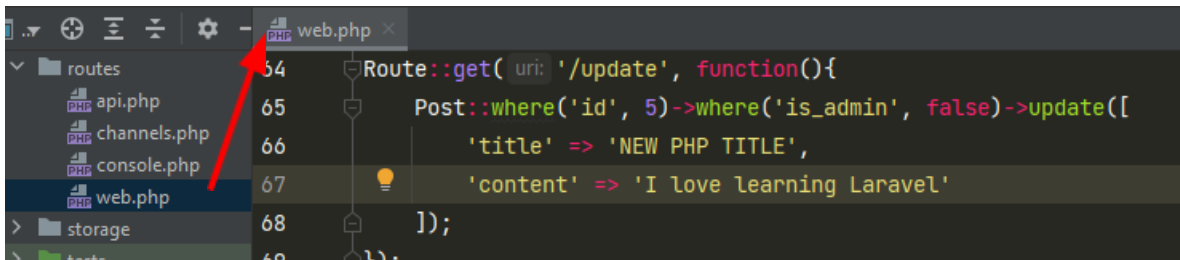
Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

+ Options

	id	title	content	created_at	updated_at	is_admin
☐ Edit ☐ Copy ☐ Delete	2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	4	New Eloquent Title	Wow Eloquent content is really cool	2022-04-08 03:40:37	2022-04-08 03:40:37	1
☐ Edit ☐ Copy ☐ Delete	5	Updated Eloquent Title	Updated Eloquent Content	2022-04-08 03:49:57	2022-04-08 05:04:51	0

Updating with Eloquent (UPDATE DARA 2)

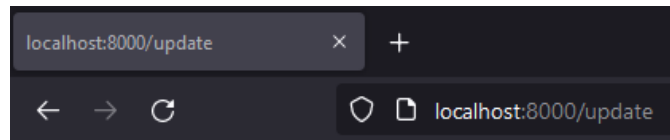
- Ketikkan kode berikut pada routes/web.php



```

64 Route::get( uri: '/update', function(){
65     Post::where('id', 5)->where('is_admin', false)->update([
66         'title' => 'NEW PHP TITLE',
67         'content' => 'I love learning Laravel'
68     ]);
69 }
  
```

2. Jalankan di browser



3. Lihat hasilnya di phpMyAdmin

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetive » Table: posts

Showing rows 0 - 3 (4 total, Query took 0.0011 seconds.)

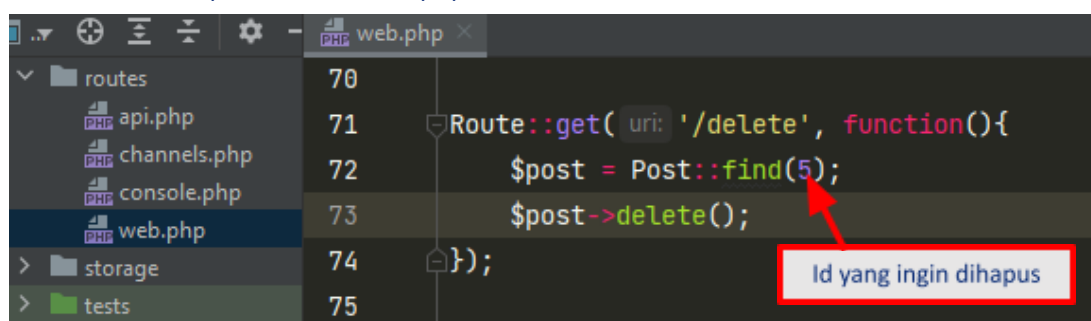
SELECT * FROM `posts`

Options: Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	id	title	content	created_at	updated_at	is_admin
☐ Edit ☐ Copy ☐ Delete	2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0
☐ Edit ☐ Copy ☐ Delete	4	New Eloquent Title	Wow Eloquent content is really cool	2022-04-08 03:40:37	2022-04-08 03:40:37	1
☐ Edit ☐ Copy ☐ Delete	5	NEW PHP TITLE	I love learning Laravel	2022-04-08 03:49:57	2022-04-08 05:10:40	0

Deleting Data (DELETE CARA 1)

1. Ketikkan kode berikut pada routes/web.php

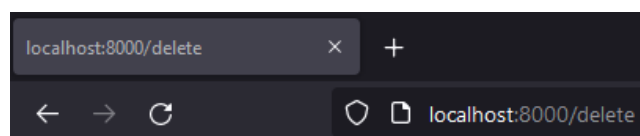


```

70
71 Route::get( uri: '/delete', function(){
72     $post = Post::find(5);
73     $post->delete();
74 });
75
  
```

Id yang ingin dihapus

2. Jalankan di browser



3. Lihat hasilnya di phpMyAdmin.

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetive » Table: posts

Showing rows 0 - 2 (3 total, Query took 0.0006 seconds.)

`SELECT * FROM `posts``

☐ Profiling [\[Edit inline\]](#) [\[Edit\]](#) [\[Explain SQL\]](#) [\[Create PHP code\]](#) [\[Refresh\]](#)

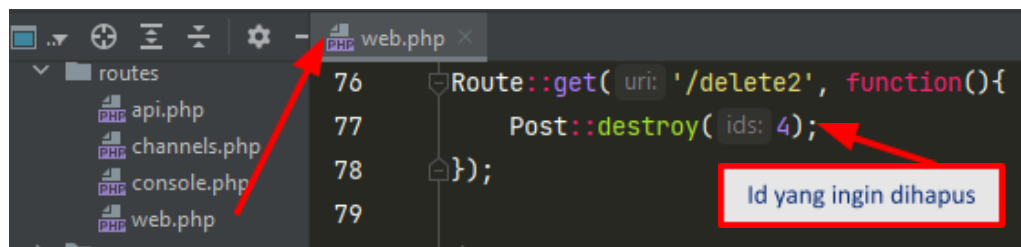
☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

+ Options

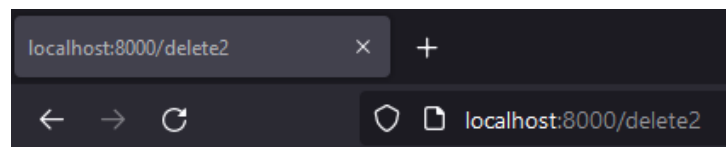
	id	title	content	created_at	updated_at	is_admin
☐ Edit Copy Delete	2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
☐ Edit Copy Delete	3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0
☐ Edit Copy Delete	4	New Eloquent Title	Wow Eloquent content is really cool	2022-04-08 03:40:37	2022-04-08 03:40:37	1

Deleting Data (DELETE CARA 2)

1. Ketikkan kode berikut pada routes/web.php



2. Jalankan di browser



3. Lihat hasilnya di phpMyAdmin

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetive » Table: posts

Showing rows 0 - 1 (2 total, Query took 0.0007 seconds.)

`SELECT * FROM `posts``

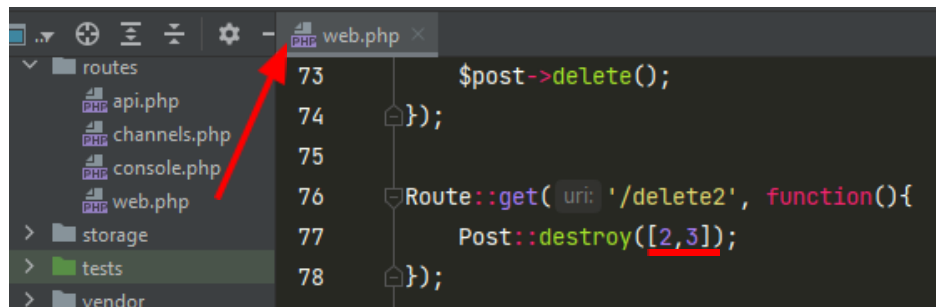
☐ Profiling [\[Edit inline\]](#) [\[Edit\]](#) [\[Explain SQL\]](#) [\[Create PHP code\]](#) [\[Refresh\]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

+ Options

	id	title	content	created_at	updated_at	is_admin
☐ Edit Copy Delete	2	Belajar PHP	Aku suka belajar PHP	NULL	NULL	0
☐ Edit Copy Delete	3	Belajar Laravel	Aku suka belajar Laravel	NULL	NULL	0

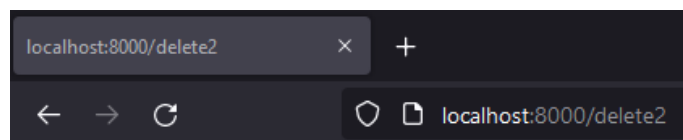
4. Dengan method yang sama, kita juga dapat menghapus beberapa record sekaligus. Ubah kode tadi menjadi seperti berikut.



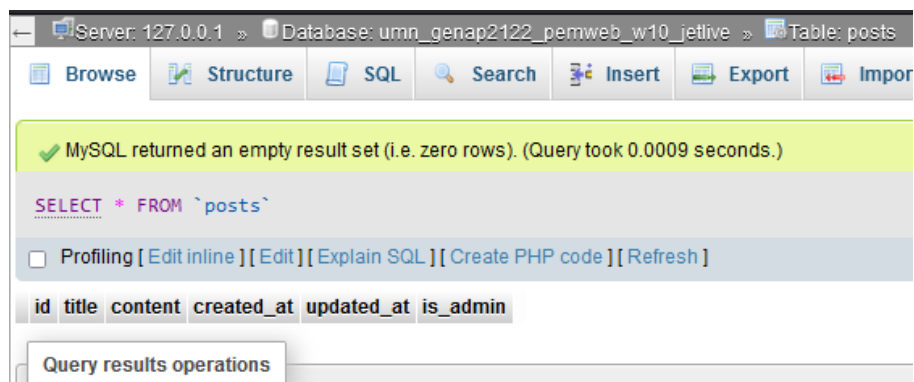
```

73      $post->delete();
74  });
75
76  Route::get( uri: '/delete2', function(){
77      Post::destroy([2,3]);
78  });
  
```

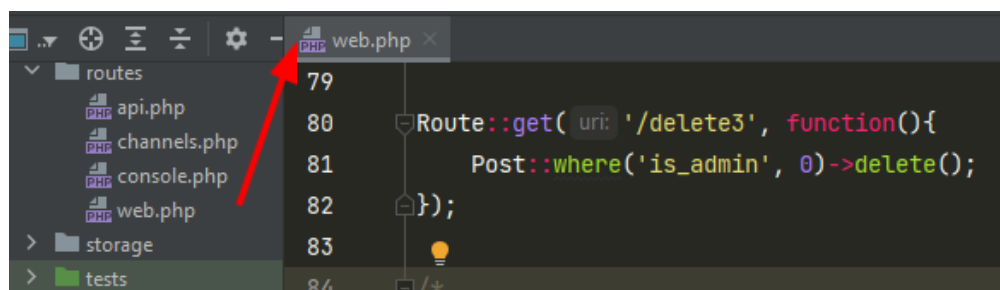
5. Jalankan di browser



6. Lihat hasilnya di phpMyAdmin



7. Kita juga dapat menggunakan query untuk memilih record yang akan dihapus dengan kode berikut:



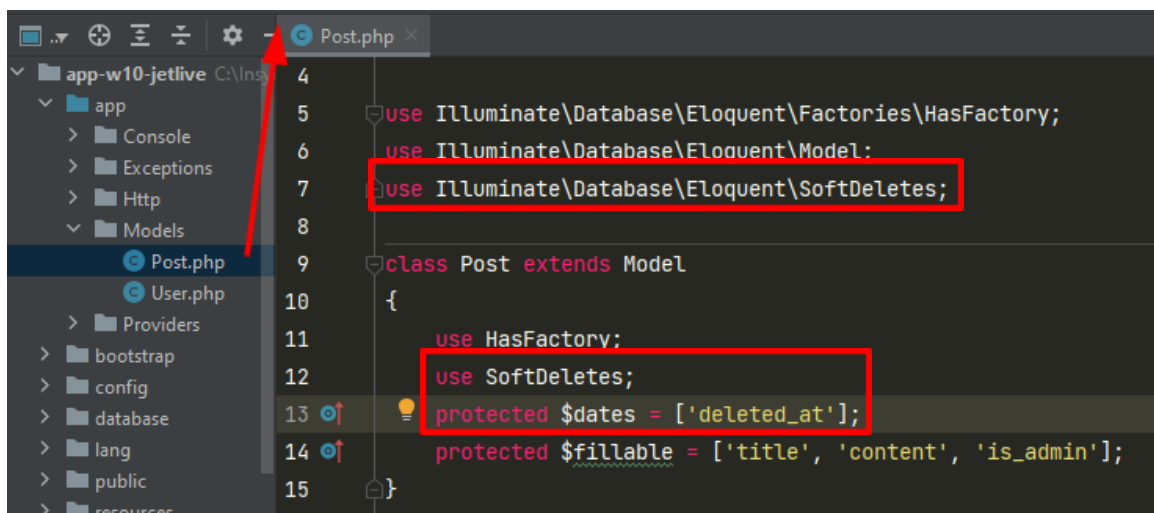
```

79
80  Route::get( uri: '/delete3', function(){
81      Post::where('is_admin', 0)->delete();
82  });
83
84  /*
  
```

8. Silahkan dicoba sendiri.

Soft Deleting / Trashing

1. Ketikkan kode berikut pada Model Post.php



```

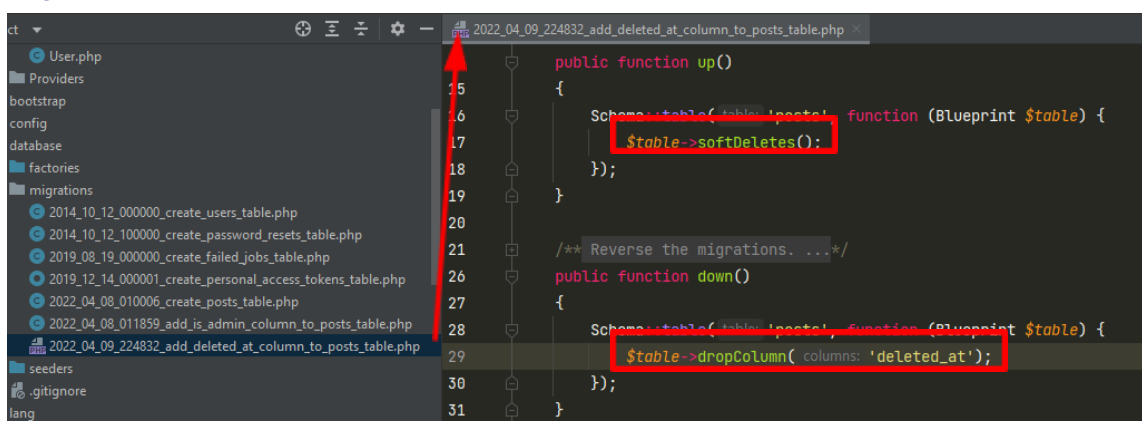
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7 use Illuminate\Database\Eloquent\SoftDeletes;
8
9 class Post extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13     protected $dates = ['deleted_at'];
14     protected $fillable = ['title', 'content', 'is_admin'];
15 }
  
```

2. Buat migration untuk menambahkan kolom "deleted_at" pada tabel posts dengan mengetikkan command berikut pada terminal.

```

0\app-w10-jetlive> php artisan make:migration add_deleted_at_column_to_posts_table --table=posts
Created Migration: 2022_04_09_224832_add_deleted_at_column_to_posts_table
  
```

3. File migration akan otomatis dibuatkan, tambahkan kode berikut pada file tersebut.



```

15 public function up()
16 {
17     Schema::table('posts', function (Blueprint $table) {
18         $table->softDeletes();
19     });
20 }
21
22 /** Reverse the migrations. ... */
23 public function down()
24 {
25     Schema::table('posts', function (Blueprint $table) {
26         $table->dropColumn('deleted_at');
27     });
28 }
  
```

4. Lakukan migrate melalui terminal dengan command berikut

```

0\app-w10-jetlive> php artisan migrate
Migrating: 2022_04_09_224832_add_deleted_at_column_to_posts_table
Migrated: 2022_04_09_224832_add_deleted_at_column_to_posts_table (17.91ms)
  
```

5. Lihat di phpMyAdmin, kolom "deleted_at" sudah ada.

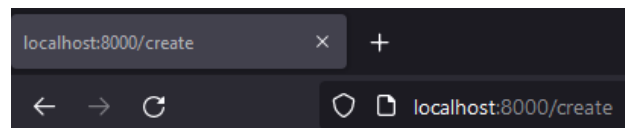
Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetive » Table: posts

Browse Structure SQL Search Insert Export Import

Table structure Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO
☐ 2	title	varchar(255)	utf8mb4_unicode_ci		No	None		
☐ 3	content	text	utf8mb4_unicode_ci		No	None		
☐ 4	created_at	timestamp			Yes	NULL		
☐ 5	updated_at	timestamp			Yes	NULL		
☐ 6	is_admin	tinyint(1)			No	0		
☐ 7	deleted_at	timestamp			Yes	NULL		

6. Jika sebelumnya tabel posts tidak memiliki data (karena sudah dihapus semua pada sesi sebelumnya), tambahkan dengan memanggil route "/create".



7. Lihat di phpMyAdmin hasilnya, perhatikan bahwa kolom deleted_at memiliki nilai NULL.

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetive » Table: posts

Browse Structure SQL Search Insert Export Import Privileges Operations Tracking %

Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)

SELECT * FROM `posts`

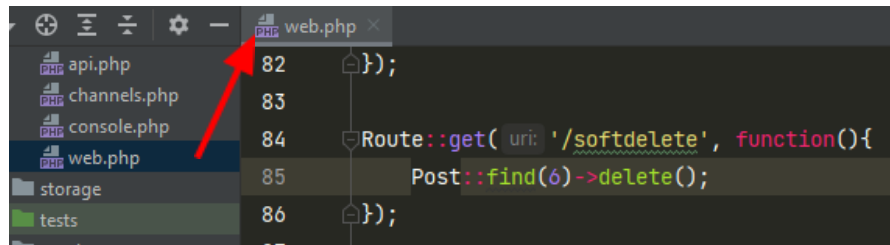
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

+ Options

	id	title	content	created_at	updated_at	is_admin	deleted_at
☐ Edit ☐ Copy ☐ Delete	6	Title with Create Method	Saya belajar banyak hari ini	2022-04-09 22:54:21	2022-04-09 22:54:21	0	NULL

8. Tambahkan kode berikut pada routes/web.php

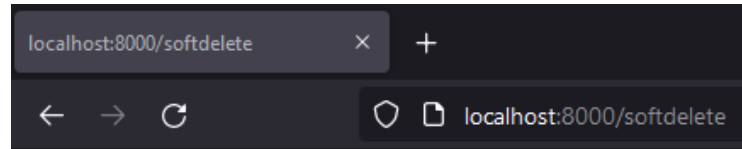


```

82  });
83
84  Route::get( uri: '/softdelete', function(){
85      Post::find(6)->delete();
86  });

```

9. Jalankan belalui browser



10. Lihat hasilnya di phpMyAdmin

Server: 127.0.0.1 > Database: umn_genap2122_pemweb_w10_jelive > Table: posts

Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)

SELECT * FROM `posts`

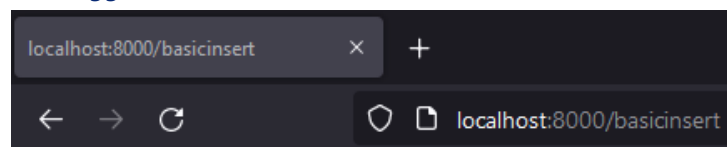
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

	id	title	content	created_at	updated_at	is_admin	deleted_at
☐ Edit ☐ Copy ☐ Delete	6	Title with Create Method	Saya belajar banyak hari ini	2022-04-09 22:54:21	2022-04-09 22:57:28	0	2022-04-09 22:57:28

Retrieving Deleted / Trashed Record

- Kita akan mencoba menampilkan data yang sudah kita softdelete. Namun sebelumnya, coba tambahkan dulu data baru sehingga kita memiliki 2 data, yaitu data yang sudah di-softdelete dan yang tidak, dengan memanggil route /basicinsert.



2. Lihat hasilnya di phpMyAdmin

Server: 127.0.0.1 > Database: umn_genap2122_pemweb_w10_jelive > Table: posts

Showing rows 0 - 1 (2 total, Query took 0.0012 seconds.)

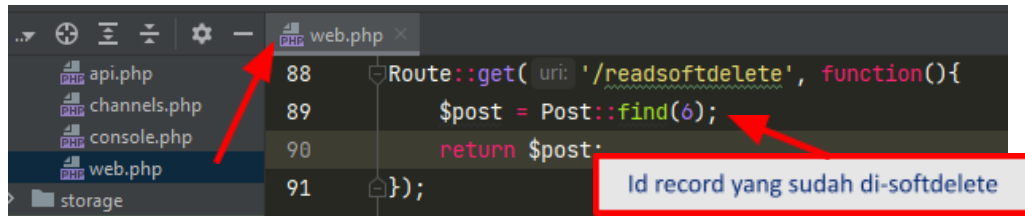
SELECT * FROM `posts`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	id	title	content	created_at	updated_at	is_admin	deleted_at
☐ Edit ☐ Copy ☐ Delete	6	Title with Create Method	Saya belajar banyak hari ini	2022-04-09 22:54:21	2022-04-09 22:57:28	0	2022-04-09 22:57:28
☐ Edit ☐ Copy ☐ Delete	7	New Eloquent Title	Wow Eloquent content is really cool	2022-04-09 23:00:20	2022-04-09 23:00:20	1	NULL

3. Ketikkan kode berikut pada routes/web.php

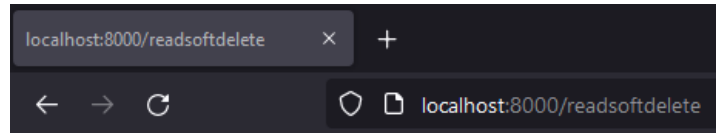


```

88 Route::get( uri: '/readsoftdelete', function(){
89     $post = Post::find(6);
90     return $post;
91 });
  
```

Id record yang sudah di-softdelete

4. Lihat hasilnya di browser



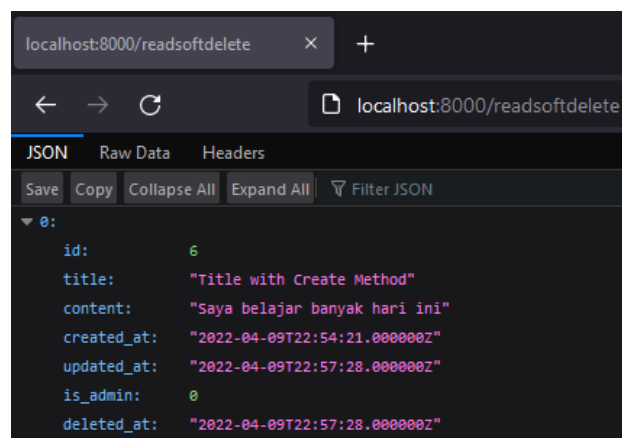
5. Ubah route menjadi seperti berikut



```

88 Route::get( uri: '/readsoftdelete', function(){
89     // $post = Post::find(6);
90     // return $post;
91     $post = Post::withTrashed()->where('id', 6)->get();
92     return $post;
93 });
  
```

6. Lihat hasilnya di browser

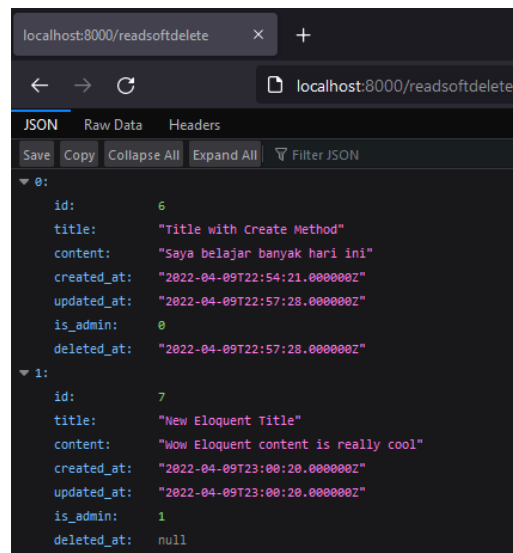


7. Ubah kode routes menjadi seperti berikut



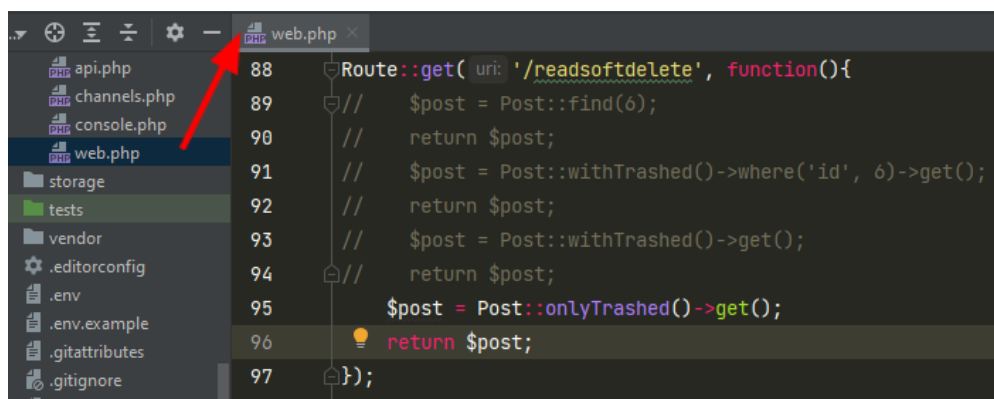
```
88 Route::get( uri: '/readsoftdelete', function(){
89     // $post = Post::find(6);
90     // return $post;
91     // $post = Post::withTrashed()->where('id', 6)->get();
92     // return $post;
93     $post = Post::withTrashed()->get();
94     return $post;
95 });
```

8. Lihat hasilnya di browser (menampilkan semua konten, baik yang sudah dihapus maupun belum)



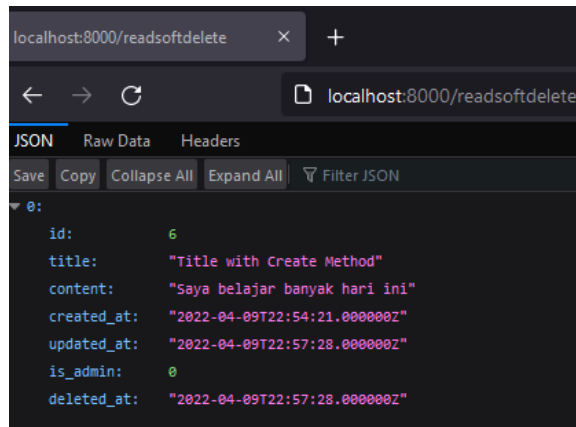
```
localhost:8000/readsoftdelete
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
0:
  id: 6
  title: "Title with Create Method"
  content: "Saya belajar banyak hari ini"
  created_at: "2022-04-09T22:54:21.000000Z"
  updated_at: "2022-04-09T22:57:28.000000Z"
  is_admin: 0
  deleted_at: "2022-04-09T22:57:28.000000Z"
1:
  id: 7
  title: "New Eloquent Title"
  content: "Wow Eloquent content is really cool"
  created_at: "2022-04-09T23:00:20.000000Z"
  updated_at: "2022-04-09T23:00:20.000000Z"
  is_admin: 1
  deleted_at: null
```

9. Ubah kode routes menjadi seperti berikut



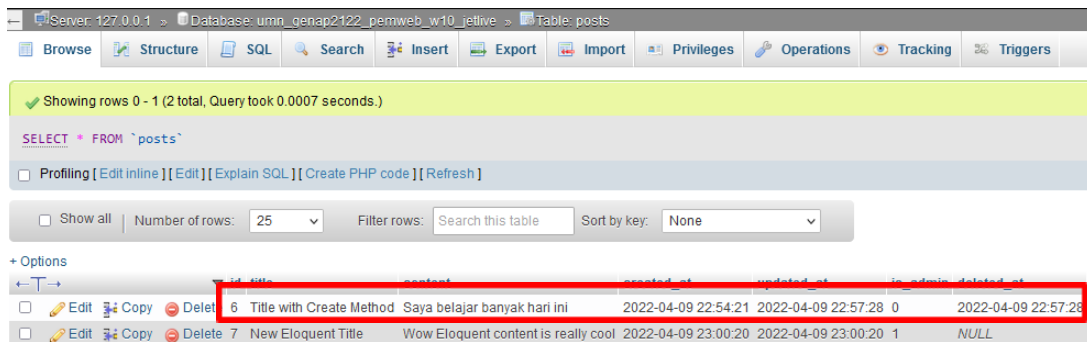
```
88 Route::get( uri: '/readsoftdelete', function(){
89     // $post = Post::find(6);
90     // return $post;
91     // $post = Post::withTrashed()->where('id', 6)->get();
92     // return $post;
93     // $post = Post::withTrashed()->get();
94     // return $post;
95     $post = Post::onlyTrashed()->get();
96     return $post;
97 });
```

10. Lihat hasilnya di browser (hanya menampilkan record yang sudah dihapus)

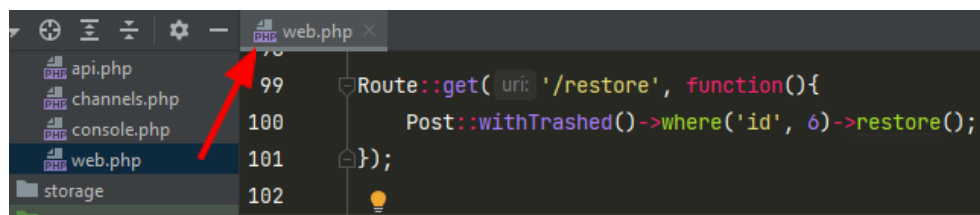


Restoring deleted / trashed records

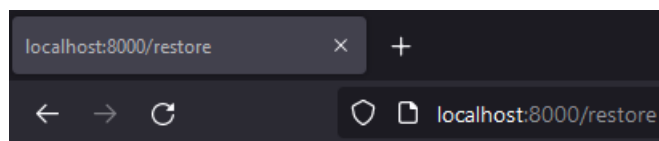
1. Perhatikan isi tabel posts melalui phpMyAdmin, kita akan me-restore record yang sudah dihapus



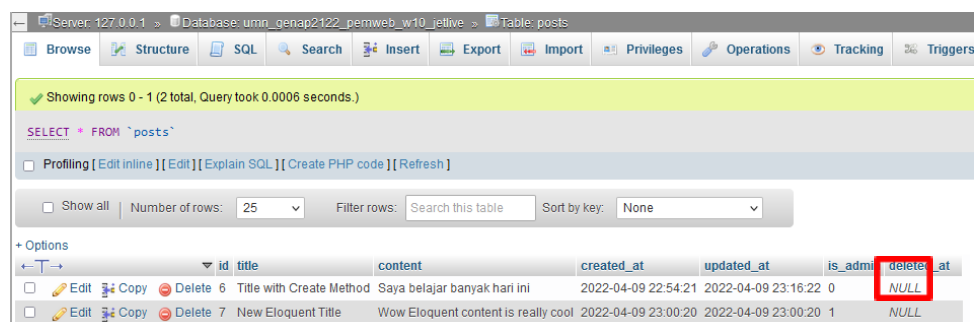
2. Ketikkan kode berikut pada routes/web.php



3. Jalankan route tersebut pada browser

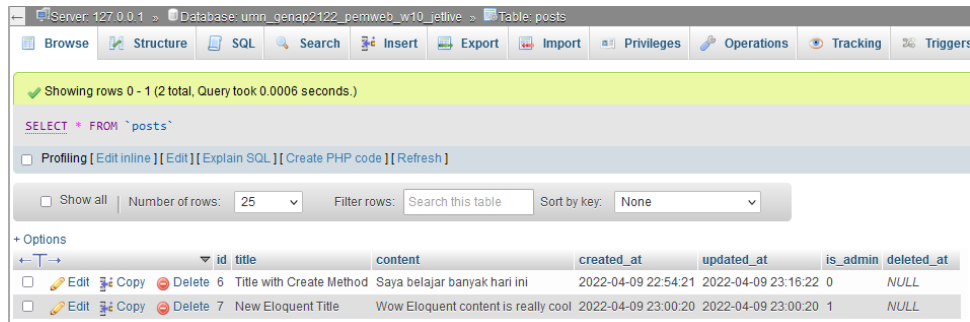


4. Lihat hasilnya pada phpMyAdmin



Deleting Record Permanently

1. Pada sesi sebelumnya kita sudah melakukan restore record yang sudah di-hapus. Pada sesi ini kita akan menghapus secara permanen. Laravel memiliki method `forceDelete()` untuk menghapus record secara permanen. Untuk itu kita review dulu kondisi database kita saat ini melalui phpMyAdmin



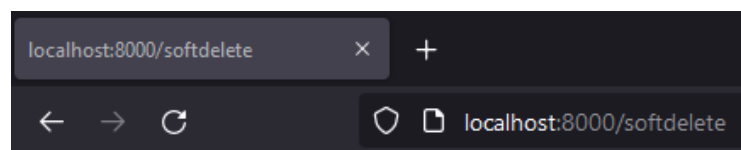
Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

`SELECT * FROM `posts``

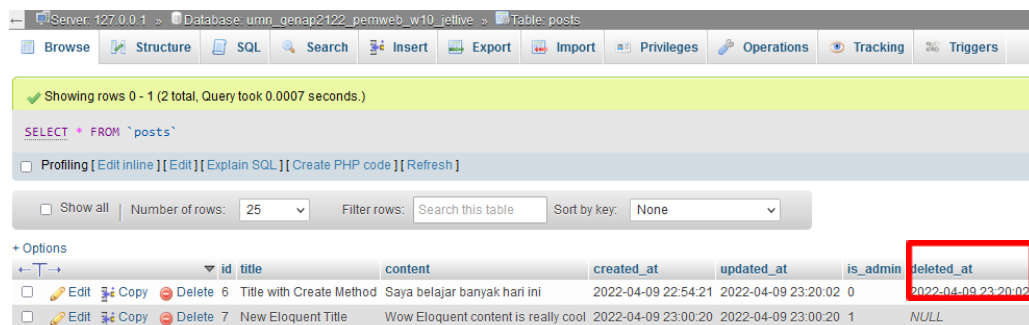
Number of rows: 25 Filter rows: Search this table Sort by key: None

	id	title	content	created_at	updated_at	is_admin	deleted_at
☐ Edit ☐ Copy ☐ Delete	6	Title with Create Method	Saya belajar banyak hari ini	2022-04-09 22:54:21	2022-04-09 23:16:22	0	NULL
☐ Edit ☐ Copy ☐ Delete	7	New Eloquent Title	Wow Eloquent content is really cool	2022-04-09 23:00:20	2022-04-09 23:00:20	1	NULL

2. Kita hapus (softdelete) salah satu record. Panggil route `/softdelete`



3. Lihat hasilnya melalui phpMyAdmin



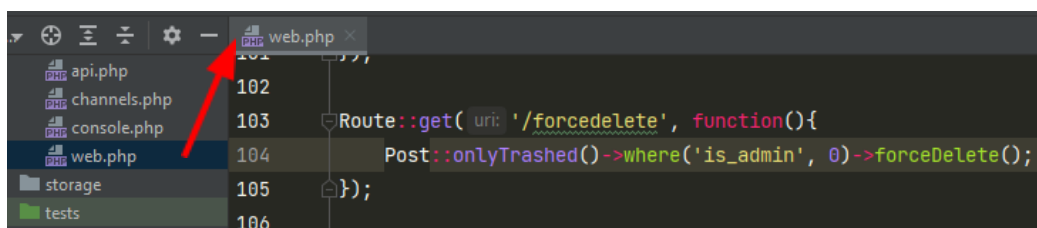
Showing rows 0 - 1 (2 total, Query took 0.0007 seconds.)

`SELECT * FROM `posts``

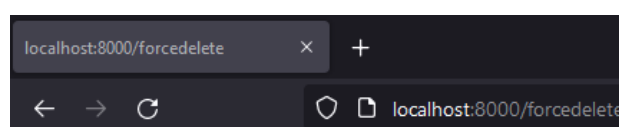
Number of rows: 25 Filter rows: Search this table Sort by key: None

	id	title	content	created_at	updated_at	is_admin	deleted_at
☐ Edit ☐ Copy ☐ Delete	6	Title with Create Method	Saya belajar banyak hari ini	2022-04-09 22:54:21	2022-04-09 23:20:02	0	2022-04-09 23:20:02
☐ Edit ☐ Copy ☐ Delete	7	New Eloquent Title	Wow Eloquent content is really cool	2022-04-09 23:00:20	2022-04-09 23:00:20	1	NULL

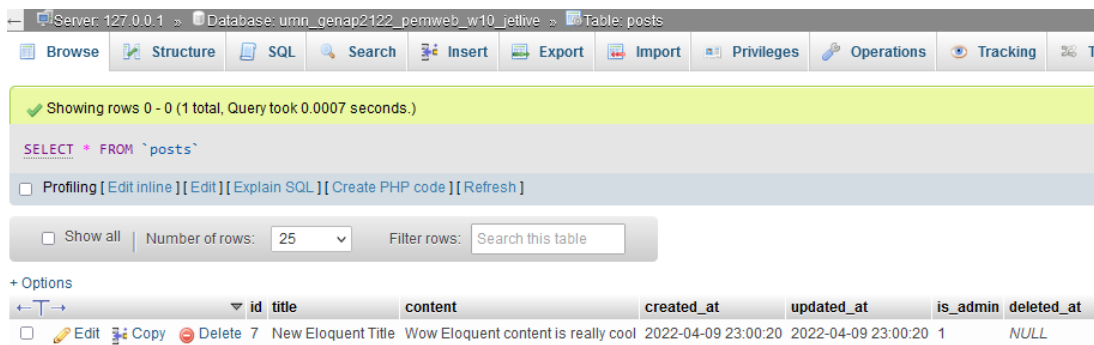
4. Ketikkan kode berikut pada routes/web.php



5. Jalankan di browser



6. Lihat hasilnya di phpMyAdmin



The screenshot shows the phpMyAdmin interface for a database named 'umn_genap2122_pemweb_w10_jetive'. The 'posts' table is selected. The SQL query bar shows 'SELECT * FROM `posts`'. Below the query bar, there are options for 'Show all', 'Number of rows: 25', and 'Filter rows: Search this table'. The table structure is displayed with columns: id, title, content, created_at, updated_at, is_admin, and deleted_at. The table contains one row of data.

id	title	content	created_at	updated_at	is_admin	deleted_at
7	New Eloquent Title	Wow Eloquent content is really cool	2022-04-09 23:00:20	2022-04-09 23:00:20	1	NULL

TUGAS

1. Buat file Model "Student.php" dan file migration untuk membuat table "students" dengan command "php artisan make:model Student -m".
2. Struktur tabel students:

id	INT	PRIMARY	AUTO_INCREMENT
nim	VARCHAR(10)	UNIQUE	
nama	VARCHAR(50)		
prodi	VARCHAR(20)		
3. Ulangi semua tahap yang sebelumnya sudah dipelajari, untuk melakukan proses CRUD pada tabel posts dengan routes "/students/THEPROCESS".

MINGGU 11

ELOQUENT RELATIONSHIP

DESKRIPSI TEMA

Mahasiswa mempraktikkan Eloquent Relationship menggunakan Framework Laravel

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menerapkan Eloquent Relationship menggunakan Framework Laravel

PERALATAN YANG DIGUNAKAN

1. Code Editor
2. XAMPP
3. Composer

LANGKAH-LANGKAH PRAKTIKUM

Praktikum minggu ini menggunakan code-base yang sama/melanjutkan dari minggu sebelumnya.

One-to-One Relationship

(Target: Menampilkan post berdasarkan user_id)

1. Kita akan menambahkan kolom user_id pada tabel posts untuk menghubungkan posts dengan users. Edit file migration yang digunakan untuk membuat tabel posts (kita akan melakukan refresh – drop all migration and migrate all, alternative lain bisa membuat file migration baru dan melakukan migrate)



```
12 2022_04_08_010006_create_posts_table.php
13
14 12
15 13
16 14
17 15
18 16
19 17
20 18
21 19
22 20
23 21
24 22
25 23
26 24
27 25
28 26
29 27
30 28
31 29
32 30
33 31
34 32
35 33
36 34
37 35
38 36
39 37
40 38
41 39
42 40
43 41
44 42
45 43
46 44
47 45
48 46
49 47
50 48
51 49
52 50
53 51
54 52
55 53
56 54
57 55
58 56
59 57
60 58
61 59
62 60
63 61
64 62
65 63
66 64
67 65
68 66
69 67
70 68
71 69
72 70
73 71
74 72
75 73
76 74
77 75
78 76
79 77
80 78
81 79
82 80
83 81
84 82
85 83
86 84
87 85
88 86
89 87
90 88
91 89
92 90
93 91
94 92
95 93
96 94
97 95
98 96
99 97
100 98
101 99
102 100
103 101
104 102
105 103
106 104
107 105
108 106
109 107
110 108
111 109
112 110
113 111
114 112
115 113
116 114
117 115
118 116
119 117
120 118
121 119
122 120
123 121
124 122
125 123
126 124
127 125
128 126
129 127
130 128
131 129
132 130
133 131
134 132
135 133
136 134
137 135
138 136
139 137
140 138
141 139
142 140
143 141
144 142
145 143
146 144
147 145
148 146
149 147
150 148
151 149
152 150
153 151
154 152
155 153
156 154
157 155
158 156
159 157
160 158
161 159
162 160
163 161
164 162
165 163
166 164
167 165
168 166
169 167
170 168
171 169
172 170
173 171
174 172
175 173
176 174
177 175
178 176
179 177
180 178
181 179
182 180
183 181
184 182
185 183
186 184
187 185
188 186
189 187
190 188
191 189
192 190
193 191
194 192
195 193
196 194
197 195
198 196
199 197
200 198
201 199
202 200
203 201
204 202
205 203
206 204
207 205
208 206
209 207
210 208
211 209
212 210
213 211
214 212
215 213
216 214
217 215
218 216
219 217
220 218
221 219
222 220
223 221
224 222
225 223
226 224
227 225
228 226
229 227
230 228
231 229
232 230
233 231
234 232
235 233
236 234
237 235
238 236
239 237
240 238
241 239
242 240
243 241
244 242
245 243
246 244
247 245
248 246
249 247
250 248
251 249
252 250
253 251
254 252
255 253
256 254
257 255
258 256
259 257
260 258
261 259
262 260
263 261
264 262
265 263
266 264
267 265
268 266
269 267
270 268
271 269
272 270
273 271
274 272
275 273
276 274
277 275
278 276
279 277
280 278
281 279
282 280
283 281
284 282
285 283
286 284
287 285
288 286
289 287
290 288
291 289
292 290
293 291
294 292
295 293
296 294
297 295
298 296
299 297
300 298
301 299
302 300
303 301
304 302
305 303
306 304
307 305
308 306
309 307
310 308
311 309
312 310
313 311
314 312
315 313
316 314
317 315
318 316
319 317
320 318
321 319
322 320
323 321
324 322
325 323
326 324
327 325
328 326
329 327
330 328
331 329
332 330
333 331
334 332
335 333
336 334
337 335
338 336
339 337
340 338
341 339
342 340
343 341
344 342
345 343
346 344
347 345
348 346
349 347
350 348
351 349
352 350
353 351
354 352
355 353
356 354
357 355
358 356
359 357
360 358
361 359
362 360
363 361
364 362
365 363
366 364
367 365
368 366
369 367
370 368
371 369
372 370
373 371
374 372
375 373
376 374
377 375
378 376
379 377
380 378
381 379
382 380
383 381
384 382
385 383
386 384
387 385
388 386
389 387
390 388
391 389
392 390
393 391
394 392
395 393
396 394
397 395
398 396
399 397
400 398
401 399
402 400
403 401
404 402
405 403
406 404
407 405
408 406
409 407
410 408
411 409
412 410
413 411
414 412
415 413
416 414
417 415
418 416
419 417
420 418
421 419
422 420
423 421
424 422
425 423
426 424
427 425
428 426
429 427
430 428
431 429
432 430
433 431
434 432
435 433
436 434
437 435
438 436
439 437
440 438
441 439
442 440
443 441
444 442
445 443
446 444
447 445
448 446
449 447
450 448
451 449
452 450
453 451
454 452
455 453
456 454
457 455
458 456
459 457
460 458
461 459
462 460
463 461
464 462
465 463
466 464
467 465
468 466
469 467
470 468
471 469
472 470
473 471
474 472
475 473
476 474
477 475
478 476
479 477
480 478
481 479
482 480
483 481
484 482
485 483
486 484
487 485
488 486
489 487
490 488
491 489
492 490
493 491
494 492
495 493
496 494
497 495
498 496
499 497
500 498
501 499
502 500
503 501
504 502
505 503
506 504
507 505
508 506
509 507
510 508
511 509
512 510
513 511
514 512
515 513
516 514
517 515
518 516
519 517
520 518
521 519
522 520
523 521
524 522
525 523
526 524
527 525
528 526
529 527
530 528
531 529
532 530
533 531
534 532
535 533
536 534
537 535
538 536
539 537
540 538
541 539
542 540
543 541
544 542
545 543
546 544
547 545
548 546
549 547
550 548
551 549
552 550
553 551
554 552
555 553
556 554
557 555
558 556
559 557
560 558
561 559
562 560
563 561
564 562
565 563
566 564
567 565
568 566
569 567
570 568
571 569
572 570
573 571
574 572
575 573
576 574
577 575
578 576
579 577
580 578
581 579
582 580
583 581
584 582
585 583
586 584
587 585
588 586
589 587
590 588
591 589
592 590
593 591
594 592
595 593
596 594
597 595
598 596
599 597
600 598
601 599
602 600
603 601
604 602
605 603
606 604
607 605
608 606
609 607
610 608
611 609
612 610
613 611
614 612
615 613
616 614
617 615
618 616
619 617
620 618
621 619
622 620
623 621
624 622
625 623
626 624
627 625
628 626
629 627
630 628
631 629
632 630
633 631
634 632
635 633
636 634
637 635
638 636
639 637
640 638
641 639
642 640
643 641
644 642
645 643
646 644
647 645
648 646
649 647
650 648
651 649
652 650
653 651
654 652
655 653
656 654
657 655
658 656
659 657
660 658
661 659
662 660
663 661
664 662
665 663
666 664
667 665
668 666
669 667
670 668
671 669
672 670
673 671
674 672
675 673
676 674
677 675
678 676
679 677
680 678
681 679
682 680
683 681
684 682
685 683
686 684
687 685
688 686
689 687
690 688
691 689
692 690
693 691
694 692
695 693
696 694
697 695
698 696
699 697
700 698
701 699
702 700
703 701
704 702
705 703
706 704
707 705
708 706
709 707
710 708
711 709
712 710
713 711
714 712
715 713
716 714
717 715
718 716
719 717
720 718
721 719
722 720
723 721
724 722
725 723
726 724
727 725
728 726
729 727
730 728
731 729
732 730
733 731
734 732
735 733
736 734
737 735
738 736
739 737
740 738
741 739
742 740
743 741
744 742
745 743
746 744
747 745
748 746
749 747
750 748
751 749
752 750
753 751
754 752
755 753
756 754
757 755
758 756
759 757
760 758
761 759
762 760
763 761
764 762
765 763
766 764
767 765
768 766
769 767
770 768
771 769
772 770
773 771
774 772
775 773
776 774
777 775
778 776
779 777
780 778
781 779
782 780
783 781
784 782
785 783
786 784
787 785
788 786
789 787
790 788
791 789
792 790
793 791
794 792
795 793
796 794
797 795
798 796
799 797
800 798
801 799
802 800
803 801
804 802
805 803
806 804
807 805
808 806
809 807
810 808
811 809
812 810
813 811
814 812
815 813
816 814
817 815
818 816
819 817
820 818
821 819
822 820
823 821
824 822
825 823
826 824
827 825
828 826
829 827
830 828
831 829
832 830
833 831
834 832
835 833
836 834
837 835
838 836
839 837
840 838
841 839
842 840
843 841
844 842
845 843
846 844
847 845
848 846
849 847
850 848
851 849
852 850
853 851
854 852
855 853
856 854
857 855
858 856
859 857
860 858
861 859
862 860
863 861
864 862
865 863
866 864
867 865
868 866
869 867
870 868
871 869
872 870
873 871
874 872
875 873
876 874
877 875
878 876
879 877
880 878
881 879
882 880
883 881
884 882
885 883
886 884
887 885
888 886
889 887
890 888
891 889
892 890
893 891
894 892
895 893
896 894
897 895
898 896
899 897
900 898
901 899
902 900
903 901
904 902
905 903
906 904
907 905
908 906
909 907
910 908
911 909
912 910
913 911
914 912
915 913
916 914
917 915
918 916
919 917
920 918
921 919
922 920
923 921
924 922
925 923
926 924
927 925
928 926
929 927
930 928
931 929
932 930
933 931
934 932
935 933
936 934
937 935
938 936
939 937
940 938
941 939
942 940
943 941
944 942
945 943
946 944
947 945
948 946
949 947
950 948
951 949
952 950
953 951
954 952
955 953
956 954
957 955
958 956
959 957
960 958
961 959
962 960
963 961
964 962
965 963
966 964
967 965
968 966
969 967
970 968
971 969
972 970
973 971
974 972
975 973
976 974
977 975
978 976
979 977
980 978
981 979
982 980
983 981
984 982
985 983
986 984
987 985
988 986
989 987
990 988
991 989
992 990
993 991
994 992
995 993
996 994
997 995
998 996
999 997
1000 998
1001 999
1002 1000
1003 1001
1004 1002
1005 1003
1006 1004
1007 1005
1008 1006
1009 1007
1010 1008
1011 1009
1012 1010
1013 1011
1014 1012
1015 1013
1016 1014
1017 1015
1018 1016
1019 1017
1020 1018
1021 1019
1022 1020
1023 1021
1024 1022
1025 1023
1026 1024
1027 1025
1028 1026
1029 1027
1030 1028
1031 1029
1032 1030
1033 1031
1034 1032
1035 1033
1036 1034
1037 1035
1038 1036
1039 1037
1040 1038
1041 1039
1042 1040
1043 1041
1044 1042
1045 1043
1046 1044
1047 1045
1048 1046
1049 1047
1050 1048
1051 1049
1052 1050
1053 1051
1054 1052
1055 1053
1056 1054
1057 1055
1058 1056
1059 1057
1060 1058
1061 1059
1062 1060
1063 1061
1064 1062
1065 1063
1066 1064
1067 1065
1068 1066
1069 1067
1070 1068
1071 1069
1072 1070
1073 1071
1074 1072
1075 1073
1076 1074
1077 1075
1078 1076
1079 1077
1080 1078
1081 1079
1082 1080
1083 1081
1084 1082
1085 1083
1086 1084
1087 1085
1088 1086
1089 1087
1090 1088
1091 1089
1092 1090
1093 1091
1094 1092
1095 1093
1096 1094
1097 1095
1098 1096
1099 1097
1100 1098
1101 1099
1102 1100
1103 1101
1104 1102
1105 1103
1106 1104
1107 1105
1108 1106
1109 1107
1110 1108
1111 1109
1112 1110
1113 1111
1114 1112
1115 1113
1116 1114
1117 1115
1118 1116
1119 1117
1120 1118
1121 1119
1122 1120
1123 1121
1124 1122
1125 1123
1126 1124
1127 1125
1128 1126
1129 1127
1130 1128
1131 1129
1132 1130
1133 1131
1134 1132
1135 1133
1136 1134
1137 1135
1138 1136
1139 1137
1140 1138
1141 1139
1142 1140
1143 1141
1144 1142
1145 1143
1146 1144
1147 1145
1148 1146
1149 1147
1150 1148
1151 1149
1152 1150
1153 1151
1154 1152
1155 1153
1156 1154
1157 1155
1158 1156
1159 1157
1160 1158
1161 1159
1162 1160
1163 1161
1164 1162
1165 1163
1166 1164
1167 1165
1168 1166
1169 1167
1170 1168
1171 1169
1172 1170
1173 1171
1174 1172
1175 1173
1176 1174
1177 1175
1178 1176
1179 1177
1180 1178
1181 1179
1182 1180
1183 1181
1184 1182
1185 1183
1186 1184
1187 1185
1188 1186
1189 1187
1190 1188
1191 1189
1192 1190
1193 1191
1194 1192
1195 1193
1196 1194
1197 1195
1198 1196
1199 1197
1200 1198
1201 1199
1202 1200
1203 1201
1204 1202
1205 1203
1206 1204
1207 1205
1208 1206
1209 1207
1210 1208
1211 1209
1212 1210
1213 1211
1214 1212
1215 1213
1216 1214
1217 1215
1218 1216
1219 1217
1220 1218
1221 1219
1222 1220
1223 1221
1224 1222
1225 1223
1226 1224
1227 1225
1228 1226
1229 1227
1230 1228
1231 1229
1232 1230
1233 1231
1234 1232
1235 1233
1236 1234
1237 1235
1238 1236
1239 1237
1240 1238
1241 1239
1242 1240
1243 1241
1244 1242
1245 1243
1246 1244
1247 1245
1248 1246
1249 1247
1250 1248
1251 1249
1252 1250
1253 1251
1254 1252
1255 1253
1256 1254
1257 1255
1258 1256
1259 1257
1260 1258
1261 1259
1262 1260
1263 1261
1264 1262
1265 1263
1266 1264
1267 1265
1268 1266
1269 1267
1270 1268
1271 1269
1272 1270
1273 1271
1274 1272
1275 1273
1276 1274
1277 1275
1278 1276
1279 1277
1280 1278
1281 1279
1282 1280
1283 1281
1284 1282
1285 1283
1286 1284
1287 1285
1288 1286
1289 1287
1290 1288
1291 1289
1292 1290
1293 1291
1294 1292
1295 1293
1296 1294
1297 1295
1298 1296
1299 1297
1300 1298
1301 1299
1302 1300
1303 1301
1304 1302
1305 1303
1306 1304
1307 1305
1308 1306
1309 1307
1310 1308
1311 1309
1312 1310
1313 1311
1314 1312
1315 1313
1316 1314
1317 1315
1318 1316
1319 1317
1320 1318
1321 1319
1322 1320
1323 1321
1324 1322
1325 1323
1326 1324
1327 1325
1328 1326
1329 1327
1330 1328
1331 1329
1332 1330
1333 1331
1334 1332
1335 1333
1336 1334
1337 1335
1338 1336
1339 1337
1340 1338
1341 1339
1342 1340
1343 1341
1344 1342
1345 1343
1346 1344
1347 1345
1348 1346
1349 1347
1350 1348
1351 1349
1352 1350
1353 1351
1354 1352
1355 1353
1356 1354
1357 1355
1358 1356
1359 1357
1360 1358
1361 1359
1362 1360
1363 1361
1364 1362
1365 1363
1366 1364
1367 1365
1368 1366
1369 1367
1370 1368
1371 1369
1372 1370
1373 1371
1374 1372
1375 1373
1376 1374
1377 1375
1378 1376
1379 1377
1380 1378
1381 1379
138
```

```

2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan migrate:refresh
Rolling back: 2022_04_09_224832_add_deleted_at_column_to_posts_table
Rolled back: 2022_04_09_224832_add_deleted_at_column_to_posts_table (47.02ms)
Rolling back: 2022_04_08_011859_add_is_admin_column_to_posts_table
Rolled back: 2022_04_08_011859_add_is_admin_column_to_posts_table (12.06ms)
Rolling back: 2022_04_08_010006_create_posts_table
Rolled back: 2022_04_08_010006_create_posts_table (17.74ms)
Rolling back: 2019_12_14_000001_create_personal_access_tokens_table
Rolled back: 2019_12_14_000001_create_personal_access_tokens_table (23.59ms)
Rolling back: 2019_08_19_000000_create_failed_jobs_table
Rolled back: 2019_08_19_000000_create_failed_jobs_table (33.88ms)
Migrated: 2022_04_08_010006_create_posts_table (29.70ms)
Migrating: 2022_04_08_011859_add_is_admin_column_to_posts_table
Migrated: 2022_04_08_011859_add_is_admin_column_to_posts_table (10.40ms)
Migrating: 2022_04_09_224832_add_deleted_at_column_to_posts_table
Migrated: 2022_04_09_224832_add_deleted_at_column_to_posts_table (9.86ms)
  
```

3. Lihat hasilnya di phpMyAdmin

Server: 127.0.0.1 » Database: umn_genap2122_pemweb_w10_jetlive » Table: posts

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT
2	user_id	int(10)		UNSIGNED	No	None		
3	title	varchar(255)	utf8mb4_unicode_ci		No	None		
4	content	text	utf8mb4_unicode_ci		No	None		
5	created_at	timestamp			Yes	NULL		
6	updated_at	timestamp			Yes	NULL		
7	is_admin	tinyint(1)			No	0		
8	deleted_at	timestamp			Yes	NULL		

4. Insert dummy data users

id	name	email	email_verified_at	password	remember_token	created_at	updated_at
1	John Thor	john.thor@email.com	NULL	123	NULL	NULL	NULL

5. Insert dummy data posts

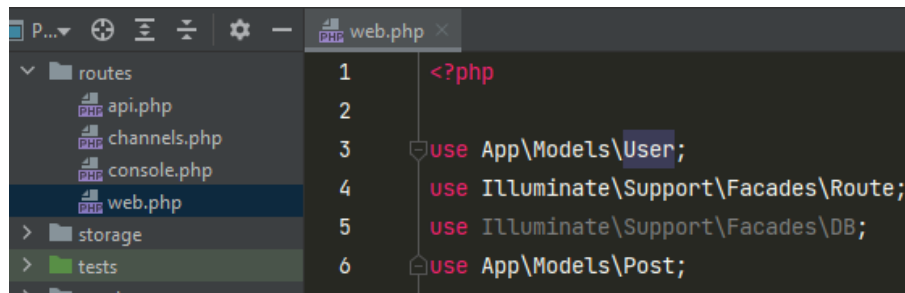
id	user_id	title	content	created_at	updated_at	is_admin	deleted_at
1	1	PHP is Good	PHP with Laravel is Awesome	NULL	NULL	0	NULL

6. Ketikkan kode berikut pada model User.php

```

44
45 public function post(){
46     return $this->hasOne( related: Post::class);
47     // hasOne(RELATEDMODEL, FOREIGNKEY);
48     // RELATEDMODEL adalah model yang ingin direlasikan dengan User, yaitu Post
49     // secara default FOREIGNKEY adalah MODELNAME_id
50     // dalam kasus ini user_id
51 }
52
  
```

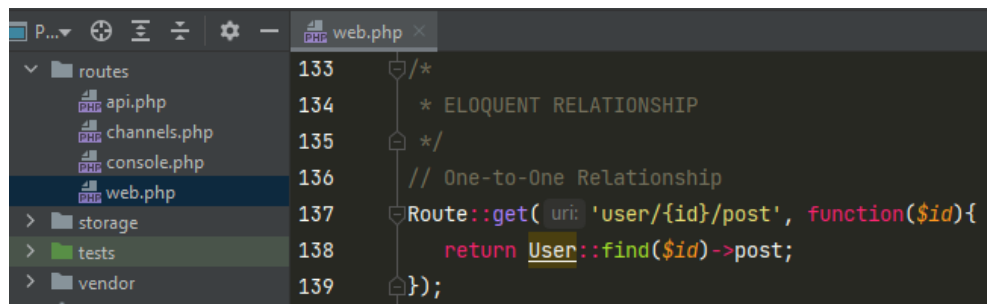
7. Tambahkan use App\User pada routes/web.php



```

1 <?php
2
3 use App\Models\User;
4 use Illuminate\Support\Facades\Route;
5 use Illuminate\Support\Facades\DB;
6 use App\Models\Post;
  
```

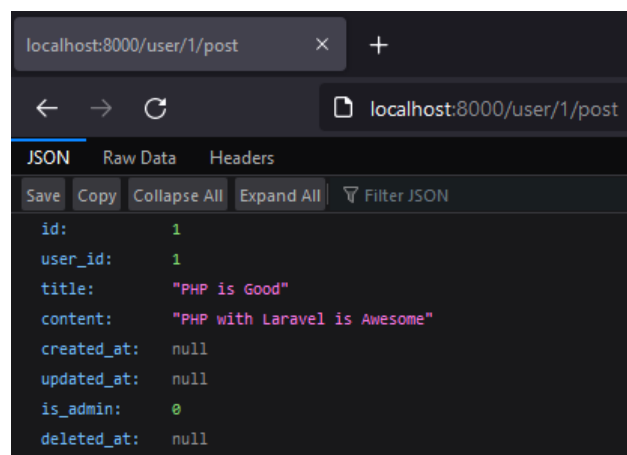
8. Tambahkan route berikut pada routes/web.php



```

133 /*
134  * ELOQUENT RELATIONSHIP
135  */
136 // One-to-One Relationship
137 Route::get( uri: 'user/{id}/post', function($id){
138     return User::find($id)->post;
139 });
  
```

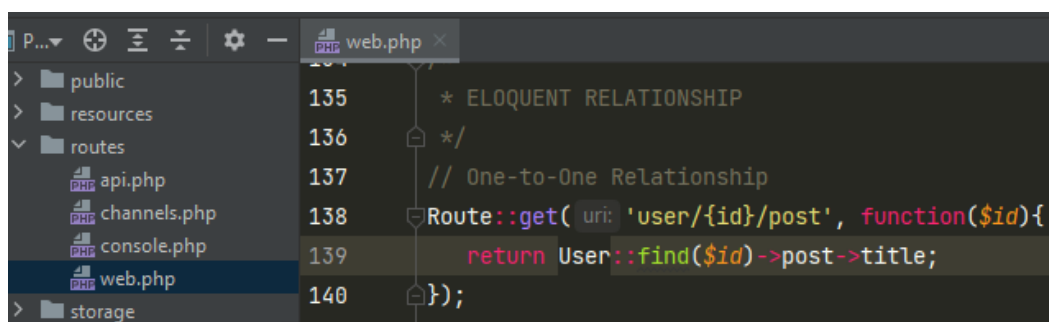
9. Lihat hasilnya di browser



```

{
  "id": 1,
  "user_id": 1,
  "title": "PHP is Good",
  "content": "PHP with Laravel is Awesome",
  "created_at": null,
  "updated_at": null,
  "is_admin": 0,
  "deleted_at": null
}
  
```

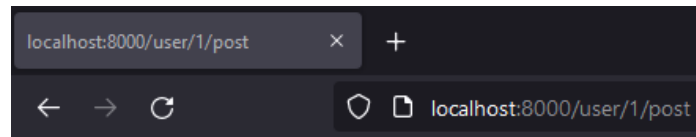
10. Jika kita ingin menampilkan salah satu kolom, misal judul, tambahkan kode berikut:



```

135 * ELOQUENT RELATIONSHIP
136 */
137 // One-to-One Relationship
138 Route::get( uri: 'user/{id}/post', function($id){
139     return User::find($id)->post->title;
140 });
  
```

11. Lihat hasilnya di browser

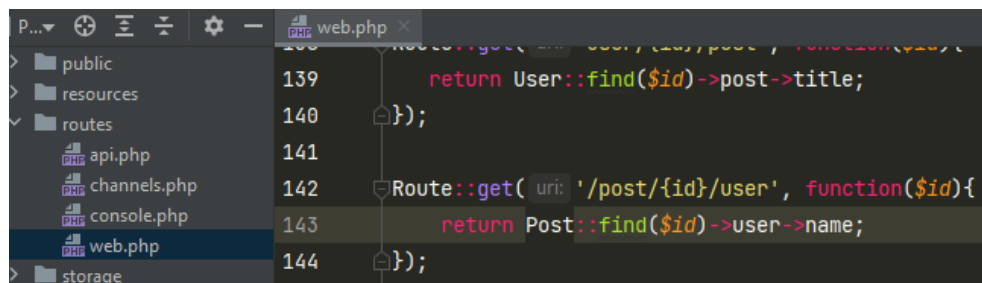


PHP is Good

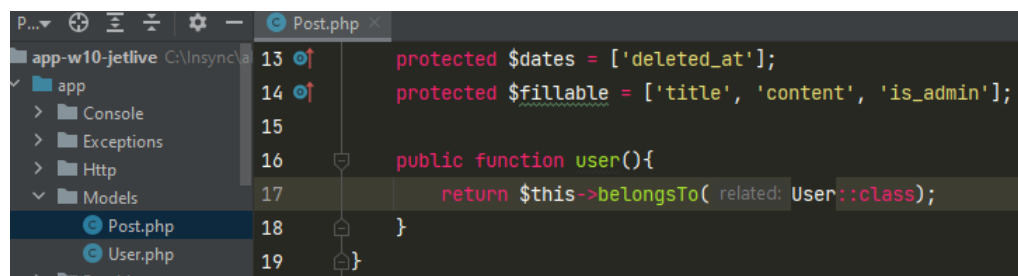
The Inverse Relation

(Target: Menampilkan user dari post tertentu)

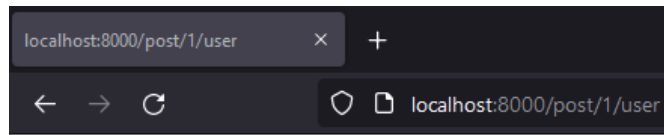
1. Buat route pada routes/web.php



2. Tambahkan kode berikut pada Post.php



3. Lihat hasilnya di browser



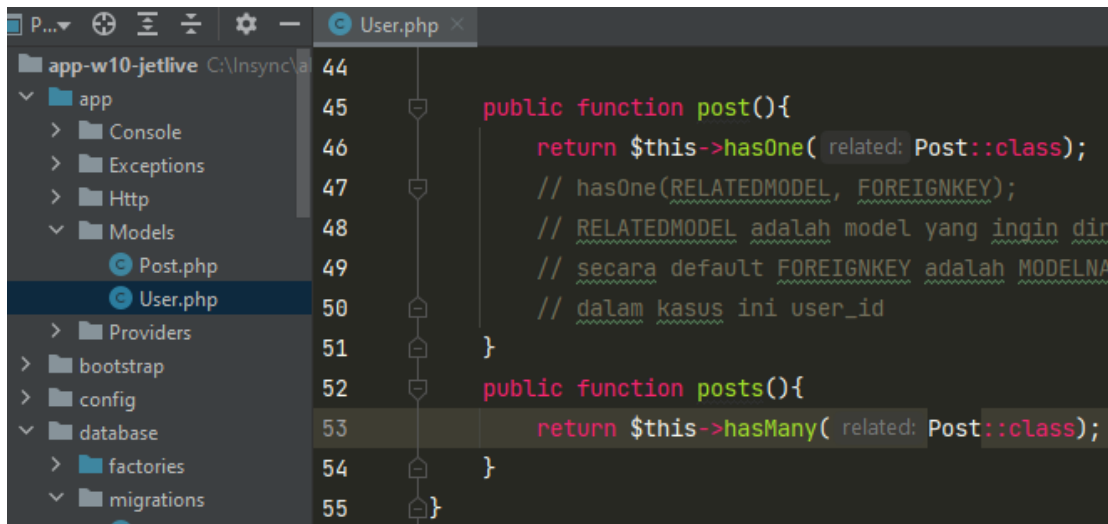
John Thor

One-to-Many Relationship

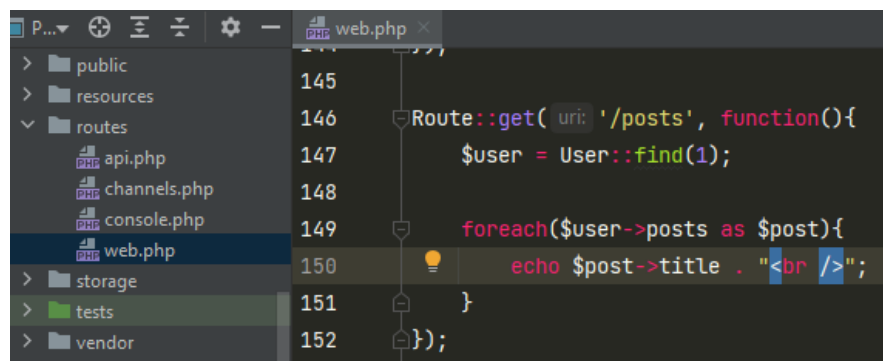
1. Insert data baru kedalam tabel posts melalui phpMyAdmin sehingga kita memiliki lebih dari 1 record.
Set user_id = 1.

id	user_id	title	content	created_at	updated_at	is_admin	deleted_at
1	1	PHP is Good	PHP with Laravel is Awesome	NULL	NULL	0	NULL
2	1	PHP with Laravel	This is the content	NULL	NULL	0	NULL

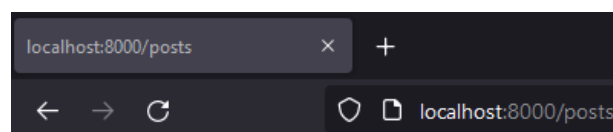
2. Buat sebuah method/function baru pada model User



3. Ketikkan kode berikut pada routes/web.php



4. Lihat hasilnya di browser



PHP is Good
 PHP with Laravel

5. Kerjakan Tugas 1.

Many-to-Many Relationship

1. Buat model Role menggunakan PHP artisan command:

```
2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan make:model Role -m
Model created successfully.
Created Migration: 2022_04_17_223912_create_roles_table
```

2. Buat tabel role_user menggunakan migration

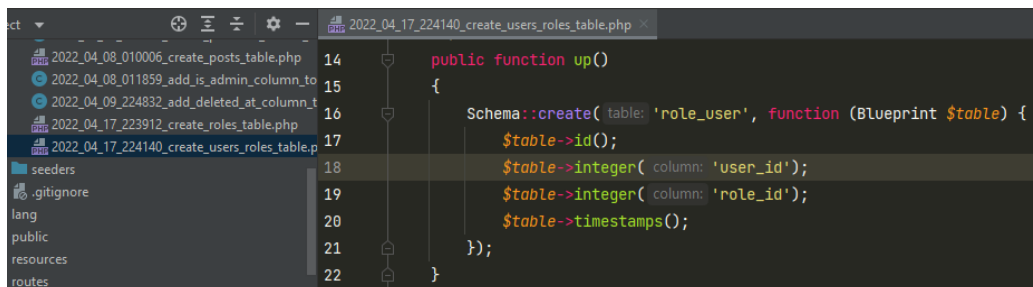
```
2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan make:migration create_users_roles_table --create=role_user
Created Migration: 2022_04_17_224140_create_users_roles_table
```

3. Buka file migration untuk tabel roles, tambahkan kolom name.



```
14 public function up()
15 {
16     Schema::create( table: 'roles', function (Blueprint $table) {
17         $table->id();
18         $table->string( column: 'name', length: 20);
19         $table->timestamps();
20     });
21 }
```

4. Pada file migration untuk membuat tabel role_user, tambahkan dua kolom untuk menghubungkan antara tabel users dan roles, yaitu user_id dan role_id.



```
14 public function up()
15 {
16     Schema::create( table: 'role_user', function (Blueprint $table) {
17         $table->id();
18         $table->integer( column: 'user_id');
19         $table->integer( column: 'role_id');
20         $table->timestamps();
21     });
22 }
```

5. Lakukan migrate

```
2021-22\Pemrograman-Web\Week-10\app-w10-jetlive> php artisan migrate
Migrating: 2022_04_17_223912_create_roles_table
Migrated: 2022_04_17_223912_create_roles_table (36.41ms)
Migrating: 2022_04_17_224140_create_users_roles_table
Migrated: 2022_04_17_224140_create_users_roles_table (30.98ms)
```

6. Lihat hasilnya di phpMyAdmin tabel roles dan role_user sudah berhasil dibuat.

☐ roles

☐ role_user

7. Tambahkan data pada tabel users sehingga ada beberapa user.

id	name	email	email_verified_at	password	remember_token	created_at	updated_at
1	John Thor	john.thor@email.com	NULL	123	NULL	NULL	NULL
2	John Wick	john.wick@email.com	NULL	123	NULL	NULL	NULL
3	John Doel	john.doel@email.com	NULL	123	NULL	NULL	NULL

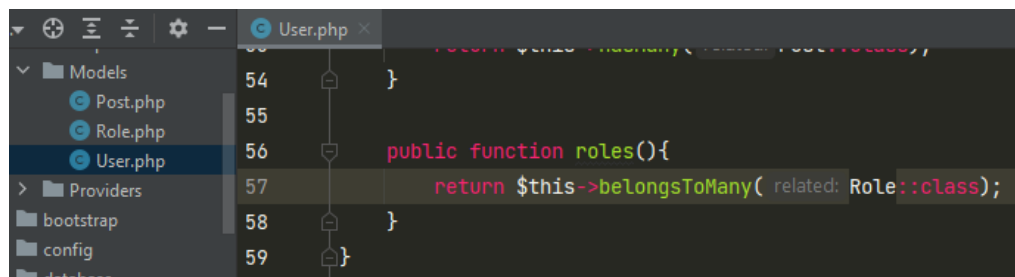
8. Tambahkan data pada tabel roles, yaitu administrator dan subscriber

id	name	created_at	updated_at
1	administrator	NULL	NULL
2	subscriber	NULL	NULL

9. Tambahkan data pada tabel role_user, kita akan menjadikan user_id 1 sebagai administrator dan user_id 2 menjadi subscriber, user_id 3 menjadi administrator sekaligus subscriber.

id	user_id	role_id	created_at	updated_at
1	1	1	NULL	NULL
2	2	2	NULL	NULL
3	3	1	NULL	NULL
4	3	2	NULL	NULL

10. Pada model User tambahkan method berikut:

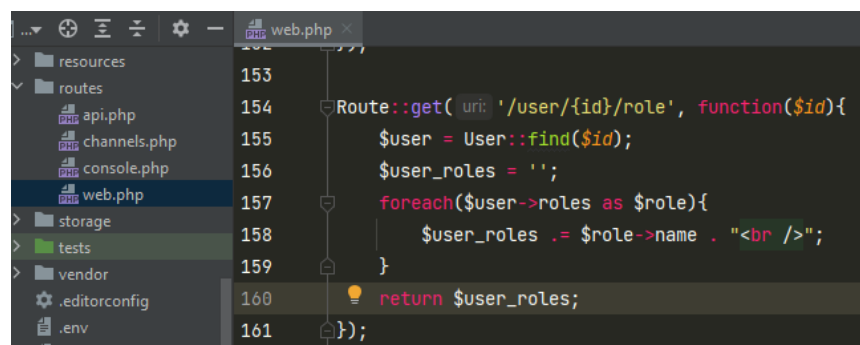


```

54 }
55
56 public function roles(){
57     return $this->belongsToMany(related: Role::class);
58 }
59

```

11. Tambahkan route baru untuk menampilkan role berdasarkan user id yang di-supply pada URL.

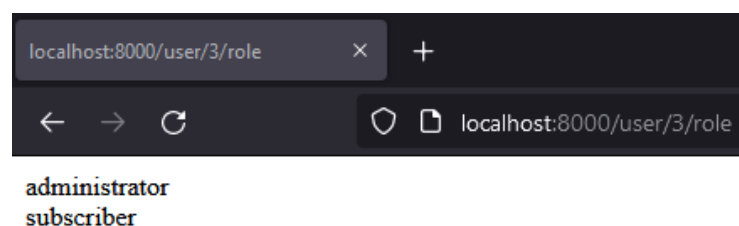


```

153
154 Route::get('uri: '/user/{id}/role', function($id){
155     $user = User::find($id);
156     $user_roles = '';
157     foreach($user->roles as $role){
158         $user_roles .= $role->name . "<br />";
159     }
160     return $user_roles;
161 });

```

12. Coba hasilnya di browser

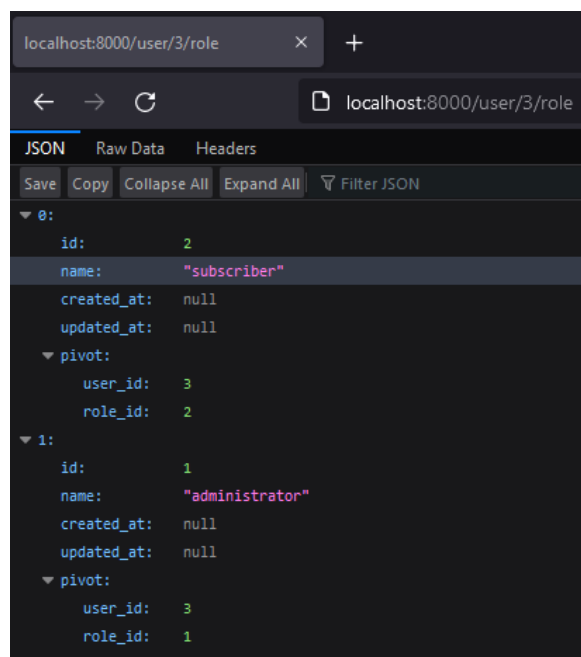


13. Cara lain, ubah kode pada route tersebut menjadi seperti berikut:

```

152 });
153
154 Route::get('user/{id}/role', function($id){
155     $user = User::find($id)->roles()->orderBy('id', 'desc')->get();
156     return $user;
157 // $user = User::find($id);
158 // $user_roles = '';
159 // foreach($user->roles as $role){
160 //     $user_roles .= $role->name . "<br />";
161 // }
162 // return $user_roles;
163 });
  
```

14. Lihat hasilnya di browser



15. Kerjakan Tugas 2

TUGAS

1. Apa yang baru saja kita lakukan menampilkan kumpulan post title dari user_id 1. Coba modifikasi sehingga pilihan user_id menjadi dinamis dengan route.
Misal: /user/3/posts akan menampilkan semua post title dari user_id 3.
2. Buat sebuah route '/role/{id}/user' yang akan menampilkan daftar user yang memiliki role berdasarkan id tertentu. Contoh hasil di browser:

```
localhost:8000/role/1/user × +
← → ↻ localhost:8000/role/1/user
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
▼ 0:
  id: 1
  name: "John Thor"
  email: "john.thor@email.com"
  email_verified_at: null
  created_at: null
  updated_at: null
  pivot:
    role_id: 1
    user_id: 1
▼ 1:
  id: 3
  name: "John Doel"
  email: "john.doel@email.com"
  email_verified_at: null
  created_at: null
  updated_at: null
  pivot:
    role_id: 1
    user_id: 3
```

MINGGU 12

FORMS & VALIDATION

DESKRIPSI TEMA

Mahasiswa mempraktikkan penggunaan form, melakukan upload file dan validasi form menggunakan framework Laravel.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu menerapkan form, upload file dan validasi menggunakan Framework Laravel.

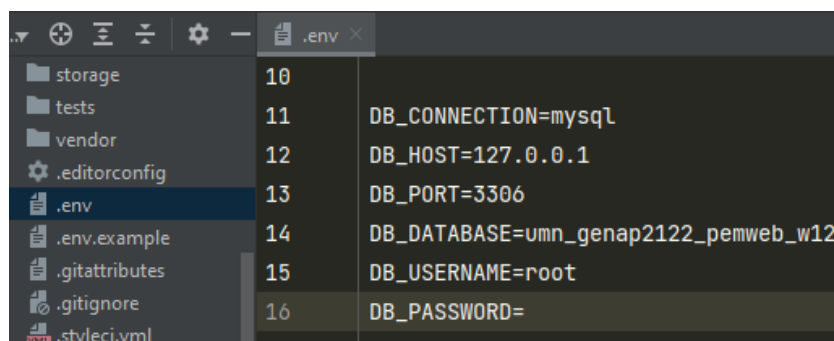
PERALATAN YANG DIGUNAKAN

1. Composer
2. XAMPP
3. Code Editor
4. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

Persiapan

1. Buatlah sebuah project Laravel baru dengan nama app-w12.
2. Sesuaikan konfigurasi database pada file .env



3. Buat model Student beserta file migration menggunakan command berikut:

```
2021-22\Pemrograman-Web\Week-12\app-w12> php artisan make:model Student -m  
Model created successfully.  
Created Migration: 2022_05_09_011735_create_students_table
```

4. Tambahkan kolom pada tabel students seperti berikut

```

2022_05_09_011735_create_students_table.php
14 public function up()
15 {
16     Schema::create( table: 'students', function (Blueprint $table) {
17         $table->id();
18         $table->string( column: 'nim', length: 10)->unique();
19         $table->string( column: 'nama', length: 50);
20         $table->string( column: 'prodi', length: 15);
21         $table->string( column: 'photo')->nullable();
22         $table->timestamps();
23     });
24 }

```

5. Lakukan migrate

```

2021-22\Pemrograman-Web\Week-12\app-w12> php artisan migrate
Migration table created successfully.
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (60.21ms)
Migrating: 2019_12_14_000001_create_personal_access_tokens_table
Migrated: 2019_12_14_000001_create_personal_access_tokens_table (77.41ms)
Migrating: 2022_05_09_011735_create_students_table
Migrated: 2022_05_09_011735_create_students_table (61.87ms)

```

6. Buat StudentController melalui terminal

```

2021-22\Pemrograman-Web\Week-12\app-w12> php artisan make:controller StudentController --resource

```

7. Buat route pada file routes/web.php

```

Project web.php
17 Route::get( uri: '/', function () {
18     return view( view: 'welcome');
19 });
20
21 Route::resource( name: 'students', controller: StudentController::class);
22

```

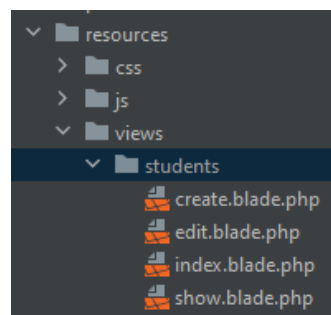
8. Check route menggunakan command

```

rograman-Web\Week-12\app-w12> php artisan route:list

GET|HEAD      / .....
POST          _ignition/execute-solution ignition.executeSolution > Spatie\LaravelIg...
GET|HEAD      _ignition/health-check ignition.healthCheck > Spatie\LaravelIgnition ...
POST          _ignition/update-config ignition.updateConfig > Spatie\LaravelIgnition...
GET|HEAD      api/user .....
GET|HEAD      sanctum/csrf-cookie ..... Laravel\Sanctum > CsrfCookieController@show
GET|HEAD      students ..... students.index > StudentController@index
POST          students ..... students.store > StudentController@store
GET|HEAD      students/create ..... students.create > StudentController@create
GET|HEAD      students/{student} ..... students.show > StudentController@show
PUT|PATCH    students/{student} ..... students.update > StudentController@update
DELETE        students/{student} ..... students.destroy > StudentController@destroy
GET|HEAD      students/{student}/edit ..... students.edit > StudentController@edit
  
```

9. Buat sebuah folder "students" dengan 4 buah file kosong dengan nama seperti berikut pada folder views.



Persiapan View

1. Tuliskan kode berikut pada file create.blade.php

```

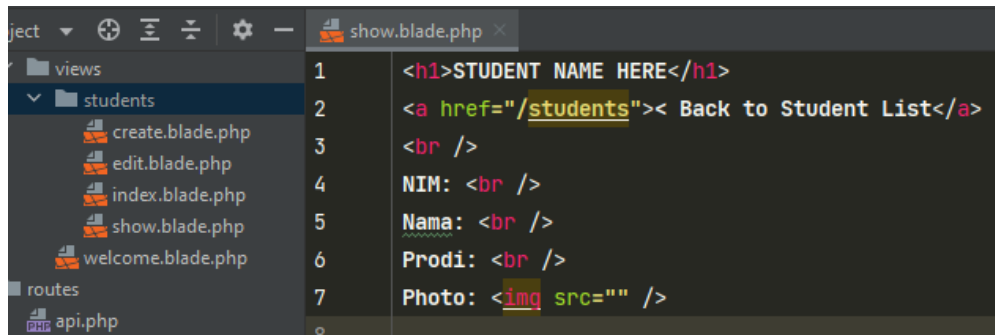
1 <h1>Create New Student</h1>
2 <form action="/students" method="post">
3     NIM: <input type="text" name="nim" /><br />
4     Nama: <input type="text" name="nama" /><br />
5     Prodi: <input type="text" name="prodi" /><br />
6     <button type="submit">Submit</button>
7 </form>
  
```

2. Tuliskan kode berikut pada file edit.blade.php

```

1 <h1>Edit Student</h1>
2 <form action="/students/STUDENT_ID" method="post">
3     NIM: <input type="text" name="nim" value="" /><br />
4     Nama: <input type="text" name="nama" value="" /><br />
5     Prodi: <input type="text" name="prodi" value="" /><br />
6     <button type="submit">Submit</button>
7 </form>
  
```

3. Tuliskan kode berikut pada file show.blade.php

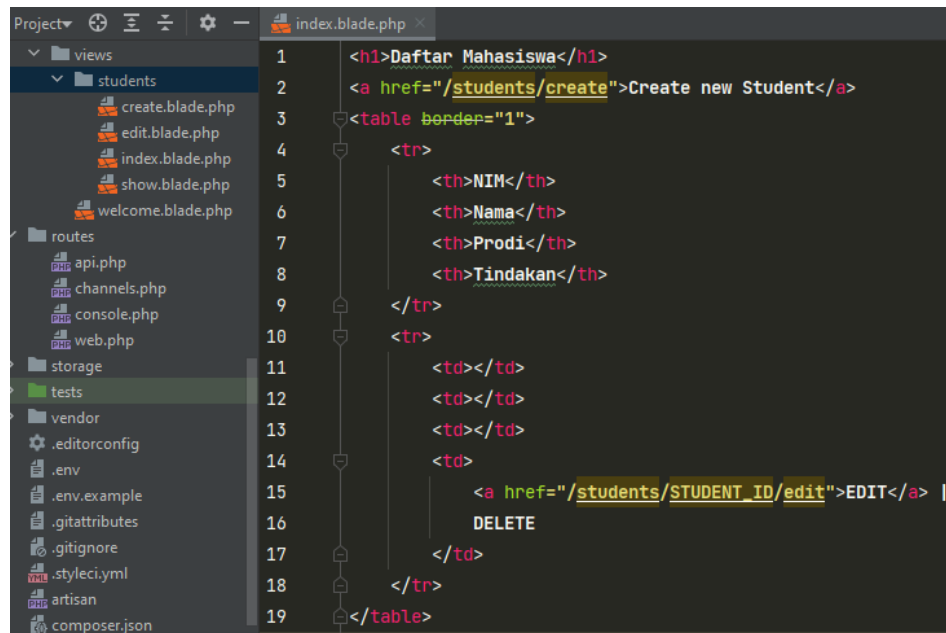


```

1 <h1>STUDENT NAME HERE</h1>
2 <a href="/students">< Back to Student List</a>
3 <br />
4 NIM: <br />
5 Nama: <br />
6 Prodi: <br />
7 Photo: <img src="" />

```

4. Tuliskan kode berikut pada file index.blade.php



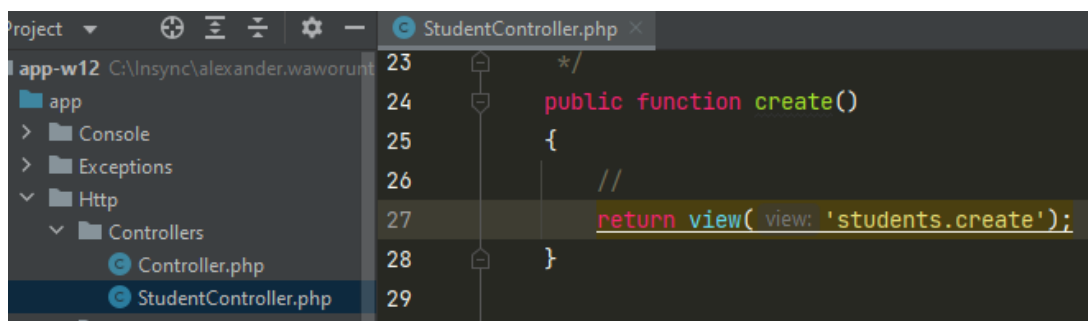
```

1 <h1>Daftar Mahasiswa</h1>
2 <a href="/students/create">Create new Student</a>
3 <table border="1">
4   <tr>
5     <th>NIM</th>
6     <th>Nama</th>
7     <th>Prodi</th>
8     <th>Tindakan</th>
9   </tr>
10  <tr>
11    <td></td>
12    <td></td>
13    <td></td>
14    <td>
15      <a href="/students/STUDENT_ID/edit">EDIT</a> |
16      DELETE
17    </td>
18  </tr>
19 </table>

```

CREATE New Student

1. Tambahkan kode berikut pada StudentController

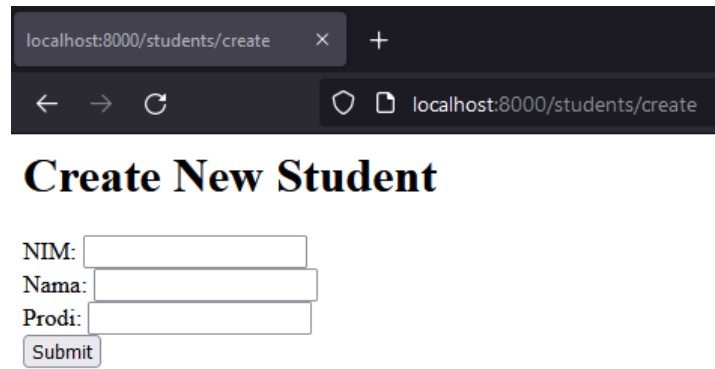


```

23 */
24 public function create()
25 {
26     //
27     return view( view: 'students.create');
28 }
29

```

2. Jalankan command php artisan serve, dan lihat hasilnya di browser



localhost:8000/students/create

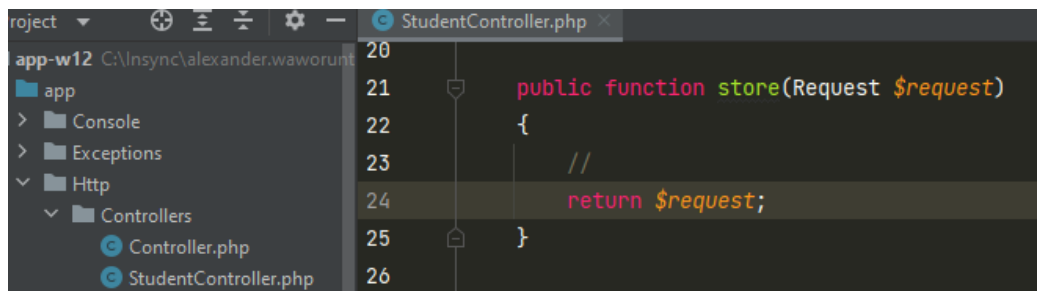
Create New Student

NIM:

Nama:

Prodi:

3. Tambahkan kode berikut pada StudentController

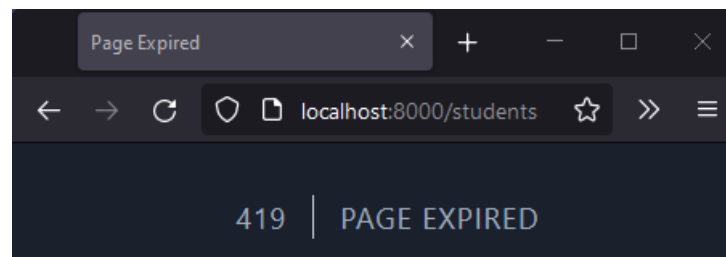


```

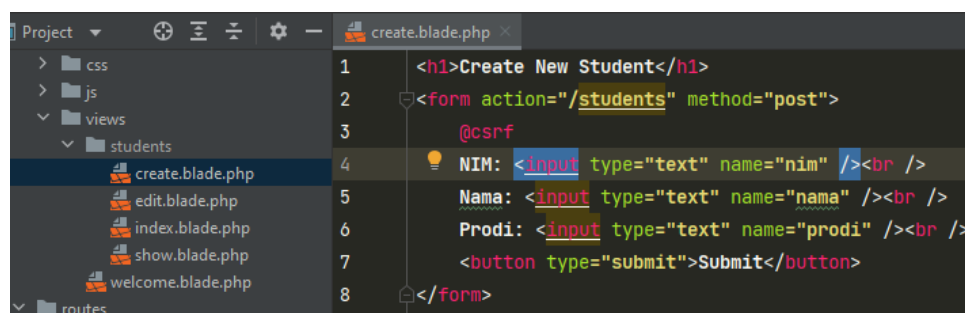
20
21 public function store(Request $request)
22 {
23     //
24     return $request;
25 }
26

```

4. Coba submit form Create new Student tadi.



5. Tambahkan kode berikut pada index.blade.php

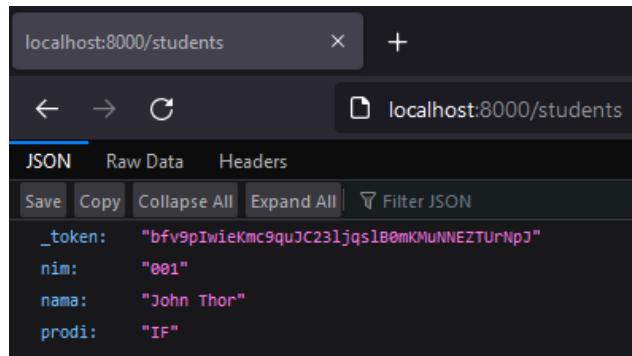


```

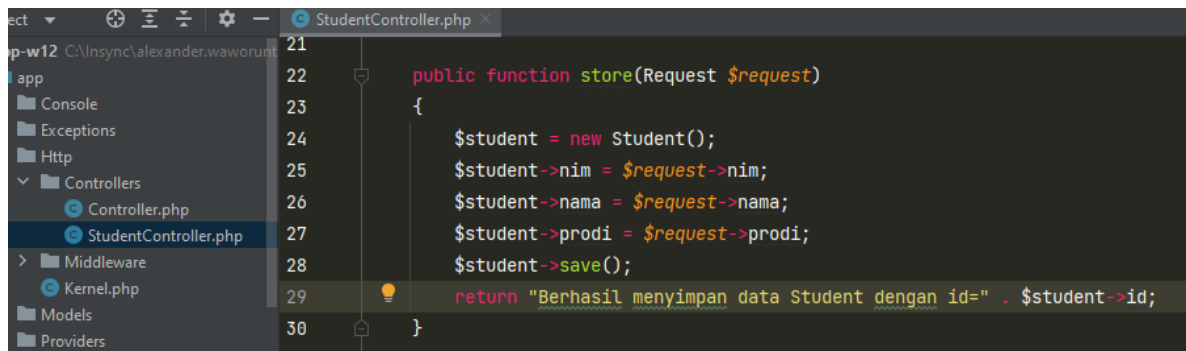
1 <h1>Create New Student</h1>
2 <form action="/students" method="post">
3     @csrf
4     NIM: <input type="text" name="nim" /><br />
5     Nama: <input type="text" name="nama" /><br />
6     Prodi: <input type="text" name="prodi" /><br />
7     <button type="submit">Submit</button>
8 </form>

```

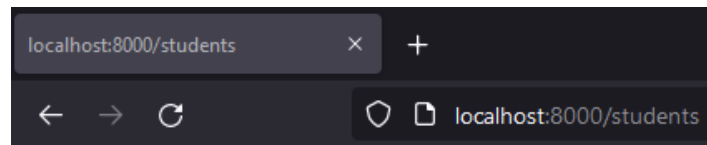
6. Coba submit kembali form Create new Student



7. Tambahkan kode berikut pada StudentController untuk menyimpan data student ke database



8. Coba submit form new Student



Berhasil menyimpan data Student dengan id=1

9. Lihat hasilnya di phpMyAdmin

	id	nim	nama	prodi	photo	created_at	updated_at
1	1	001	John Thor	IF	NULL	2022-05-09 01:53:11	2022-05-09 01:53:11

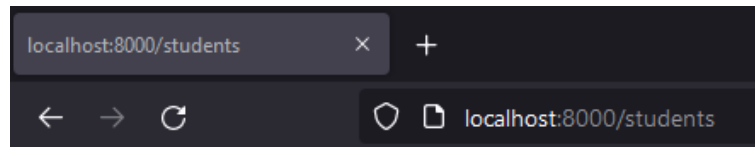
10. Ubah kode pada StudentController agar setelah menyimpan ke database, otomatis redirect ke daftar mahasiswa

```

21
22 public function store(Request $request)
23 {
24     $student = new Student();
25     $student->nim = $request->nim;
26     $student->nama = $request->nama;
27     $student->prodi = $request->prodi;
28     $student->save();
29     // return "Berhasil menyimpan data Student dengan id=" . $student->id;
30     return redirect(to: '/students');
31 }

```

11. Coba submit kembali form Create new Student.



RETRIEVE Students

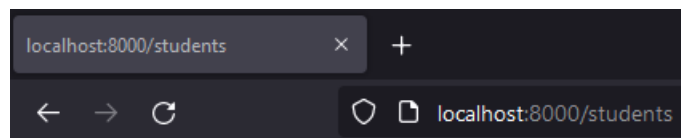
1. Tuliskan kode berikut pada StudentController untuk menampilkan view index

```

10
11 public function index()
12 {
13     return view( view: 'students.index');
14 }
15
16 public function create()

```

2. Coba buka di browser



Daftar Mahasiswa

[Create new Student](#)

NIM	Nama	Prodi	Tindakan
			EDIT DELETE

3. Tuliskan kode berikut pada StudentController untuk mengambil data dari database dan mengirimkan ke view index.

```

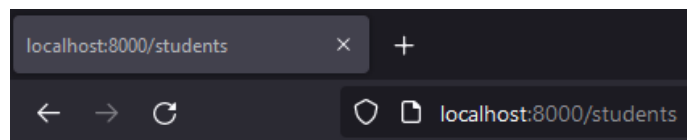
StudentController.php
10
11 public function index()
12 {
13     $students = Student::all();
14     return view( view: 'students.index', ['students' => $students]);
15 }
16
  
```

4. Tambahkan kode berikut pada index.blade.php untuk menampilkan daftar mahasiswa

```

index.blade.php
9 </tr>
10 @foreach($students as $student)
11 <tr>
12     <td>{{ $student->nim }}</td>
13     <td>{{ $student->nama }}</td>
14     <td>{{ $student->prodi }}</td>
15     <td>
16         <a href="/students/{{ $student->id }}">SHOW</a> |
17         <a href="/students/{{ $student->id }}/edit">EDIT</a> |
18         DELETE
19     </td>
20 </tr>
21 @endforeach
22 </table>
  
```

5. Lihat hasilnya di browser



Daftar Mahasiswa

[Create new Student](#)

NIM	Nama	Prodi	Tindakan		
001	John Thor	IF	SHOW	EDIT	DELETE
002	John Wick	TK	SHOW	EDIT	DELETE

6. Tambahkan kode berikut pada StudentController untuk menampilkan view show

```

StudentController.php
34 public function show($id)
35 {
36     $student = Student::findOrFail($id);
37     return view( view: 'students.show', ['student' => $student]);
38 }
39
40 public function edit($id)
  
```

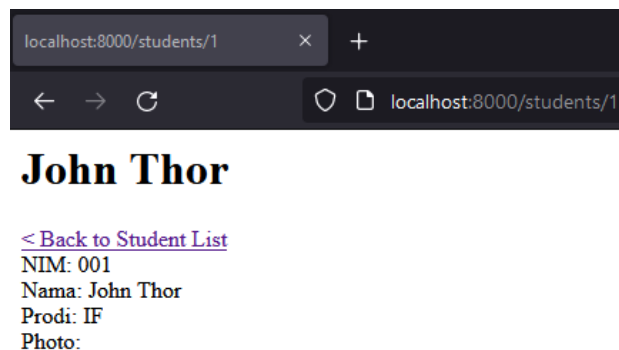
7. Tambahkan kode berikut pada show.blade.php

```

show.blade.php
1 <h1>{{ $student->nama }}</h1>
2 <a href="/students">< Back to Student List</a>
3 <br />
4 NIM: {{ $student->nim }}<br />
5 Nama: {{ $student->nama }}<br />
6 Prodi: {{ $student->prodi }}<br />
7 Photo: <img src="" />

```

8. Coba hasilnya di browser, dengan meng-klik link SHOW pada index.



EDIT Student

1. Tambahkan kode berikut pada StudentController untuk menampilkan view Edit.

```

StudentController.php
39 public function edit($id)
40 {
41     $student = Student::findOrFail($id);
42     return view('students.edit', ['student' => $student]);
43 }

```

2. Tambahkan kode berikut pada edit.blade.php untuk menampilkan form edit dengan data lama yang berasal dari database.

```

edit.blade.php
1 <h1>Edit Student</h1>
2 <form action="/students/{{ $student->id }}" method="post">
3     NIM: <input type="text" name="nim" value="{{ $student->nim }}" /><br />
4     Nama: <input type="text" name="nama" value="{{ $student->nama }}" /><br />
5     Prodi: <input type="text" name="prodi" value="{{ $student->prodi }}" /><br />
6     <button type="submit">Submit</button>
7 </form>

```

3. Coba lihat hasilnya di browser dengan meng-klik link EDIT pada index.

localhost:8000/students/1/edit

Edit Student

NIM:

Nama:

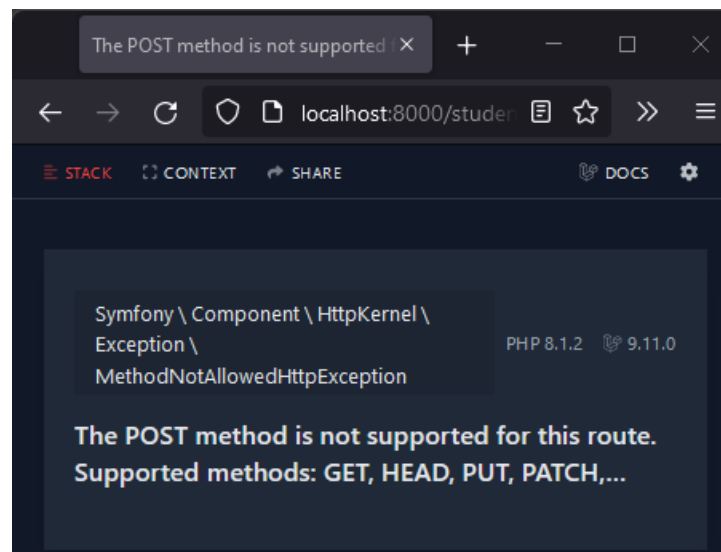
Prodi:

4. Tambahkan kode berikut pada StudentController untuk menyimpan perubahan

```

StudentController.php
45 public function update(Request $request, $id)
46 {
47     $student = Student::findOrFail($id);
48     $student->nim = $request->nim;
49     $student->nama = $request->nama;
50     $student->prodi = $request->prodi;
51     $student->save();
52     return redirect(to: '/students');
53 }
  
```

5. Coba lakukan perubahan data Student - ERROR

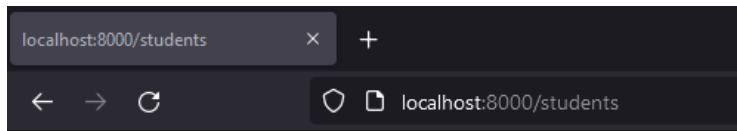


6. Tambahkan kode berikut pada edit.blade.php

```

edit.blade.php
1 <h1>Edit Student</h1>
2 <form action="/students/{{ $student->id }}" method="post">
3     @method('PUT')
4     @csrf
5     NIM: <input type="text" name="nim" value="{{ $student->nim }}" /><br />
6     Nama: <input type="text" name="nama" value="{{ $student->nama }}" /><br />
7     Prodi: <input type="text" name="prodi" value="{{ $student->prodi }}" /><br />
8     <button type="submit">Submit</button>
9 </form>
  
```

7. Coba kembali melakukan update data



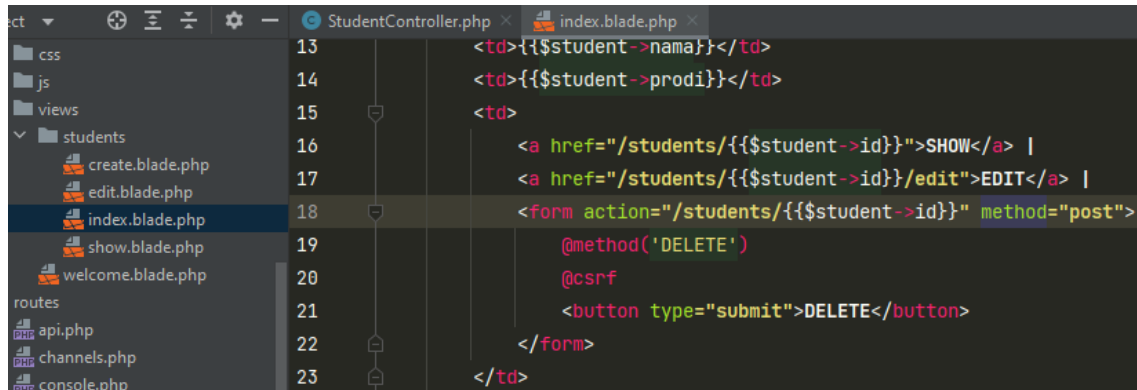
Daftar Mahasiswa

[Create new Student](#)

NIM	Nama	Prodi	Tindakan
001 EDITED	John Thor EDITED	IF EDITED	SHOW EDIT DELETE
002	John Wick	TK	SHOW EDIT DELETE

DELETE Student

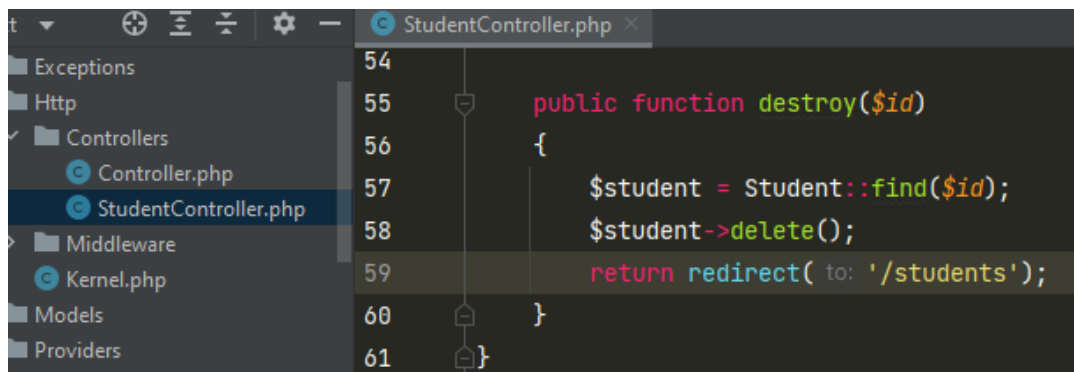
1. Tambahkan kode berikut pada index.blade.php untuk membuat action delete



```

13 | 
14 | <td>{{ $student->nama }}</td>
15 | 
16 | <td>{{ $student->prodi }}</td>
17 | 
18 | <td>
19 |     <a href="/students/{{ $student->id }}">SHOW</a> |
20 |     <a href="/students/{{ $student->id }}/edit">EDIT</a> |
21 |     <form action="/students/{{ $student->id }}" method="post">
22 |         @method('DELETE')
23 |         @csrf
24 |         <button type="submit">DELETE</button>
25 |     </form>
26 | </td>
  
```

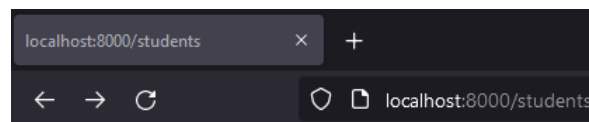
2. Tambahkan kode berikut pada StudentController untuk menghapus data



```

54 | 
55 | public function destroy($id)
56 | {
57 |     $student = Student::find($id);
58 |     $student->delete();
59 |     return redirect(to: '/students');
60 | }
61 | 
  
```

3. Coba lakukan delete data



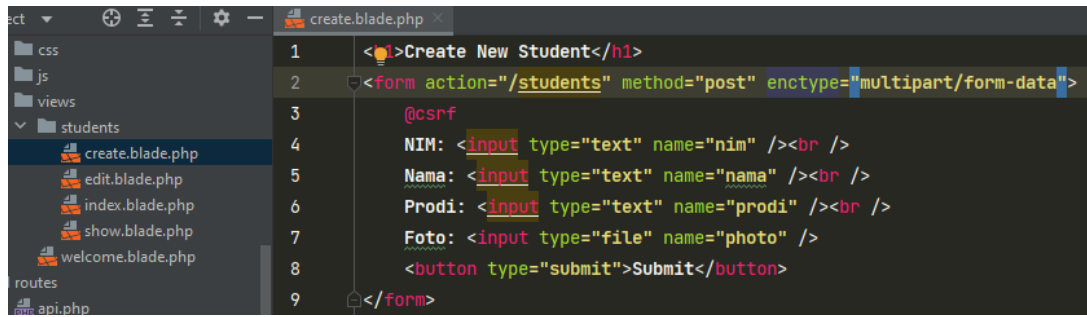
Daftar Mahasiswa

[Create new Student](#)

NIM	Nama	Prodi	Tindakan
002	John Wick	TK	SHOW EDIT DELETE

File Upload

1. Tambahkan kode berikut pada create.blade.php untuk melakukan file Upload

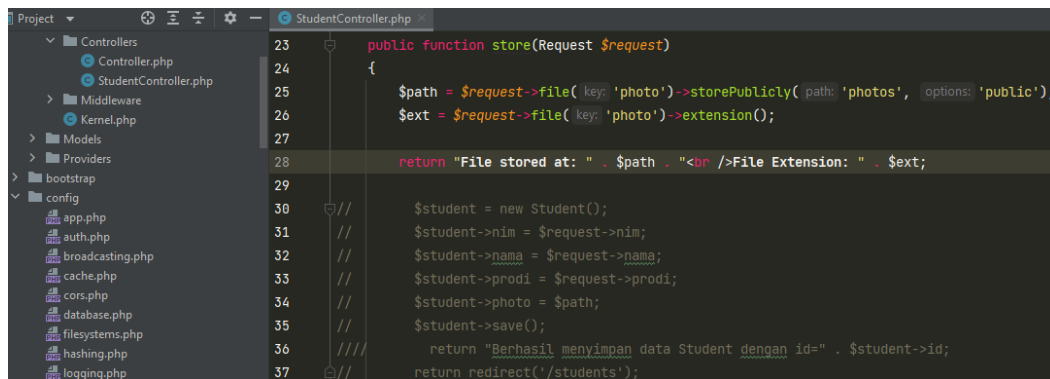


```

1 <h1>Create New Student</h1>
2 <form action="/students" method="post" enctype="multipart/form-data">
3     @csrf
4     NIM: <input type="text" name="nim" /><br />
5     Nama: <input type="text" name="nama" /><br />
6     Prodi: <input type="text" name="prodi" /><br />
7     Foto: <input type="file" name="photo" />
8     <button type="submit">Submit</button>
9 </form>

```

2. Tambahkan kode berikut pada StudentController untuk menyimpan file

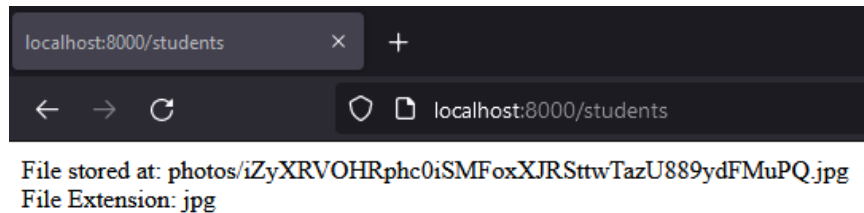


```

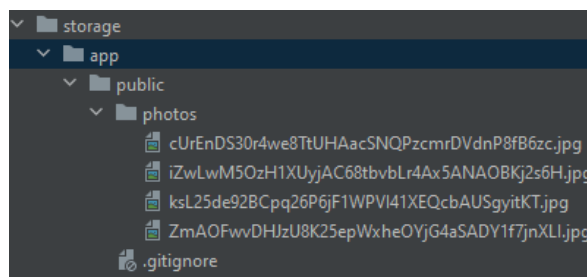
23 public function store(Request $request)
24 {
25     $path = $request->file( key: 'photo')->storePublicly( path: 'photos', options: 'public');
26     $ext = $request->file( key: 'photo')->extension();
27
28     return "File stored at: " . $path . "<br />File Extension: " . $ext;
29
30     // $student = new Student();
31     // $student->nim = $request->nim;
32     // $student->nama = $request->nama;
33     // $student->prodi = $request->prodi;
34     // $student->photo = $path;
35     // $student->save();
36     /// return "Berhasil menyimpan data Student dengan id=" . $student->id;
37     return redirect('/students');

```

3. Coba lakukan upload file



4. Perhatikan folder storage, file yang diupload disimpan pada folder storage/app/photos



5. Ubah StudentController.php untuk menyimpan path ke database

```

23 public function store(Request $request)
24 {
25     $path = $request->file( key: 'photo')->storePublicly( path: 'photos', options: 'public');
26     $ext = $request->file( key: 'photo')->extension();
27
28     $student = new Student();
29     $student->nim = $request->nim;
30     $student->nama = $request->nama;
31     $student->prodi = $request->prodi;
32     $student->photo = $path;
33     $student->save();
34     // return "Berhasil menyimpan data Student dengan id=" . $student->id;
35     return redirect( to: '/students');
36 }

```

6. Tambahkan kode berikut pada StudentController untuk mengirimkan informasi foto ke view

```

38 public function show($id)
39 {
40     $student = Student::findOrFail($id);
41     $photo = Storage::url($student->photo);
42     return view( view: 'students.show', ['student' => $student, 'photo' => $photo]);
43 }

```

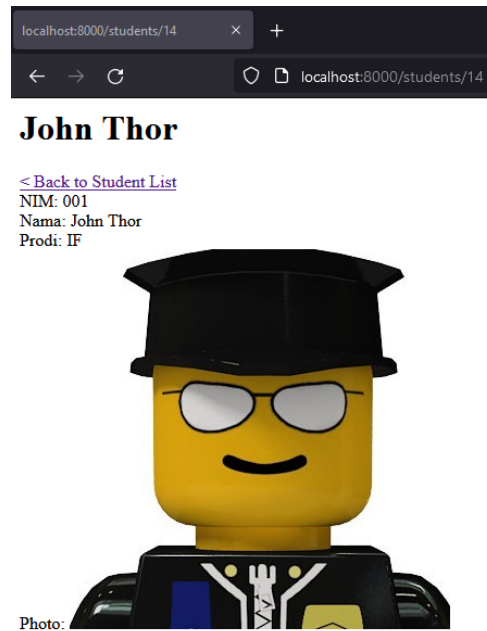
7. Tambahkan kode berikut pada show.blade.php

```

1 <h1>{{$student->nama}}</h1>
2 <a href="/students">< Back to Student List</a>
3 <br />
4 NIM: {{$student->nim}}<br />
5 Nama: {{$student->nama}}<br />
6 Prodi: {{$student->prodi}}<br />
7 Photo: 
8

```

8. Coba lihat hasilnya di browser



Form Validation

1. Tambahkan kode berikut pada StudentController

```

Project  app-w12  C:\Insync\alexander.waworuntu\...  StudentController.php
├── app
│   ├── Console
│   ├── Exceptions
│   ├── Handler.php
│   └── Http
│       ├── Controllers
│       │   ├── Controller.php
│       │   └── StudentController.php
│       ├── Middleware
│       └── Kernel.php
├── Models
├── Providers
├── bootstrap
└── config
    ├── app.php
    ├── auth.php
    ├── broadcasting.php
    ├── cache.php
    ├── cors.php
    ├── database.php
    ├── filesystems.php
    └── hashing.php

23  public function store(Request $request)
24  {
25      $this->validate($request, [
26          'nim' => 'required|max:10|unique:students',
27          'nama' => 'required|max:50'
28      ]);
29
30      $path = $request->file('photo')->storePublicly(['path' => 'photos', 'options' => 'public']);
31      $ext = $request->file('photo')->extension();
32
33      $student = new Student();
34      $student->nim = $request->nim;
35      $student->nama = $request->nama;
36      $student->prodi = $request->prodi;
37      $student->photo = $path;
38      $student->save();
39      // return "Berhasil menyimpan data Student dengan id=" . $student->id;
40      return redirect(to: '/students');
41  }
    
```

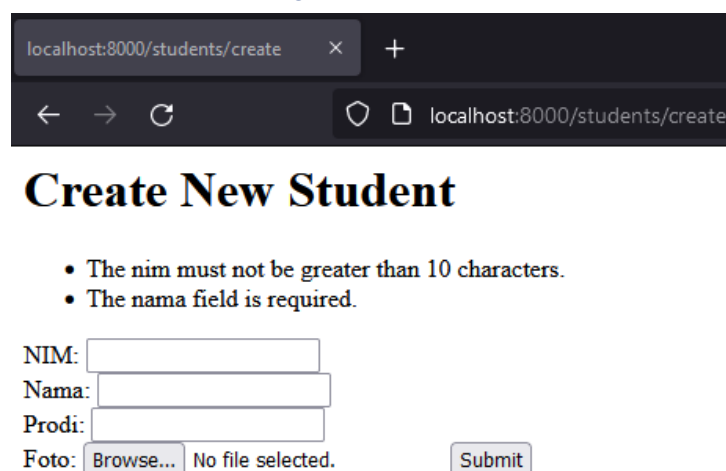
2. Tambahkan kode berikut pada create.blade.php

```

1      <h1>Create New Student</h1>
2
3      @if($errors)
4          <ul>
5              @foreach($errors->all() as $error)
6                  <li>{{$error}}</li>
7              @endforeach
8          </ul>
9      @endif
10
11     <form action="/students" method="post" enctype="multipart/form-data">

```

3. Coba hasilnya dengan men-submit form yang tidak valid untuk NIM dan Nama



localhost:8000/students/create

Create New Student

- The nim must not be greater than 10 characters.
- The nama field is required.

NIM:

Nama:

Prodi:

Foto: No file selected.

TUGAS

Buatlah sebuah project Laravel untuk melakukan CRUD pada sebuah tabel, dengan fitur file upload dan validasi keseluruhan field. Jumlah field/kolom ditentukan berdasarkan angka terakhir dari NIM Anda.

- NIM: xxx0, xxx1: Jumlah kolom: 7
- NIM: xxx2, xxx3: Jumlah kolom: 8
- NIM: xxx4, xxx5: Jumlah kolom: 9
- NIM: xxx6, xxx7: Jumlah kolom: 10
- NIM: xxx8, xxx9: Jumlah kolom: 11

Jumlah kolom termasuk (timestamp: created_at dan updated_at), Anda bisa memodifikasi tidak menggunakan timestamp. Nama tabel dan nama kolom bebas. Dikerjakan secara individu.

MINGGU 13

SECURITY & AUTHENTICATION



DESKRIPSI TEMA

Mahasiswa mempraktikkan implementasi Authentikasi pada proyek Laravel

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa mampu merancang dan membangun aplikasi web yang aman menggunakan Framework Laravel

PERALATAN YANG DIGUNAKAN

1. Code Editor
2. XAMPP
3. Composer
4. Web Browser

LANGKAH-LANGKAH PRAKTIKUM

Laravel Breeze

1. Buatlah sebuah project Laravel baru.
2. Instal Laravel Breeze menggunakan composer dengan command seperti berikut:

```
composer require laravel/breeze --dev
```

3. Selanjutnya, jalankan command berikut

```
php artisan breeze:install  
  
npm install  
npm run dev  
php artisan migrate
```

4. Coba jalankan aplikasi, perhatikan pada pojok kanan atas terdapat link Login dan Register

[Log in](#) [Register](#)

5. Coba lakukan register dan login.
6. Catatan: Sebelumnya jangan lupa untuk melakukan setup database terlebih dahulu.

Retrieving Authenticated User

1. Modifikasi file migration users menjadi seperti berikut

```
public function up()
{
    Schema::create( table: 'users', function (Blueprint $table) {
        $table->id();
        $table->string( column: 'name');
        $table->string( column: 'email')->unique();
        $table->integer( column: 'role_id')
            ->comment( comment: '1:admin, 2:member')->default( value: 2);
        $table->timestamp( column: 'email_verified_at')->nullable();
        $table->string( column: 'password');
        $table->rememberToken();
        $table->timestamps();
    });
}
```

2. Buat sebuah Model "Book" beserta dengan file migration-nya. Tuliskan kode berikut pada Model Book

```
class Book extends Model
{
    use HasFactory;

    public function user(){
        return $this->belongsTo( related: User::class);
    }
}
```

3. Tuliskan kode berikut pada Model User

```
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;

    public function books(){
        return $this->hasMany( related: Book::class);
    }
}
```

4. Buatlah sebuah view create book seperti berikut

```
<h1>Create New Book</h1>
<form action="/books" method="post">
  @csrf
  Judul:
  <input type="text" name="title" /><br />
  Keterangan:
  <textarea name="description"></textarea><br />
  <button type="submit">Submit</button>
</form>
```

5. Buatlah sebuah view index book seperti berikut

```
<h1>{{ $title }} Buku</h1>
<form action="/books/search" method="post">
  @csrf
  <input type="text" name="q" />
  <button type="submit">Search</button>
</form>
<table>
  <tr>
    <th>Judul</th>
    <th>Deskripsi</th>
    <th>User</th>
    <th>Tindakan</th>
  </tr>
  @foreach($books as $book)
  <tr>
    <td>{{ $book->title }}</td>
    <td>{{ $book->description }}</td>
    <td>{{ $book->user->name }}</td>
    <td>
      <a href="/books/{{ $book->id }}/edit">Edit</a>
    </td>
  </tr>
  @endforeach
</table>
```

6. Buat route untuk books. Perhatikan terdapat middleware('auth') merupakan mekanisme untuk memproteksi route sehingga hanya bisa diakses oleh user yang sudah login.

```
Route::resource( name: 'books', controller: BookController::class)
->middleware( middleware: 'auth');
```

7. Buat BookController, tuliskan kode berikut untuk function index(), create() dan store()

```
public function index()
{
    $books = Book::all();
    $title = "INDEX";
    return view( view: 'books.index', ['books' => $books, 'title'=> $title]);
}
```

```
public function create()
{
    //
    return view( view: 'books.create');
}
```

```
41 public function store(Request $request)
42 {
43     $book = new Book();
44     $book->title = $request->title;
45     $book->description = $request->description;
46     //     $book->user_id = $request->user()->id;
47     $book->user_id = Auth::id();
48     $book->save();
49     return redirect( to: '/books');
50 }
```

Perhatikan pada baris ke46 dan 47 merupakan cara untuk mendapatkan user_id dari user yang sedang login saat ini.

Gates & Authorizing Actions

1. Pada file AuthServiceProvider.php, tuliskan kode berikut. Disini kita mendefinisikan aturan bahwa yang bisa melakukan create book hanya user yang memiliki role_id = 1 dan yang bisa melakukan edit/update book adalah user yang membuat buku tersebut (user_id pada books sama dengan user_id yang sedang login saat ini).

```
public function boot()
{
    $this->registerPolicies();

    // Admin bisa melakukan create
    Gate::define(ability: 'create-book', function(User $user){
        return $user->role_id == 1;
    });

    Gate::define(ability: 'update-book', function(User $user, Book $book){
        return $user->id == $book->user_id;
    });

    // Member hanya bisa melihat
}
}
```

2. Modifikasi function store() pada BookController menjadi seperti berikut

```
public function store(Request $request)
{
    if(!Gate::allows(ability: 'create-book')){
        abort(code: '403');
    }

    $book = new Book();
    $book->title = $request->title;
    $book->description = $request->description;
    $book->user_id = $request->user()->id;
    $book->user_id = Auth::id();
    $book->save();
    return redirect(to: '/books');
}
```

3. Tuliskan kode berikut pada function edit() pada BookController

```
public function edit($id)
{
    $book = Book::find($id);
    if(!Gate::allows('update-book', $book)){
        return "Buku ini bukan punya kamu, qak bisa di edit";
    }else{
        return view('books.edit');
    }
}
```

TUGAS

Lengkapi proses CRUD untuk books.

MINGGU 14 (TEMA)



DESKRIPSI TEMA

(Apa yang akan dipelajari mahasiswa pada minggu ini)

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

(Capaian pembelajaran pada minggu ini)

PERALATAN YANG DIGUNAKAN

1. ...
2. ...
3. dst

(+ Perlengkapan Apd/Alat Pelindung Diri Yang Harus Digunakan, Jika Ada)

LANGKAH-LANGKAH PRAKTIKUM

...

REFERENSI